WNDRVR
SUPPORT NETWORK

API REFERENCE: KERNEL CONNECTIVITY, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

API Reference: Kernel Connectivity, SR0650

20 July 2020

1. OSSLIB

Name

ossLib - O/S-independent services for vxWorks

Layer

USB

Routines

ossMalloc( ) - Master USB memory allocation routine. **ossCalloc( )** - Allocates memory initialized to zeros. **ossFree( )** - Master USB memory free routine.

Description

Implements functions defined by **ossLib.h**. See **ossLib.h** for a complete description of these functions.

Include files

ossLib.h

Name

ossMalloc( ) - Master USB memory allocation routine.

Layer

USB

Synopsis

```
void * ossMalloc
    (
    size_t numBytes
    )
```

Description

ossMalloc( ) calls the malloc routine installed in the global variable ossMallocFuncPtr. These default to **ossPartMalloc( )**, but can be changed by the users to their own defined malloc routine or to a non-partition method of malloc/free by calling **ossOldInstall( )**.

Returns

Pointer to allocated buffer or **NULL**

Errno

None

WNDRVR

See also

ossLib

Name

ossCalloc( ) - Allocates memory initialized to zeros.

Layer

USB

Synopsis

```
pVOID ossCalloc
    (
    size_t numBytes   /* size of buffer to allocate */
    )
```

Description

ossCalloc( ) uses **ossMalloc( )** to allocate a block of memory and then initializes it to zeros. Memory allocated using this function should be freed using **ossFree( )**.

Returns

Pointer to allocated buffer, or **NULL**

Errno

None

See also

ossLib

Name

ossFree( ) - Master USB memory free routine.

Layer

USB

Synopsis

```
void ossFree
    (
    pVOID bfr
    )
```

Description

ossFree( ) calls the free routine installed in the global variable ossFreeFuncPtr. This defaults to **ossPartFree( )**, but can be changed by the users to their own defined free routine or to a non-partition method of malloc/free by calling **ossOldInstall( )**.

Returns

N/A

Errno

None

See also

ossLib

2. USB2HELPER

Name

usb2Helper - USB Helper Function Set for USB GEN2 Class drivers

Layer

USB_HOST

Routines

usb2FindInterface( ) - find an interface descriptor on a given device **usb2FindEndpoint( )** - find an endpoint address on a given interface **usb2FindEndpointHelperDescr( )** - find a helper descriptor for an give endpoint **usb2DriverIsEmpty( )** - check if a driver still has a device in use **usb2ClassDeviceDelete( )** - delete class device generic structures **usb2ClassDeviceCreate( )** - create class device generic structures **usb2ClassDeviceIsValid( )** - check if the device still exists (mounted) **usb2TakeDevice( )** - take device ownership **usb2ReleaseDevice( )** - release device ownership **usb2TakeDriver( )** - take driver ownership **usb2ReleaseDriver( )** - release driver ownership **usb2VendorClassSpecific( )** - issue vendor or class specific request **usb2ClassDeviceAddCallbackSet( )** - set a callback to be called upon device add **usb2ClassDeviceDelCallbackSet( )** - set a callback to be called upon device remove **usb2DeleteClassDriver( )** - delete the class driver generic structures **usb2ClassDriverCreate( )** - create the class driver generic structures **usb2DeviceDeleteMsgPost( )** - post a message to helper task to delete a device **usb2DeviceDeleteHelperInit( )** - initialize the device delete helper **usb2DeviceDeleteHelperDeinit( )** - deinitialize the device delete helper

Description

This helper module provides a set of functionalities that would be common to many class drivers. This set of functionalities includes:

- List management: Each class driver uses a list to maintain its list of devices. By creating a USB2_CLASS_DRIVER, the list is created. Then, creating a USB2_CLASS_DEVICE adds the device to the driver's list.

- Device Validity checking: Because the IO to devices can happen at any time, it could also happen during the removal / adding phase of a device. This module provides a utility to check whether a device is in the process of being removed.

- Adequate Resource locking: A generic process to lock the device, with validation check.

- Endpoint Finding facility: Class drivers may need to search for adequate endpoints when devices are being added.

- Interface Finding facility: Class drivers may need to search for adequate interfaces when devices are being added.

- Class Specific / Vendor specific request handler: Class drivers may need to send Class Specific requests to the device (resets / status / reads from specific registers...).

- Application notification: When a USB2_CLASS_DEVICE / USB2_CLASS_DRIVER is used, the application can be notified through a callback mechanism

that a device is connected / removed. The application can decide to allow the connection or refuse it.

Data flow

A class driver can call **usb2ClassDriverCreate( )** upon initialization. This initializes the device list with a default base name which can be used by the devices (if needed). It also creates the driver Mutex that is needed to search across the device list and add/remove entries. **usb2ClassDriverDelete( )** frees all the created structures when the driver terminates.

When a device corresponding to that type is added by the USBD, the class driver add callback can use **usb2ClassDeviceCreate( )**, which will add the device to the list and compute a proper name for it (if naming is required). If application callbacks are installed, it calls a proper callback, and depending on its error code, allows/disallows the mounting of the device. **usb2ClassDeviceDelete( )** does the exact contrary (removes the device from the list and deletes all the resources).

usb2TakeDevice( ) takes ownership of the device, and searches the list to see if the device is really there. **usb2ReleaseDevice( )** does the contrary (when IO is done, for example).

usb2TakeDriver( ) takes ownership of the whole driver. **usb2ReleaseDriver( )** does the exact contrary (when the device is added to or removed from the list, for example).

usb2VendorClassSpecific( ) can be done anytime, if required (such as when the device is mounted).

usb2FindInterface( ) can be used to locate the address of the interface descriptor in the device's current configuration descriptor buffer, using the specified interface number and alternate setting number.

usb2FindEndpoint( ) can be used to find a specific endpoint descriptor in the specific interface and alternate setting, according to the endpoint type match criteria.

Include files

vxWorks.h semLib.h logLib.h usb2/usb2Helper.h

Name

usb2FindInterface( ) - find an interface descriptor on a given device

Layer

USB_HOST

Synopsis

```
USBHST_INTERFACE_DESCRIPTOR * usb2FindInterface
    (
    pUSBD_DEVICE_INFO pDeviceInfo,     /* Device Information      */
    UINT16            uInterfaceNum,   /* Interface number        */
    UINT16            uAltIndex        /* Alternate setting index */
    )
```

Description

This routine searches for an interface descriptor on a given device.

The pInterfaceDesc returned by this routine is a pointer to the desired interface descriptor, with interface number bInterfaceNumber and alternate setting uAlternateSetting, which is inside the configuration buffer. The class specification (or vendor device specification) may define some class specific (or vendor specific) descriptors immediately following the interface descriptor. For example, the HID class specification defines an HID descriptor following the interface descriptor. If there are no class specific (or vendor specific) descriptors following the interface descriptor, the endpoint descriptors for that interface immediately follow the interface descriptor; if there is any class specific (or vendor specific) descriptor, then the endpoint descriptor appears immediately after the end of the last class specific (or vendor specific) descriptor.

If you are writing your own class driver, and your class has a definition for any class specific (or vendor specific) descriptors, you can start parsing these class specific (or vendor specific) descriptors from pInterfaceDesc, adding an offset of sizeof(**USBHST_INTERFACE_DESCRIPTOR**) to reach the start of the class specific descriptors, as follows:

```
char * pClassSpecificDescrs = ((char *)pInterfaceDesc) +
sizeof(USBHST_INTERFACE_DESCRIPTOR);

USBHST_DESCRIPTOR_HEADER * pDescrHeader = (USBHST_DESCRIPTOR_HEADER *)
pClassSpecificDescrs;
```

You can then convert the buffer to the pointer type of your class specific (or vendor specific) descriptors.

Parameters

pDeviceInfo

This specifies the device to search.

uInterfaceNum

This parameter specifies the number of the interface to search for.

uAltIndex

This is an alternate index setting for the location of the interface descriptor.

Example

```
USB2_MSE_VDBG("usb2MseDeviceAdd(): Searching for endpoints... \n",
                1, 2, 3, 4, 5, 6);

        pInterfaceDesc = usb2FindInterface(pDeviceInfo,
                                           bInterfaceNumber,
                                           uAlternateSetting);
        if (NULL == pInterfaceDesc)
            {
            USB2_MSE_ERR("usb2MseDeviceAdd(): No such interface"
                         " bInterfaceNumber %d, uAlternateSetting %d on device\n",
                         (int) bInterfaceNumber, uAlternateSetting, 3, 4, 5, 6);
            return USBHST_FAILURE;
            }
```

Returns

The Interface Descriptor for this device, depending on the parameters.

Errno

N/A

See also

usb2Helper

Name

usb2FindEndpoint( ) - find an endpoint address on a given interface

Layer

USB_HOST

Synopsis

```
USBHST_ENDPOINT_DESCRIPTOR * usb2FindEndpoint
    (
    UINT8 * pInterfaceStart,      /* Ptr to interface buffer in config */
    UINT8   uTransferType,        /* Transfer type           */
    UINT8   uTransferDir,         /* Transfer direction      */
    UINT8   uSyncType,            /* Synchronisation Type (Iso Mode) */
    UINT8   uUsageType,           /* Usage Type (Iso Mode)   */
    UINT8   uMatchEndpointIndex   /* index of matching endpoint type */
    )
```

Description

This routine searches on the given interface to find the endpoint descriptor with some specified attributes.

The attributes are drawn from the USB standard endpoint descriptor fields. For example, we can specify the uTransferType parameter as USB2_ATTR_INTERRUPT because we are finding an interrupt endpoint. The uTransferDir can simply take USB2_ENDPOINT_OUT or USB2_ENDPOINT_IN. The uSyncType and uUsageType are for isochronous endpoints where further attributes like Synchronization and Usage types are required; for non-isochronous endpoints these parameters are not used.

Parameters

pInterfaceStart

This specifies the start address of the string descriptor describing this interface.

uTransferType

Specifies the endpoint's transfer type attribute.

uTransferDir

Specifies the endpoint transfer direction.

uSyncType

Specifies the synchronization type for isochronous endpoints.

uUsageType

This specifies the usage type for isochronous endpoints.

uMatchEndpointIndex

Specifies the endpoint index to match.

Example

```
      /* Find the must-have interrupt in endpoint */

      pEndPointDesc = usb2FindEndpoint((UCHAR* )pInterfaceDesc,
                                        USB2_ATTR_INTERRUPT,
                                        USB2_ENDPOINT_IN,
                                        0,
                                        0,
                                        0);
      if (NULL == pEndPointDesc)
              {
              USB2_MSE_ERR("usb2MseDeviceAdd(): NO IN endpoint found\n",
                      1, 2, 3, 4, 5, 6);
              return USBHST_FAILURE;
              }
      uInEndPointAddr = pEndPointDesc->bEndpointAddress;
```

Returns

The endpoint descriptor that has been requested.

Errno

N/A

See also

usb2Helper

Name

usb2FindEndpointHelperDescr( ) - find a helper descriptor for an give endpoint

Layer

USB_HOST

Synopsis

```
VOID * usb2FindEndpointHelperDescr
    (
    UINT8 * pEndpointStart,    /* Ptr to endpoint buffer in config */
    UINT8   uHelperDescrType   /* Helper Descriptor type           */
    )
```

Description

This routine find a helper descriptor for an give endpoint.

Parameters

pEndpointStart

This specifies the start address of the endpoint descriptor.

uHelperDescrType

Specifies the endpoint's helper descriptor type attribute. This might be any helper descriptor types such as Endpoint Companion Descritpr introduced in USB 3.0, or a Pipe Usage Descriptor specified in the USB Attached SCSI (UAS) protocol, or any other desired class/vendor defined descriptor types which futher describes the endpoint.

Example

```
/* Find the must-have UAS Pipe Usage Descriptor for an UAS endpoint */

pPipeUsageDescr = usb2FindEndpoint((UCHAR* )pEndpointDescr,
                                        USB_DESCR_UAS_PIPE_USAGE);
if (NULL == pPipeUsageDescr)
        {
        USB2_MSC_ERR("usb2MscDeviceAdd(): NO UAS Pipe Usage Descriptor found\n",
                1, 2, 3, 4, 5, 6);
        return USBHST_FAILURE;
        }
uPipeId = pPipeUsageDescr->bPipeId;
```

Returns

The endpoint helper descriptor that has been requested.

Errno

N/A

See also

usb2Helper

Name

usb2DriverIsEmpty( ) - check if a driver still has a device in use

Layer

USB_HOST

Synopsis

```
BOOL usb2DriverIsEmpty
    (
    USB2_CLASS_DRIVER * pDriver   /* Driver used to search */
    )
```

Description

This routine checks in the linked list for this driver, and returns whether there is any device still in use.

Parameters

pDriver

Specifies the driver to search.

Example

Returns

TRUE if the driver is empty (no more devices), **FALSE** if the driver still has a device in use (not completely unmounted).

Errno

N/A

See also

usb2Helper

Name

usb2ClassDeviceDelete( ) - delete class device generic structures

Layer

USB_HOST

Synopsis

```
void usb2ClassDeviceDelete
    (
    USB2_CLASS_DEVICE * pClassDevice   /* Device structure */
    )
```

Description

This routine deletes class device generic structures (list entry / removes from IOLIB).

Parameters

pClassDevice

This is a pointer to the device structure to delete.

Example

Returns

N/A

Errno

N/A

See also

usb2Helper

Name

usb2ClassDeviceCreate( ) - create class device generic structures

Layer

USB_HOST

Synopsis

```
USB2_CLASS_DEVICE * usb2ClassDeviceCreate
    (
    USB2_CLASS_DRIVER * pDriver,
    UINT32              hDevice,
    UINT8               configurationIndex,
    UINT8               bInterfaceNumber,
    UINT8               uAlternateSetting
    )
```

Description

This routine creates class device generic structures (list entry / adds to IOLIB).

The usb2ClassDeviceCreate( ) creates the USB2_CLASS_DEVICE structure. This routine not only creates the USB2_CLASS_DEVICE structure, but it also adds the structure into the device list of the class driver. It is a global list access so we need to protect against concurrent access by using driver a mutex semaphore. usb2TakeDriver( ) and usb2ReleaseDriver( ) provide ways to do the mutex semaphore take and give operations for the class driver.

Parameters

pDriver

This is a pointer to the class driver.

hDevice

This specifies the device type of the interface.

configurationIndex

This parameter specifies the index location.

bInterfaceNumber

This parameter specifies the number for this interface.

uAlternateSetting

This specifies the alternate setting for this interface.

Example

The following example is taken from Step 8 in the section "Example: Creating a Mouse Class Driver" in Chapter 3 of the Wind River USB Programmer's Guide.

```
            /* Take the driver mutex as we are going to
             * modify the device list of this driver
             */
    status = usb2TakeDriver(pUsb2MseDriver);
            if (OK != status)
                    {
                    USB2_MSE_ERR("usb2MseDeviceAdd(): Driver mutex not taken\n",
                                    1, 2, 3, 4, 5, 6);
                    return USBHST_FAILURE;
                    }
            /* Create the generic class device structure and add it to
             * the device list of this driver */

            pUsb2ClassDevice = usb2ClassDeviceCreate (pUsb2MseDriver,
                                                      hDevice,
                                                      configurationIndex,
                                                      bInterfaceNumber,
                                                      uAlternateSetting);
            if (NULL == pUsb2ClassDevice)
                    {
                    USB2_MSE_ERR("usb2MseDeviceAdd(): usb2ClassDeviceCreate failed\n",
                                    1, 2, 3, 4, 5, 6);
                    usb2ReleaseDriver(pUsb2MseDriver);

                    return USBHST_FAILURE;
                    }
```

Returns

Pointer to USB2_CLASS_DEVICE, or **NULL** if failed to create.

Errno

N/A

See also

usb2Helper

Name

usb2ClassDeviceIsValid( ) - check if the device still exists (mounted)

Layer

USB_HOST

Synopsis

```
int usb2ClassDeviceIsValid
    (
    USB2_CLASS_DRIVER * pDriver,
    USB2_CLASS_DEVICE * pClassDevice
    )
```

Description

This routine checks if a device still exists (is still mounted).

Parameters

pDriver

This is the device driver.

pClassDevice

This is the device class.

Example

Returns

The index of the device in the list of devices or **ERROR**.

Errno

N/A

See also

usb2Helper

Name

usb2TakeDevice( ) - take device ownership

Layer

USB_HOST

Synopsis

```
STATUS usb2TakeDevice
    (
    USB2_CLASS_DEVICE * pClassDevice
    )
```

Description

This routine takes ownership of a specified device.

Parameters

pClassDevice

Pointer to the class device to take ownership of.

Example

```
      {
              USB2_MSE_DEVICE * pDevice = (USB2_MSE_DEVICE * ) pDevHdr;
              USB2_FILE_DESC * pFd; /* File descriptor structure used */
              USBHST_STATUS hstStatus;

              USB2_MSE_VDBG ("usb2MseIosOpen(): Opening the Mouse : "
                              "mode %p, flags %p\n", mode, flags, 3, 4, 5, 6);
              if (OK != usb2TakeDevice (pDevice->pClassDevice))
                      {
                      USB2_MSE_ERR ("usb2MseIosOpen(): Cannot take"
                                      "device ownership\n", 1, 2, 3, 4, 5, 6);

                      errnoSet(EINVAL);

                      return ERROR;
                      }
```

Returns

OK if the ownership succeeded, **ERROR** if not, this means probably the device is beeing torn down.

Errno

N/A

See also

usb2Helper

Name

usb2ReleaseDevice( ) - release device ownership

Layer

USB_HOST

Synopsis

```
STATUS usb2ReleaseDevice
    (
    USB2_CLASS_DEVICE * pClassDevice
    )
```

Description

This routine releases ownership of a device.

Parameters

pClassDevice

Pointer to the class device for which to release ownership.

Example

```
/*
 * We don't want the device to be created and do not support
 * write to a mouse device, so refuse these flags
 */

if ((O_RDWR & flags) || (O_WRONLY & flags) || (O_CREAT & flags ))
        {
        USB2_MSE_ERR("usb2MseIosOpen():"
                "device not writable/creatable \n",
                1, 2, 3, 4, 5, 6);

usb2ReleaseDevice(pDevice->pClassDevice);

errnoSet(EINVAL);

return ERROR;
}
```

Returns

OK, or **ERROR** if there is any failure.

Errno

N/A

See also

usb2Helper

Name

usb2TakeDriver( ) - take driver ownership

Layer

USB_HOST

Synopsis

```
STATUS usb2TakeDriver
    (
    USB2_CLASS_DRIVER * pDriver
    )
```

Description

This routine takes ownership of a driver.

Parameters

pDriver

Pointer to the driver to take ownership of.

Example

```
            /* Take the driver mutex as we are going to
             * modify the device list of this driver
             */
     status = usb2TakeDriver(pUsb2MseDriver);
            if (OK != status)
                    {
                    USB2_MSE_ERR("usb2MseDeviceAdd(): Driver mutex not taken\n",
                                    1, 2, 3, 4, 5, 6);
                    return USBHST_FAILURE;
                    }
            /* Create the generic class device structure and add it to
             * the device list of this driver */

            pUsb2ClassDevice = usb2ClassDeviceCreate (pUsb2MseDriver,
                                                      hDevice,
                                                      configurationIndex,
                                                      bInterfaceNumber,
                                                      uAlternateSetting);
            if (NULL == pUsb2ClassDevice)
                    {
                    USB2_MSE_ERR("usb2MseDeviceAdd(): usb2ClassDeviceCreate failed\n",
                                    1, 2, 3, 4, 5, 6);
                    usb2ReleaseDriver(pUsb2MseDriver);
```

```
                        return USBHST_FAILURE;
                        }
```

Returns

OK if the ownership succeeded; **ERROR** if it did not succeed, which probably means that the driver is being torn down.

Errno

N/A

See also

usb2Helper

Name

usb2ReleaseDriver( ) - release driver ownership

Layer

USB_HOST

Synopsis

```
STATUS usb2ReleaseDriver
    (
    USB2_CLASS_DRIVER * pDriver
    )
```

Description

This routine releases ownership of a driver.

Parameters

pDriver

Pointer to the driver for which to release ownership.

Example

Returns

OK, or **ERROR** if there is any failure.

Errno

N/A

See also

usb2Helper

Name

usb2VendorClassSpecific( ) - issue vendor or class specific request

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usb2VendorClassSpecific
    (
    USB2_CLASS_DEVICE * pClassDevice,   /* USB2_CLASS_DEVICE structure */
    UINT8               requestType,    /* bmRequestType as per spec */
    UINT8               request,        /* request as per spec */
    UINT16              value,          /* value as per spec */
    UINT16              index,          /* index value as per spec */
    UINT16 *            pActLen,        /* n bytes supposed to send */
    pUINT8              pBfr,           /* Buffer to store / send data to */
    int                 timeoutMs       /* Timeout in ms, until transfer abort*/
    )
```

Description

This routine can be called from any class driver to send vendor or class specific requests through the default (control) pipe.

Parameters

pClassDevice

This is a pointer to the class device structure.

requestType

This parameter specifies the request type being made.

request

This specifies the request.

value

This sets the value, as specified in the USB specification.

index

This indicates the index in the structure, as specified by the USB specification.

pActLen

This sets the number of bytes of data to be sent.

pBfr

This specifies the buffer used to store and send data.

timeoutMs

This specifies the timeout, in milliseconds, until a transfer is aborted.

Example

Returns

USBHST_SUCCESS or other **USBHST_STATUS** error code if failed.

Errno

N/A

See also

usb2Helper

Name

usb2ClassDeviceAddCallbackSet( ) - set a callback to be called upon device add

Layer

USB_HOST

Synopsis

```
void usb2ClassDeviceAddCallbackSet
    (
    FUNCPTR callback
    )
```

Description

This routine sets a callback given in parameter callback that is called when a device is added, if the class driver manages the drivers through the API described above.

Parameters

None

Example

Returns

N/A

Errno

N/A

See also

usb2Helper

Name

usb2ClassDeviceDelCallbackSet( ) - set a callback to be called upon device remove

Layer

USB_HOST

Synopsis

```
void usb2ClassDeviceDelCallbackSet
    (
    FUNCPTR callback
    )
```

Description

This routine sets a callback given in parameter callback that is called when a device is removed, if the class driver manages the drivers through the API described above.

Parameters

None

Example

Returns

N/A

Errno

N/A

See also

usb2Helper

Name

usb2DeleteClassDriver( ) - delete the class driver generic structures

Layer

USB_HOST

Synopsis

```
void usb2ClassDriverDelete
    (
    USB2_CLASS_DRIVER * pDriver
    )
```

Description

This routine deletes a driver and its associated list, mutex, and buffers.

Parameters

None

Example

Returns

N/A

Errno

N/A

See also

usb2Helper

Name

usb2ClassDriverCreate( ) - create the class driver generic structures

Layer

USB_HOST

Synopsis

```
USB2_CLASS_DRIVER * usb2ClassDriverCreate
    (
    char * pDriverDefaultName   /* Default driver iolib base name for driver */
    )
```

Description

This routine can be called upon class driver initialization. It initializes the device list with a default base name which can be used by the devices (if needed). It also creates the driver mutex that is needed to search accross the device list and add/remove entries.

This routine accepts one parameter, a string containing the default base name that will be used by each device instance. By default the routine **usb2ClassDeviceCreate( )** will then (if called) add the suffix **index** : "name"**index**. Index will be a decimal number computed from the linked list which describes the number of connected devices of the same type.

For example, the printer driver provides "/usb2Prn/" as a default name. Anytime a printer is added, it will appear as "/usb2Prn/0" in the **IoLib**. If another printer is added, "/usb2Prn/1" will exist. If the first printer is removed, and another one is added, again "/usb2Prn/0" will be mounted.

As this driver implementation works with a linked list of devices, there is no limitation on the number of devices mountable (apart from the size of physical memory). If you want to set a limit, please use the callback mechanism.

This parameter can also be set to **NULL**, so no names will be computed, since some device instances do not need names.

Parameters

pDriverDefaultName

This is the default driver IO library base name for the driver.

Example

Returns

Pointer to USB2_CLASS_DRIVER if the driver gets created, **NULL** if it failed.

Errno

N/A

See also

usb2Helper

Name

usb2DeviceDeleteMsgPost( ) - post a message to helper task to delete a device

Layer

USB_HOST

Synopsis

```
STATUS usb2DeviceDeleteMsgPost
    (
    USB2_CLASS_DEVICE * pClassDevice,   /* Usb2 Device structure */
    FUNCPTR             pCallback,      /* Delete Callback function */
    void *              pDevice         /* Device to delete */
    )
```

Description

This routine tells the device delete helper task that there is a device driver that wants to delete device structures. The helper task will call the device driver delete callback, that returns **ERROR** when there is still an active transfer. It returns **OK** when all is deleted.

As long as the callback returns **ERROR**, the helper tries at regular attempts to call that callback. Make sure that the device mutex is still taken (with usb2TakeDevice) when the class device exits the remove function in order that no other function can take it any time.

Parameters

pClassDevice

This is a pointer to the class device structure.

pCallback

This specifies the Delete Callback function.

pDevice

This parameter specifies the device to delete.

Example

Returns

OK if information has been passed to the helper task, **ERROR** if not.

Errno

N/A

See also

usb2Helper

Name

usb2DeviceDeleteHelperInit( ) - initialize the device delete helper

Layer

USB_HOST

Synopsis

```
STATUS usb2DeviceDeleteHelperInit
    (
    int maxParallelDevices,     /* Maximum amount of devices in parallel handle  */
    int helperTaskPrio,         /* Delete helper Task priority                   */
    int helperTaskLatencyMSec   /* Latency (ms) of task if one delete  is not OK */
    )
```

Description

The Device Delete Helper is used when ongoing transfers are still not yet terminated by the stack, so we must wait until all the transfers are completed when we delete a device.

It creates a task that is pending on a queue. When a device driver wants to delete structures, it needs to call usb2DeviceDeleteHelper that feeds that task. The task attempts at regular intervals to delete the driver structures. As long as a transfer is active the helper tries to delete the structures, until the driver gives the **OK**.

Parameters

maxParallelDevices

This specifies the maximum number of parallel devices in a handle.

helperTaskPrio

This sets the priority level of the Device Delete Helper task.

helperTaskLatencyMSec

This specifies the latency of the task if one deletion attempt is not sufficient.

Example

Returns

OK, or **ERROR** if there is any failure.

Errno

N/A

See also

usb2Helper

Name

usb2DeviceDeleteHelperDeinit( ) - deinitialize the device delete helper

Layer

USB_HOST

Synopsis

```
void usb2DeviceDeleteHelperDeInit(void);
```

Description

This routine deletes the Device Delete Helper service task. The Device Delete Helper is used when ongoing transfers are still not terminated by the stack, so we must wait until all the transfers are completed when we delete a device.

Parameters

None

Example

Returns

N/A

Errno

N/A

See also

usb2Helper

3. USB2HID

Name

usb2Hid - USB HID class specific functions

Layer

USB_HOST

Routines

usb2HidDescriptorGet( ) - find a HID descriptor on a given interface **usb2HidReportDescriptorGet( )** - issue a get report descriptor request for HID device **usb2HidIdleTimeSet( )** - set the Idle time of the HID device **usb2HidIdleTimeGet( )** - get the Idle time of the HID device **usb2HidProtoSet( )** - set the protocol for HID device **usb2HidProtoGet( )** - get the protocol for HID device **usb2HidReportSet( )** - issue a set report request for HID device **usb2HidReportGet( )** - issue a get report request for HID device **usb2HidCalculateIdleTimeFromMs( )** - calculate the time in hid format **usb2HIDReportDescDump( )** - dump HID report descriptor

Description

This module provides some HID specific helper routines.

Include files

vxWorks.h usb2Hid.h usb2Helper.h

Name

usb2HidDescriptorGet( ) - find a HID descriptor on a given interface

Layer

USB_HOST

Synopsis

```
USBHST_HID_DESCRIPTOR * usb2HidDescriptorGet
    (
    pUSBD_DEVICE_INFO pDeviceInfo,     /* Device Information      */
    UINT16            uInterfaceNum,   /* Interface number        */
    UINT16            uAltIndex        /* Alternate setting index */
    )
```

Description

This routine finds a HID descriptor on a given interface

Returns

the HID descriptor that has been requested

Errno

N/A

See also

usb2Hid

Name

usb2HidReportDescriptorGet( ) - issue a get report descriptor request for HID device

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usb2HidReportDescriptorGet
    (
    USB2_CLASS_DEVICE * pDevice,      /* usb2 class device */
    UINT16              reportType,   /* report ID AND TYPE */
    pUINT8              pReport,      /* report value */
    UINT16*             pActLen,      /* length of report */
    int                 timeout       /* time in ms before timeout error */
    )
```

Description

This routine issues a get report descriptor request for HID device.

Returns

USBHST_SUCCESS or any other error value

Errno

N/A

See also

usb2Hid

Name

usb2HidIdleTimeSet( ) - set the Idle time of the HID device

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usb2HidIdleTimeSet
    (
    USB2_CLASS_DEVICE * pDevice,    /* usb2 class device */
    UINT16              idleTime,   /* idle time as of HID spec */
    int                 timeout     /* time to wait in ms before timeout */
    )
```

Description

This routine sets the idle time of the HID device.

Returns

USBHST_SUCCESS or any other error value

Errno

N/A

See also

usb2Hid

Name

usb2HidIdleTimeGet( ) - get the Idle time of the HID device

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usb2HidIdleTimeGet
    (
    USB2_CLASS_DEVICE * pDevice,     /* usb2 class device */
    UINT8 *             pIdleTime,   /* ptr to idle time returned */
    int                 timeout      /* time to wait in ms before timeout */
    )
```

Description

This routine sets the idle time of the HID device.

Returns

USBHST_SUCCESS or any other error value

Errno

N/A

See also

usb2Hid

Name

usb2HidProtoSet( ) - set the protocol for HID device

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usb2HidProtoSet
    (
    USB2_CLASS_DEVICE * pDevice,   /* usb2 class device */
    UCHAR               value,     /* protocol value as of HID spec */
    int                 timeout    /* time in ms before timeout error */
    )
```

Description

This routine sets the protocol for the HID device to boot or report protocol.

Returns

USBHST_SUCCESS or any other error value

Errno

N/A

See also

usb2Hid

Name

usb2HidProtoGet( ) - get the protocol for HID device

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usb2HidProtoGet
    (
    USB2_CLASS_DEVICE * pDevice,       /* usb2 class device */
    UINT8 *             pProtoValue,   /* pointer to protocol value */
    int                 timeout        /* time in ms before timeout error */
    )
```

Description

This routine gets the protocol for the HID device to boot or report protocol.

Returns

USBHST_SUCCESS or any other error value

Errno

N/A

See also

usb2Hid

Name

usb2HidReportSet( ) - issue a set report request for HID device

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usb2HidReportSet
    (
    USB2_CLASS_DEVICE * pDevice,      /* usb2 class device */
    UINT16              reportType,   /* report TYPE AND ID */
    pUINT8              pReport,      /* report value */
    UINT16 *            pActLen,      /* length of report */
    int                 timeout       /* time in ms before timeout error */
    )
```

Description

This routine issues a set report request for HID device.

Returns

USBHST_SUCCESS or any other error value

Errno

N/A

See also

usb2Hid

Name

usb2HidReportGet( ) - issue a get report request for HID device

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usb2HidReportGet
    (
    USB2_CLASS_DEVICE * pDevice,      /* usb2 class device */
    UINT16              reportType,   /* report ID AND TYPE */
    pUINT8              pReport,      /* report value */
    UINT16*             pActLen,      /* length of report */
    int                 timeout       /* time in ms before timeout error */
    )
```

Description

This routine issues a get report request for HID device.

Returns

USBHST_SUCCESS or any other error value

Errno

N/A

See also

usb2Hid

Name

usb2HidCalculateIdleTimeFromMs( ) - calculate the time in hid format

Layer

USB_HOST

Synopsis

```
UINT16 usb2HidCalculateIdleTimeFromMs
    (
    UINT32 idleTimeMs
    )
```

Description

This routine calculates the time in HID format (as specified by the spec) from a value in ms.

Returns

USB2_HID_IDLE_MIN, USB2_HID_IDLE_INFINITE, or a converted value

Errno

N/A

See also

usb2Hid

Name

usb2HIDReportDescDump( ) - dump HID report descriptor

Layer

USB_HOST

Synopsis

```
STATUS usb2HIDReportDescDump
    (
    unsigned char * pBuffer,
    unsigned int    size
    )
```

Description

This routine dump HID report descriptor.

Returns

OK or **ERROR**

Errno

Not Available

See also

usb2Hid

4. USBD

Name

usbd - USBD Routines

Layer

USB_HOST

Routines

usbdMatchDriverId( ) - match device information from device information list **usbHstDriverRegister( )** - register class driver **usbHstDriverDeregister( )** - deregister USB class driver **usbHstHCDRegister( )** - register Host Controller Driver with USBD **usbHstHCDDeregister( )** - deregister a Host Controller Driver **usbHstBusRegister( )** - register an USB Bus **usbHstBusDeregister( )** - deregister a USB Bus **usbHstCtlrAdd( )** - add an USB Host Controller instance **usbHstCtlrRemove( )** - remove an USB Host Controller instance **usbHstBusAdd( )** - add an USB Bus **usbHstBusRemove( )** - remove a USB Bus **usbMsgCallbackAdd( )** - add a callback to specific message code **usbMsgCallbackDel( )** - delete a callback from callback list by a handle value **usbMsgPost( )** - post a message to callback functions

Description

This file initializes the global variables for USB2.0/3.0 USBD module and registers itself with the class drivers and host controller driver modules.

Host Controller Driver Registration with USBD - The host controller driver module register themselves with USBD by calling the routine usbHstHCDRegister (). The host controller driver is also registered with the vxBus as a bus type specifying appropriate busID. Subsequent to this routine, the host controller driver calls the routine usbHstBusRegister () for every host controller device of the particular HCD. This routine, registers a bus for every host controller device.

Class Driver Registration with USBD - The Class Drivers register with the USBD by calling the routine usbHstDriverRegister (). In this routine, the structure **DRIVER_REGISTRATION** is populated and the class driver is in turn registered with vxBus.

Device Connection - On a new device notifcation, the USBD module will call vxbDeviceAnnounce () to announce the new device. Subsquently vxBus will call usbdDriverFind () routine to look for a matching driver for the device. If the matching driver is found, its corresponding routine is called to configure the device.

Include files

usbd.h usbOtg.h drv/erf/erfLib.h vxbUsbd.h usbdevice.h urb.h

Name

usbdMatchDriverId( ) - match device information from device information list

Layer

USB_HOST

Synopsis

```
pUSBHST_DEVICE_ID usbdMatchDriverId
    (
    pUSBHST_DEVICE_ID pDevId,
```

```
    pUSBHST_DEVICE_ID pDrvDevId
    )
```

Description

This routine matches device information from device information list by new matching rules, return matched pointer if there is a match node.

This function can be used for any scene for matching device id from a USB link list.

Returns

Match node pointer or **NULL** for no match.

Errno

N/A

See also

usbd

Name

usbHstDriverRegister( ) - register class driver

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usbHstDriverRegister
    (
    pUSBHST_DEVICE_DRIVER   pDeviceDriverInfo,   /* Ptr to Device Driver info  */
    void                 ** pContext,            /* Ptr to context information */
    char                  * pDrvName             /* name of the driver */
    )
```

Description

This routine registers the class driver with the USB Host Stack. The function also registers the class driver with vxBus.

Returns

USBHST_INVALID_PARAMETER, **USBHST_SUCCESS**, **USBHST_FAILURE** if Driver is already registered

Errno

N/A

See also

usbd

Name

usbHstDriverDeregister( ) - deregister USB class driver

Layer

USB_HOST

Synopsis

```
USBHST_STATUS  usbHstDriverDeregister
    (
    pUSBHST_DEVICE_DRIVER pDeviceDriverInfo   /* Ptr to Device Driver info */
    )
```

Description

This routine deregisters the class driver with the USB Stack.

Returns

USBHST_INVALID_PARAMETER, **USBHST_SUCCESS**, **USBHST_FAILURE** if Driver is not found or if it is a hub class driver and there are some functional devices present

Errno

N/A

See also

usbd

Name

usbHstHCDRegister( ) - register Host Controller Driver with USBD

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usbHstHCDRegister
    (
    pUSBHST_HC_DRIVER pHCDriver,     /* Ptr to Host Controller driver */
    UINT32            *phHCDriver,   /* Ptr to Host Controller handle */
    void *            pContext,      /* pContext information */
    UINT32            busID          /* bus Id */
    )
```

Description

This routine registers a Host Controller Driver with the USB Stack.

Returns

USBHST_INVALID_PARAMETER, **USBHST_INSUFFICIENT_RESOURCE**, **USBHST_SUCCESS** if Host Controller Driver is registered successfully

Errno

N/A

See also

usbd

Name

usbHstHCDDeregister( ) - deregister a Host Controller Driver

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usbHstHCDDeregister
    (
    UINT32 hHCDriver   /* Host Controller Driver handle */
    )
```

Description

This routine deregisters a Host Controller Driver with the USB Stack.

Returns

USBHST_SUCCESS, **USBHST_INVALID_PARAMETER**, **USBHST_FAILURE** if bus count is not zero

Errno

N/A

See also

usbd

Name

usbHstBusRegister( ) - register an USB Bus

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usbHstBusRegister
    (
    UINT32        hHCDriver,      /* Host Controller Driver handle */
    UINT8         uSpeed,         /* USB Bus speed                 */
    ULONG         hDefaultPipe,   /* Default pipe handle           */
    VXB_DEVICE_ID pDev            /* struct vxbDev  */
    )
```

Description

This routine registers an USB Bus corresponding to the host controller. This routine also announces the host controller device to vxBus

Returns

USBHST_SUCCESS, **USBHST_INVALID_PARAMETER**, **USBHST_INSUFFICIENT_RESOURCE**, **USBHST_FAILURE** if USB Bus is already registered

Errno

None

See also

usbd

Name

usbHstBusDeregister( ) - deregister a USB Bus

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usbHstBusDeregister
    (
    UINT32 hHCDriver,           /* Host Controller Driver handle */
    UINT32 uRelativeBusIndex,   /* Bus index being deregistered  */
    ULONG  hDefaultPipe         /* Default pipe for USB bus      */
    )
```

Description

This routine deregisters an USB Bus corresponding to the controller.

Returns

USBHST_SUCCESS, **USBHST_INVALID_PARAMETER**, **USBHST_INSUFFICIENT_RESOURCE**, **USBHST_FAILURE** when Attempt to deregister the USB Bus while there are functional devices on it

Errno

None

See also

usbd

Name

usbHstCtlrAdd( ) - add an USB Host Controller instance

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usbHstCtlrAdd
    (
    UINT32        hHCDriver,   /* Host Controller Driver handle */
    VXB_DEVICE_ID pDev         /* struct vxbDev for Host Ctlr   */
    )
```

Description

This routine adds an USB Host Controller instance.

Returns

USBHST_SUCCESS, **USBHST_INVALID_PARAMETER**, **USBHST_INSUFFICIENT_RESOURCE**, or **USBHST_FAILURE**.

Errno

N/A

See also

usbd

Name

usbHstCtlrRemove( ) - remove an USB Host Controller instance

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usbHstCtlrRemove
    (
    UINT32        hHCDriver,   /* Host Controller Driver handle */
    VXB_DEVICE_ID pDev         /* struct vxbDev for Host Ctlr   */
    )
```

Description

This routine removes an USB Host Controller instance.

Returns

USBHST_SUCCESS, **USBHST_INVALID_PARAMETER**, or **USBHST_FAILURE**.

Errno

N/A

See also

usbd

Name

usbHstBusAdd( ) - add an USB Bus

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usbHstBusAdd
    (
    UINT32        hHCDriver,     /* Host Controller Driver handle */
    VXB_DEVICE_ID pDev,          /* struct vxbDev for host ctrlr  */
    UINT8         uBusSpeed,     /* USB Bus speed                 */
    ULONG         hDefaultPipe   /* Default pipe handle           */
    )
```

Description

This routine adds an USB Bus corresponding to the host controller bus instance.

Returns

USBHST_SUCCESS, **USBHST_INVALID_PARAMETER**, **USBHST_INSUFFICIENT_RESOURCE**, or **USBHST_FAILURE**

Errno

N/A

See also

usbd

Name

usbHstBusRemove( ) - remove a USB Bus

Layer

USB_HOST

Synopsis

```
USBHST_STATUS usbHstBusRemove
    (
    UINT32        hHCDriver,     /* Host Controller Driver handle */
    VXB_DEVICE_ID pDev,          /* struct vxbDev for host ctlr   */
    UINT8         uBusSpeed,     /* USB Bus speed                 */
    ULONG         hDefaultPipe   /* Default pipe handle           */
    )
```

Description

This routine removes an USB Bus corresponding to the controller.

Returns

USBHST_SUCCESS, **USBHST_INVALID_PARAMETER**, or **USBHST_FAILURE**.

Errno

N/A

See also

usbd

Name

usbMsgCallbackAdd( ) - add a callback to specific message code

Layer

USB_HOST

Synopsis

```
void * usbMsgCallbackAdd
    (
    UINT32          msgCode,
    USBMSG_CALLBACK callback,
    void *          userData,
```

```
    UINT32          flags
    )
```

Description

This routine is used for adding a callback function to a specific message code. msgCode points out which message want to be match. userData will be send to callback function when the specific message is posted. flags is used for some extend settings. Currently, flags can indicate the callback is a asynchronous call or a synchronous call.

Do not add a callback in a callback synchronous callback routine, it may cause deadlock or other unknown issue.

Returns

a non-null handle with type of (void *) if success or **NULL** when error occurred

Errno

N/A

See also

usbd

Name

usbMsgCallbackDel( ) - delete a callback from callback list by a handle value

Layer

USB_HOST

Synopsis

```
STATUS usbMsgCallbackDel
    (
    void * handle
    )
```

Description

This routine is used for deleting a callback from callback list by a handle value.

Do not delete a callback in a callback synchronous callback routine, it may cause deadlock or other unknown issue.

Returns

OK, delete success **ERROR**, some error occur

Errno

N/A

See also

usbd

Name

usbMsgPost( ) - post a message to callback functions

Layer

USB_HOST

Synopsis

```
void * usbMsgPost
    (
    UINT32 msgCode,
    void * pModule,
    void * param
    )
```

Description

This routine posts a message to callback functions. It will check all callback functions and do callback if message code is matched.

For asynchronous callback, it uses ERF task. For synchronous callback, it will directly do callback.

Returns

NULL, always

Errno

N/A

See also

usbd

5. USBDESCRCOPYLIB

Name

usbDescrCopyLib - USB descriptor copy utility functions

Layer

USB

Routines

No user-callable routines

Description

This module contains miscellaneous functions which may be used by the USB driver (USBD), USB HCD (USB Host Controller Driver), USB HCD (USB Target Controller Driver) or by USBD clients.

Include files

usbDescrCopyLib.h

6. USBFDT

Name

usbFdt - USB FDT lib

Layer

USB

Routines

usbFdtBusResAlloc( ) - allocate resource for child device **usbFdtBusResFree( )** - free the resource for child device
usbFdtResListGet( ) - get the resource list of specific device

Description

This module implements FDT interface for USB, FDT type USB device driver could use this library to handle properties.

Include files

vxWorks.h vxbLib.h vxbFdtLib.h usbFdt.h

Name

usbFdtBusResAlloc( ) - allocate resource for child device

Layer

USB

Synopsis

```
VXB_RESOURCE * usbFdtBusResAlloc
    (
    VXB_DEV_ID pDev,      /* USB glue bus device */
    VXB_DEV_ID pChild,    /* USB child device to allocate a resource for */
    UINT32     id         /* ID of the resource to find */
    )
```

Description

This routine allocate resource for child device and go corresponding operation based on its type.

Returns

a pointer to a VXB resource if allocate is successful, otherwise **NULL**

Errno

N/A

See also

usbFdt

Name

usbFdtBusResFree( ) - free the resource for child device

Layer

USB

Synopsis

```
STATUS usbFdtBusResFree
    (
    VXB_DEV_ID     pDev,     /* USB glue bus device */
    VXB_DEV_ID     pChild,   /* USB child device */
    VXB_RESOURCE * pRes      /* resource to be freed */
    )
```

Description

This routine free the resource allocated by calling simpleBusResAlloc

Returns

OK if free successfully, otherwise **ERROR**

Errno

N/A

See also

usbFdt

Name

usbFdtResListGet( ) - get the resource list of specific device

Layer

USB

Synopsis

```
VXB_RESOURCE_LIST * usbFdtBusResListGet
    (
    VXB_DEV_ID pDev,    /* USB glue bus device */
    VXB_DEV_ID pChild   /* USB child device */
    )
```

Description

This routine get the resource list of specific device

Returns

resource list pointer, or **NULL** for error conditions.

Errno

N/A

See also

usbFdt

7. USBJOBQ

Name

usbJobQ - USB job Queue library

Layer

USB

Routines

usbJobQTestJob( ) - the job to for test.

Description

This file provides USB job queue for USB modules.

Include files

usbJobQ.h

Name

usbJobQTestJob( ) - the job to for test.

Layer

USB

Synopsis

```
void usbJobQTestJob
    (
    void * arg
    )
```

Description

none

Returns

N/A

Errno

N/A

See also

usbJobQ

8. USBLIB

Name

usbLib - USB utility functions

Layer

USB

Routines

usbDescrParseSkip( ) - search for a descriptor and increment buffer. **usbDescrParse( )** - search a buffer for the a particular USB descriptor

Description

This module contains miscellaneous functions which may be used by the USB driver (USBD), USB HCD (USB Host Controller Driver), or by USBD clients.

Include files

usbLib.h

Name

usbDescrParseSkip( ) - search for a descriptor and increment buffer.

Layer

USB

Synopsis

```
pVOID usbDescrParseSkip
    (
    pUINT8  *ppBfr,          /* buffer to parse */
    pUINT16 pBfrLen,         /* length of buffer to parse */
    UINT8   descriptorType   /* type of descriptor being sought */
    )
```

Description

This searches ppBfr up to pBfrLen bytes for a descriptor of a type matching descriptorType and returns a pointer to the descriptor if found. ppBfr and pBfrLen are updated to reflect the next location in the buffer and the remaining size of the buffer, respectively.

Returns

pointer to indicated descriptor, or **NULL** if descr not found.

Errno

None

See also

usbLib

Name

usbDescrParse( ) - search a buffer for the a particular USB descriptor

Layer

USB

Synopsis

```
pVOID usbDescrParse
    (
    pUINT8 pBfr,            /* buffer to parse */
    UINT16 bfrLen,          /* length of buffer to parse */
    UINT8  descriptorType   /* type of descriptor being sought */
    )
```

Description

Searches pBfr up to bfrLen bytes for a descriptor of a type matching descriptorType and returns a pointer to the descriptor if found.

Returns

pointer to indicated descriptor, or **NULL** if descr not found

Errno

None

See also

usbLib

9. USBLISTLIB

Name

usbListLib - Linked list utility functions

Layer

USB

Routines

usbListLink( ) - Adds an element to a linked list. **usbListLinkProt( )** - Adds an element to a list guarded by a mutex. **usbListUnlink( )** - Removes an entry from a linked list. **usbListUnlinkProt( )** - Removes an element from a list guarged by a mutex. **usbListFirst( )** - Returns first entry on a linked list. **usbListNext( )** - Retrieves the next pStruct in a linked list.

Description

This file inmplements a set of general-purpose linked-list functions which are portable across operating systems. Linked lists are collections of link structures. Each link structure contains forward and backward list pointers and a pStruct field which typically points to the caller's structure that contains the link structure.

usbListLink( ) and **usbListUnlink( )** are used to add and remove link structures in a linked list. The link field may be placed anywhere in the client's structure. The client's structure may even contain more than one link field, allowing the structure to be linked to multiple lists simultaneously.

usbListFirst( ) retrieves the first structure on a linked list and **usbListNext( )** retrieves subsequent structures.

Include files

usbListLib.h

Name

usbListLink( ) - Adds an element to a linked list.

Layer

USB

Synopsis

```
VOID usbListLink
    (
    pLIST_HEAD pHead,      /* list head */
    pVOID      pStruct,    /* ptr to base of structure to be linked */
    pUSB_LINK  pLink,      /* ptr to LINK structure to be linked */
    UINT16     flag        /* indicates LINK_HEAD or LINK_TAIL */
    )
```

Description

Using the link structure pLink, add pStruct to a list of which the list head is pHead. flag must be **LINK_HEAD** or **LINK_TAIL**.

Returns

N/A

Errno

None

See also

usbListLib

Name

usbListLinkProt( ) - Adds an element to a list guarded by a mutex.

Layer

USB

Synopsis

```
VOID usbListLinkProt
    (
    pLIST_HEAD   pHead,     /* list head */
    pVOID        pStruct,   /* ptr to base of structure to be linked */
    pUSB_LINK    pLink,     /* ptr to LINK structure to be linked */
    UINT16       flag,      /* indicates LINK_HEAD or LINK_TAIL */
    MUTEX_HANDLE mutex      /* list guard mutex */
    )
```

Description

This routine is similar to **linkList( )** except that it will take the mutex before manipulating the list.

Note

The routine will block forever if the mutex does not become available.

Returns

N/A

Errno

None

See also

usbListLib

Name

usbListUnlink( ) - Removes an entry from a linked list.

Layer

USB

Synopsis

```
VOID usbListUnlink
    (
    pUSB_LINK pLink   /* LINK structure to be unlinked */
    )
```

Description

Removes pLink from a linked list.

Returns

N/A

Errno

None

See also

usbListLib

Name

usbListUnlinkProt( ) - Removes an element from a list guarged by a mutex.

Layer

USB

Synopsis

```
VOID usbListUnlinkProt
    (
    pUSB_LINK    pLink,   /* LINK structure to be unlinked */
    MUTEX_HANDLE mutex    /* list guard mutex */
    )
```

Description

This routine is the same as **usbListUnlink( )** except that it will take the mutex before manipulating the list.

Note

The function will block forever if the mutex does not become available.

Returns

N/A

Errno

None

See also

usbListLib

Name

usbListFirst( ) - Returns first entry on a linked list.

Layer

USB

Synopsis

```
pVOID usbListFirst
    (
    pLIST_HEAD pListHead   /* head of linked list */
    )
```

Description

This routine returns the pointer to the first structure in a linked list given a pointer to **LIST_HEAD**.

Returns

pStruct of first structure on list or **NULL** if list empty

Errno

None

See also

usbListLib

Name

usbListNext( ) - Retrieves the next pStruct in a linked list.

Layer

USB

Synopsis

```
pVOID usbListNext
    (
    pUSB_LINK pLink   /* LINK structure */
    )
```

Description

This routine returns the pointer to the next structure in a linked list given a pLink pointer. The value returned is the pStruct of the element in the linked list which follows the current pLink, not a pointer to the following pLink. (Typically, a client is more interested in walking its own list of structures than in the link structures used to maintain the linked list.

Returns

pStruct of next structure in list or **NULL** if end of list.

Errno

None

See also

usbListLib

10. USBOTG

Name

usbOtg - USB OTG Framework with OTG State Machine and API routines

Layer

USB_OTG

Routines

usrUsbOtgVbusStateNameGet( ) - convert OTG VBUS state into printable name string **usrUsbOtgStateNameGet( )** - convert OTG state into printable name string **usrUsbOtgTimerNameGet( )** - convert the OTG state into printable name string **usrUsbOtgOpen( )** - open an OTG controller **usrUsbOtgClose( )** - close an OTG controller **usrUsbOtgStateGet( )** - get OTG state of an OTG controller **usrUsbOtgVbusStateGet( )** - get VBUS state of an OTG controller **usrUsbOtgDeviceTypeGet( )** - get device type of an OTG controller **usrUsbOtgStackModeGet( )** - get stack mode of an OTG controller **usrUsbOtgHostRequest( )** - request to work in host role for an USB OTG controller **usrUsbOtgHostGiveup( )** - request to giveup host role for an USB OTG controller **usrUsbOtgVbusPowerOn( )** - request to power on VBUS for an OTG controller **usrUsbOtgVbusPowerOff( )** - request to power off VBUS for an OTG controller **usrUsbOtgCallbackInstall( )** - install user message callback for an OTG controller **usbOtgShow( )** - show the internal information of the OTG Manager

Description

This file is the VxWorks USB OTG Framework, with OTG State Machine and Core API routines.

The following documentation is provided as a reference for developers to understand the details of the OTG stack. Controller driver developers are encouraged to understand the component model and the driver initialization. Application developers may just need to understand the user level APIs. Further implementation details are also provided here but they are only for stack maintainers.

1 vxworks usb otg component model

1.1 vxworks usb otg stack overview

This documentation will use the following concepts;

- an USB Controller could be an integration of Host Controller, Target Controller, and OTG Controller; the combination can make OTG Dual-Role capable device, or SRP-capable peripheral-only B-device, or SRP-capable Host-only A-device, or simply normal single role Host or Target (we call then Pure Host or Pure Target).

- an OTG Controller Driver (OCD) implements **USBOTG_OCD** interfaces.

- a Host Controller Driver (HCD) implements **USBHST_HC_DRIVER** interfaces.

- a Target Controller Driver (TCD) implements **USBTGT_TCD** interfaces.

- an OTG Manager is an instance of **USBOTG_MANAGER** which forms OTG Framework.

The chart below is a simplified architecture of the VxWorks USB OTG Stack with OTG Manager as the core control unit.

```
                        VxWorks USB OTG Stack Overview
  |-----------------------------------------------------------------|
  |                          User Interfaces                        |
  |       (VxWorks Middleware, e.g, FS, NWK, WIND ML, or iosLib)    |
```


WNDRVR

```
  |-------------@------------------@-------------------@-------------|
                |                  |                   |
  |-------------@-----------\      |      /------------@-------------|
  | Host Class Drivers      |      |      | Target Function Drivers  |
  |---^------\              |      |      |             /-------@----|
      |RPT   | USB Ethernet |      |      | USB RNDIS   |       |
  |---@---|  | HID          |      |      | USB Serial  |  |----@----|
  |  Bus  |  | Mass Storage |      |      | Mass Storage|  |Composite|
  |Monitor|  | ...          |      |      | ...         |  |Framework|
  |---@---|  |----@---------/      |      \----------@--|  |---@-----|
      |           |                |                 |         |
      |           |      |---------@----------|      |         |
      |           |   RPT|                    |RPT   |         |
      |           |   -->|    OTG Manager     |<--   |         |
  |---@-----------@--/   |   (State Machine   |   \--@---------@-----|
  |     Host USBD    |   |    Stack Switch)   |   |   Target TML     |
  |                  |   \------^------@------/   |                  |
  |---^------@-------/          |      |          \-------^------@---|
     R|      |C                R|      |C                R|      |C
     P|      |T                P|      |T                P|      |T
     T|      |L                T|      |L                T|      |L
  |---@------v-----|            |      |             |----@------v---|
  |   Host Ctlr    |LD/ULD |----@------v----| LD/ULD |   Target Ctlr |
  |     Driver     |<------|                |------->|    Driver     |
  |--------@-----^-|EN/DIS |    OTG Ctlr    | EN/DIS |-^-----@-------|
           |  RPT|         |     Driver     |          |RPT  |
           |  |--@---------/                 \-------@----|  |
           |  |         Common Interrupt Handling         |  |
           |  |---------------------@---------------------|  |
           |                        |                        |
  |--------@------------------------@------------------------@-------|
  |                        USB OTG Dual-Role Ctlr                     |
  |            (Composed by Host, Target, and OTG Controller)         |
  |--------------------------------------------------------------------|

RPT = Report
CTL = Control
EN  = Enable
DIS = Disable
LD  = Load
ULD = Unload
```

1.2 vxworks usb otg driver components

When creating drivers for an OTG Dual-Role capable device, the developer needs to create the following:

- one OTG Controller Driver (OCD)

- one Host Controller Driver (HCD)

- one Target Controller Driver (TCD)

When creating drivers for a SRP-capable peripheral-only B-device, the developer needs to create the following:

- one OTG Controller Driver (OCD)

- one Target Controller Driver (TCD)

When creating drivers for a SRP-capable Host-only A-device, the developer needs to create the following:

- one OTG Controller Driver (OCD)

- one Host Controller Driver (HCD)

The following description uses the MUSBMHDRC as an example to show the required settings.

1.3 vxworks usb stack cdf file

The VxWorks CDF file will need to have one component created for each of the Controller Drivers. For example, the MUSBMHDRC (such as one in TI OMAP3530) is an OTG Dual-Role capable USB Controller, it will have following components:

- **INCLUDE_USB_MHDRC_HCD_INIT**

- **INCLUDE_USB_MHDRC_TCD_INIT**

- **INCLUDE_USB_MHDRC_OCD_INIT**

Besides the dependency on both **INCLUDE_USB_MHDRC_HCD_INIT** and **INCLUDE_USB_MHDRC_TCD_INIT**, the **INCLUDE_USB_MHDRC_OCD_INIT** will have dependency on the OTG Framework component which is implemented by the **INCLUDE_USB_OTG** component:

- **INCLUDE_USB_OTG**

The user could select either **INCLUDE_USB_MHDRC_HCD_INIT** or **INCLUDE_USB_MHDRC_TCD_INIT** to create Non-OTG usage model (either Host or Target). However, if the user wants to use it as OTG model, then **INCLUDE_USB_MHDRC_OCD_INIT** must be selected, and that will pull in all supporting components.

Note

There are five usage models that can be supported by the above components using an OTG Dual-Role capable USB controller.

- Pure Host
- Pure Target
- SRP capable Host-only A-device
- SRP capable Peripheral-only B-device
- OTG Dual-Role capable device

To aid users to select the desired usage models, some initialization components can be implemented.

- **INCLUDE_USB_MHDRC_HCD_INIT**

Initializes HCD and registers the driver with VxBus; when this component is selected, it will only pull in the HCD and exclude TCD component. This makes the Pure Host usage model.

Requires **INCLUDE_USB_MHDRC**;

- **INCLUDE_USB_MHDRC_TCD_INIT**

Initializes TCD and registers the driver with VxBus; when this component is selected, it will only pull in the TCD and exclude HCD component. This makes the Pure Target usage model.

Requires **INCLUDE_USB_MHDRC**;

- **INCLUDE_USB_MHDRC_OCD_INIT**

Initializes OCD and registers the driver with VxBus; this component is hidden from user selection but is set as dependent component for the other two OTG usage model components, namely the **INCLUDE_USB_MHDRC_OCD_HCD_INIT** and the **INCLUDE_USB_MHDRC_OCD_HCD_INIT**. So this component will be automatically selected when OTG usage model is selected by the user.

Requires **INCLUDE_USB_MHDRC**.

- **INCLUDE_USB_MHDRC_OCD_HCD_INIT**

Initializes both OCD and HCD, and registers them with VxBus; when this component is selected, the OCD and HCD component as well as OCD initialization components are included, but it also excludes the pure HCD and pure TCD initialization components. This makes the SRP-capable Host-only A-device usage model.

Requires **INCLUDE_USB_MHDRC_HCD_INIT** and **INCLUDE_USB_MHDRC_OCD_INIT**.

- **INCLUDE_USB_MHDRC_OCD_TCD_INIT**

Initializes both OCD and TCD, and registers them with VxBus; when this component is selected, the OCD and TCD component as well as OCD initialization components are included, but it also excludes the pure HCD and pure TCD initialization components. This makes the SRP-capable Peripheral-only B-device usage model.

Requires **INCLUDE_USB_MHDRC_TCD_INIT** and **INCLUDE_USB_MHDRC_OCD_INIT**.

Note

There is no dedicated component for OTG Dual-Role usage model, this model is supported when both the **INCLUDE_USB_MHDRC_OCD_TCD_INIT** and **INCLUDE_USB_MHDRC_OCD_TCD_INIT** components are selected.

Note

The **INCLUDE_USB_MHDRC_OCD_INIT** component is not exported in the Workbench Component Configuration page (hidden from user selection). This is because the OCD cannot work alone; it needs at least either HCD or TCD, or both HCD and TCD to make a valid usage model.

Note

If the OTG Controller hardware supports only SRP-capable Host-only A-device or SRP-capable Peripheral-only B-device usage models, the CDF component definition should eliminate the non-available TCD or HCD component, and the corresponding OCD/TCD or OCD/HCD initialization component.

Note

Some fake "OTG Controller" hardware does not support both SRP and HNP, but it could report ID pin status, reflecting it is A-device or B-device, and thus could be initialized to be working in either Host or Target mode. Using such fake "OTG Controller", users could only implement "insertion based stack switching", so that when Mini-A plug is inserted (as A-device), switch to Host Stack, and when no plug or Mini-B plug is inserted, switch to Target Stack. For such fake "OTG Controller", all driver components should be implemented, only that the OCD doesn't report SRP or HNP related attributes and events. The functionalities of **usrUsbOtgHostRequest( )** and **usrUsbOtgHostGiveup( )** APIs are also limited for this kind of controllers.

2 controller drivers initialization

The Controller Driver initialization depends on the usage model, i.e, what components are selected.

Both Non-OTG and OTG usage models are described below, taking the MUSBMHDRC as an example.

2.1 non-otg usage model

When only **INCLUDE_USB_MHDRC_HCD_INIT** or **INCLUDE_USB_MHDRC_TCD_INIT** is selected to work as Non-OTG usage model, the controller driver initialization is the same as typical VxBus device initialization procedure.

1) The HCD/TCD will register a **DRIVER_REGISTRATION** to VxBus Framework.

2) Then the VxBus creates a **VXB_DEVICE_ID** as the instance of the Host/Target Controller, and calls the **drvBusFuncs** of the HCD/TCD **DRIVER_REGISTRATION**. The HCD/TCD should create a **USBHST_HCD**/**USBTGT_TCD** instance for the Host/Target Controller instance.

3) During the initialization, the driver needs to populate the hardware information by probing the hardware or retrieving user configurable parameters from the BSP.

4) As the last step of initialization, call Host or Target stack API routine to add the new HCD or TCD instance.

With this setup, it doesn't change too much to the existing model of VxBus device initialization.

2.2 otg usage model

When **INCLUDE_USB_MHDRC_OCD_INIT** is selected, **INCLUDE_USB_MHDRC_HCD_INIT** and **INCLUDE_USB_MHDRC_TCD_INIT** are included:

1) With this setup, OCD, HCD, and TCD will all register their **DRIVER_REGISTRATION** to VxBus Framework normally.

2) However, the VxBus creates only one single instance of **VXB_DEVICE_ID** for the OTG Controller (representing the whole USB Controller). The VxBus then calls the **drvBusFuncs** of OCD **DRIVER_REGISTRATION**.

3) The following operations will happen during the OCD initialization for this OTG Controller.

a) Create an instance of **USBOTG_OCD** and a common data structure which will be used to save common information among OCD, HCD and TCD.

Note

The driver developer should decide what are common among OCD, HCD and TCD, and then define a common data structure to hold these information; These drivers may need some other driver specific information, they could encapsulate these together with the common data structure (as a pointer) to form a driver private data structure. When the HCD/TCD is used in Non-OTG single role model, all these information should be populated by the HCD/TCD itself; However, when the HCD/TCD is used in OTG model, the OCD is responsible to populate the common information, the HCD/TCD can retrieve the common information when they are loaded.

b) Get the user configurable parameters, and probe the hardware to retrieve the common hardware resource information to be saved in the common data structure.

c) Reset the OTG Controller and do any IP specific hardware initialization so that it is able to detect the ID pin state, and thus able to distinguish the default role (A-device or B-device).

d) Populate the following data structure elements:

- The **USBOTG_OCD::bmAttributes** must be set with the OTG SRP/HNP/ADP support attributes bitwise OR'ed mask at this stage, by probing the hardware or through programming manual the driver writer should know what OTG capability the OTG controller supports. This will be used to construct the OTG descriptor when the hardware is working as peripheral.

- The **USBOTG_OCD::pDev** must be set to point to the **VXB_DEVICE_ID** passed in to **drvBusFuncs**. This **VXB_DEVICE_ID** will be used across OCD, HCD and TCD as the single instance for the whole USB Controller.

- The **USBOTG_OCD::pOcdFuncs** must be set to point to the **USBOTG_OCD_FUNCS** which has been filled with function pointers of the OCD. See **DATA STRUCTURES** section for what functions pointers should be provided by an OCD.

- The **VXB_DEVICE_ID::pDrvCtrl** must be set to point to the common data structure. This can be used by the HCD and TCD to retrieve the common information when they are loaded by the OTG Manager Framework.

e) Call **usbOtgOcdAdd( )** using the created **USBOTG_OCD** instance; The following actions will be taken during this call:

- Create an **USBOTG_MANAGER** as the OTG Manager instance.

- Associate the **USBOTG_OCD** and **USBOTG_MANAGER** so that **USBOTG_MANAGER::pOCD** points to the **USBOTG_OCD** instance, and the **USBOTG_OCD::pOTG** points to the **USBOTG_MANAGER** instance.

- Create other OTG Manager Framework resources such as a watchdog timer which could be used as the OTG timer if the OCD doesn't provide hardware OTG timers.

- Load Host and Target Controller Drivers by calling **USBOTG_OCD::pLoadHCD( )** and **USBOTG_OCD::pLoadTCD( )** interface routines of the OTG Controller Driver. See **HOST AND TARGET CONTROLLER DRIVER OTG SUPPORT CONSIDERATIONS** for details.

- Based on the current ID pin state, either the Host or the Target Stack will be enabled.

- Create OTG Manager Task to advance the OTG State Machine according to various OTG parameters.

- Add a character device into the **iosLib** which allows user application to access and control the OTG Manager such as to request to become host role.

f) The OTG Controller Driver is responsible for interrupt handling for both the Host and Target Controllers. Upon returning from **usbOtgOcdAdd( )**, the OTG Controller Driver connects and enables interrupts to get the stack running.

3. host and target controller driver otg support considerations

3.1 controller driver loading support for otg usage model

As described in the **Controller Driver Initialization** section, no matter if OTG support is enabled or not, the Host Controller Driver and Target Controller Driver can be registered to VxBus Framework normally (without the knowledge of **INCLUDE_USB_OTG**).

When an instance of a Host/Target is to be added by the **drvBusFuncs** of the **DRIVER_REGISTRATION** registered by the HCD/TCD, it means the VxBus Framework is adding a controller instance for Non-OTG usage model. In this case the normal VxBus device initialization procedure applies.

To support OTG usage model, the HCD/TCD should be responsible to provide a routine to allow the OCD to load the HCD or TCD driver instance for the Host/Target. It is up to the driver developer to decide how this routine is constructed, but it should pass the common hardware information (including the **VXB_DEVICE_ID** of the controller instance) to the HCD/TCD.

For example, the OCD for MUSBMHDRC could implement **USBOTG_OCD::pLoadHCD( )** as something like below:

1) In the code which is shared among HCD/TCD/OCD, a global function pointer variable is declared and is set as **NULL**:

```
usbMhdrcHcdTcdInterfacePrototype gpUsbMhdrcHcdUnLoad = NULL;
```

2) In the HCD initialization code **usbMhdrcHcdInit( )**, the global function pointer is set to the actual HCD instance add routine.

```
gpUsbMhdrcHcdLoad = usbMhdrcHcdInstanceAdd;
```

3) The component initialization process makes sure both HCD and TCD are initialized before OCD is initialized. This assures that if HCD or TCD components are included, then corresponding function pointers are initialized to the actual function; but if HCD or TCD is not included, then the function pointers are still kept as **NULL**. Using function pointers (instead of calling the actual function directly) makes the HCD or TCD components not pulled into the final image if they are not needed, thus reduces the image size.

4) In the OCD, for the **USBOTG_OCD::pLoadHCD( )**, the the global function pointer is called if it is not **NULL**. If the global function pointer is still **NULL**, then it indicates no HCD component is included, thus it should return **ERROR** to tell the OTG Manager.

The following are the general guidelines for the HCD/TCD loading routine:

1) Do only minimum Host/Target specific hardware initialization in this routine, leave all common hardware initialization to the OCD.

2) The OCD is also responsible for handling common interrupts, and only delivers the Host/Target specific interrupts to the HCD/TCD. So this routine doesn't need to connect or enable ISR (except for the ISR that is specific to either the Host or Target).

3) Retrieve the common hardware information the parameter. This requires the HCD/TCD to have agreement on the common hardware information, and the HCD/TCD specific data structure should contain a pointer to that common hardware information structure. (If it is not OTG enabled, then the HCD/TCD should retrieve the information by its own means.)

4) Get "**VXB_DEVICE_ID** pDev" from the common data structure passed in to this routine.

5) Call Host or Target Stack API to add the HCD/TCD instance.

Note

It is recommended that both HCD and TCD could implement the driver controller addition process with modular sub-routines:

- sub-routine to get common hardware information and resources
- sub-routine to add the controller instance to Host or Target Stack
- sub-routine to connect and enable interrupts
- sub-routine to enable the HCD or TCD for operation (transfer scheduling)

The controller removal process would be better to use the reverse procedure of these above sub-routines:

- sub-routine to disable HCD or TCD operation
- sub-routine to disable interrupts

- sub-routine to remove controller instance from Host or Target Stack
- sub-routine to release common hardware information and resources

With the above modular design, it is easier to use these common sub-routines across OCD, HCD and TCD.

The MUSBMHDRC controller drivers supported by this stack can be used as examples to implement such modular and staged driver load, unload, enable and disable interfaces.

3.2 otg controller driver interrupt handling

3.2.1 reporting otg related events to otg manager

The OCD ISR may report the following events to the OTG Manager through the OTG event notification API **usbOtgEventRaise( )**.

1) ID pin state

If the OTG Controller could generate interrupt if the ID pin state changes, it should be reported with **USBOTG_EVENT_ID_ID_STATE_CHANGED**.

This interrupt is not delivered to HCD or TCD.

Note

The OCD must implement the **USBOTG_OCD_FUNCS::pGetIdState( )** interface routine so that the OTG Manager could also poll the ID state.

2) VBUS state

If the OTG Controller could generate interrupt if the VBUS state changes across these OTG defined VBUS thresholds, specially, when there is VBUS error conditions, the current VBUS state should be reported with **USBOTG_EVENT_ID_VBUS_STATE_CHANGED**.

This interrupt is not delivered to HCD or TCD.

Note

The OCD must implement the **USBOTG_OCD_FUNCS::pGetVbusState( )** interface routine so that the OTG Manager could poll also the VBUS state.

3) SRP detected event

If the OTG Controller could generate interrupt if it detects a SRP request when it is A-device, the event should be reported with **USBOTG_EVENT_ID_SRP_DETECTED**.

This interrupt is not delivered to HCD or TCD.

Note

The OCD may choose to implement the **USBOTG_OCD_FUNCS::pGetSrpDetState( )** interface routine so that the OTG Manager could poll the SRP detect state when it is in OTG a_idle state. But this interface routine is not required, since most OTG Controllers could report the event by interrupt.

4) Bus Reset event

The Bus Reset event must be reported to the OTG Manager as it is an indication to set the active stack to Target Stack (if it is not already in Target Stack). This is reported with **USBOTG_EVENT_ID_BUS_RESET_DETECTED**.

The Bus Reset event will also be delivered to the TCD.

5) Connect and Disconnect events

The Connect and Disconnect event must be reported to the OTG Manager as it may be used as an indication to set the active stack to Host Stack (if it is not already in Host Stack). This is reported with **USBOTG_EVENT_ID_CONNECT_DISCONNECT_DETECTED**.

The Connect and Disconnect events will also be delivered to the HCD.

6) Suspend and Resume (or Remote Wakeup) events

The Suspend and Resume (or Remote Wakeup) events must be reported to the OTG Manager as they are very important to advance the OTG State Machine for HNP. This is reported with **USBOTG_EVENT_ID_SUSPEND_RESUME_DETECTED**.

Suspend and Resume events will also be reported to the TCD, and Remote Wakeup event will be reported to the HCD.

3.2.2 deliver interrupts to hcd or tcd

Some OTG related events such as ID change or SRP detection should be reported only to OTG Manager; some interrupts such as the Connect and Disconnect events should be reported to both the OTG Manager and the HCD or TCD. Some interrupts, such as the data transfer related interrupts must only be reported to the proper controller driver (based on the current active stack) to be processed.

The OCD implemented for the MUSBMHDRC can be used as an example for how to report the OTG related events to OTG manager, as well as deliver specific interrupts to HCD or TCD.

Note

Because the OTG Manager depends on the "Bus Reset" and "Device Connect" event to do the actual stack mode switching if it is not in the proper stack mode. The OCD should implement logic to make sure the "Bus Reset" event is always reported to OTG Manager and TCD (even if the current stack mode is still in Host mode); and the "Device Connect" event is always reported to OTG Manager and HCD (even if the current stack mode is still in Target mode).

4 data structures

4.1 **usbotg_manager**

Each USB OTG Controller will have a **USBOTG_MANAGER** data structure, which plays the core role in managing the OTG states. The structure is defined as below:

```
/*
 * USB OTG Manager Abstraction
 */

typedef struct usbotg_manager
    {
    DEV_HDR                 devHdr;     /* I/O system device header */
    NODE                    node;       /* Node in the OTG controller list */
    atomic_t                devOpened;  /* TRUE if the OTG manager is opened */
    VXB_DEVICE_ID           pDev;       /* OTG Controller instance */
    pUSBOTG_OCD             pOCD;       /* Pointer to OTG Controller Driver */
    pUSBOTG_OCD_FUNCS       pOcdFuncs;  /* Short cut to OCD Functions */
```

```
    BOOL                    isHcdLoaded;/* Is HCD loaded */
    BOOL                    isTcdLoaded;/* Is TCD loaded */
    USBOTG_STATE            prevState;  /* Previous OTG state */
    USBOTG_STATE            currState;  /* Current OTG state */
    USBOTG_ID_STATE         idState;    /* Current ID state */
    USBOTG_VBUS_STATE       vbusState;  /* Current VBUS state */
    USBOTG_PARAM            param;      /* Current OTG parameters */
    USBOTG_MODE             stackMode;  /* Host or Target Stack active */
    USBOTG_TIMER            otgTimer;   /* Active OTG Timer ID */
    WDOG_ID                 wdId;       /* Watchdog Timer (maybe OTG timer) */
    atomic_t                wdInUse;    /* TRUE if watchdog used as OTG Timer */
    SEM_ID                  otgLock;    /* Lock for the OTG instance */
    SEM_ID                  paramSem;   /* Parameter changed notification */
    TASK_ID                 mgrTaskId;  /* Task ID for OTG manager */
    int                     mgrInterval;/* OTG manager task polling interval */
    UINT16                  otgVersion; /* bcdOTG version that we support */
    UINT32                  hOtgDevice; /* OTG device handle if non-zero */
    UINT32                  uValidMagic;/* Check if the structure is valid */
    USBOTG_USR_CALLBACK *   pUsrCallback; /* User notification callback */
    void *                  pUsrContext;/* User callback context */
    }USBOTG_MANAGER, * pUSBOTG_MANAGER;
```

The following items describe the major elements of the **USBOTG_MANAGER** data structure.

4.1.1 **usbotg_manager::devhdr** and **usbotg_manager::devopened**

These two elements are related the **iosLib** interface to allow user applications to communicate with the OTG Manager. Each **USBOTG_MANAGER** will create one **iosLib** device entry. Most interactions between user application and OTG Manger is through **ioctl** interface, the detailed ioctl command codes are described in the comment for **usbOtgManagerIoctl( )** routine.

4.1.2 **usbotg_manager::node**

This is used to link the OTG manager instance into a global list of OTG manager instances. Used for track and show the internal states.

4.1.3 **usbotg_manager::pocd** and **usbotg_manager::pocdfuncs**

When an OTG controller instance is added, it creates a **USBOTG_OCD** data structure, populates it with proper attributes and the pointer to the **USBOTG_OCD_FUNCS**, and calls **usbOtgOcdAdd( )** to add that OCD instance. The **USBOTG_MANAGER::pOCD** is set to point to that OCD instance. The OTG Manager will need to call the OCD functions frequently to update OTG parameters or control OTG hardware behavior, so the **USBOTG_MANAGER::pOcdFuncs** is set to be a shot cut to the OCD functions (so it is the same as **USBOTG_OCD::pOcdFuncs**).

4.1.4 **usbotg_manager::ishcdloaded** and **usbotg_manager::istcdloaded**

Flags used for indicating if HCD or TCD instance is loaded. Used to determine if the related operations for HCD ot TCD can be performed (for example, if HCD is not loaded, HNP can not be supported).

4.1.5 **usbotg_manager::prevstate** and **usbotg_manager::currstate**

These two elements are the current OTG state and the previous state it was in. We keep the **USBOTG_MANAGER::prevState** so that we know where we come from, in some cases it could help to decide what actions to take while entering a state.

4.1.6 **usbotg_manager::idstate** and **usbotg_manager::vbusstate**

Latest ID and VBUS state. They can be returned to the user application for diagnostics or take specific actions to control the OTG Manager (and thus the OTG State Machine).

4.1.7 **usbotg_manager::param**

The OTG State Machine is advanced as a result of parameter changes. The parameters are classified into various groups according to the OTG specification. We also added an "extend" parameter group, which is handy to keep some stack implementation related variables. See the comments in **usbOtg.h** for more details.

```
/*
 * USB OTG State Machine Parameters
 */

typedef struct usbOtgParam
    {
    USBOTG_INPUT_PARAM      input;
    USBOTG_INTERNAL_PARAM   internal;
    USBOTG_TIMEOUT_PARAM    timeout;
    USBOTG_OUTPUT_PARAM     output;
    USBOTG_EXTEND_PARAM     extend;
    }USBOTG_PARAM;
```

4.1.8 **usbotg_manager::stackmode**

This is the current stack active mode. This can be used by the common ISR to direct the interrupts to either HCD or TCD for further processing.

4.1.9 **usbotg_manager::otgtimer** and **usbotg_manager::wdid/wdinuse**

The OTG State Machine involves several timers in specific states. At one point in time, there may be at most one active timer. The **USBOTG_MANAGER::otgTimer** specifies the current active timer.

Some OTG Controllers may provide hardware timer facility. For such hardware, the OCD may implement the **USBOTG_OCD_FUNCS::pOtgTimerStart( )/pOtgTimerStop( )** to let the OTG Manager to take advantage of the hardware timer facility. If there is no such hardware timer facility, or the timer can not be started successfully, then the OTG Manager will fall back to use the OS specific watchdog timer.

4.1.10 **usbotg_manager::otglock**

This is a mutex semaphore which is used to protect the **USBOTG_MANAGER** elements being concurrent updated, especially the **USBOTG_MANAGER::param**. Because the **USBOTG_MANAGER::param** may be changed by the user requests, or by OTG Notifier functions reporting events, or by timers timeout, or by the internal OTG Manager task while advancing the OTG State Machine.

4.1.11 **usbotg_manager::mgrtaskid** and **usbotg_manager::paramsem** and

USBOTG_MANAGER::mgrInterval

The **USBOTG_MANAGER::mgrTaskId** specifies the task which will wait for the parameter changed event notification **USBOTG_MANAGER::paramSem** with a predefined timeout value **USBOTG_MANAGER::mgrInterval**, but it does not always depend on the event notification, thus it does not check if the wait exits because of timeout or not. This is to implement the following behavior:

1) If the parameters are changed due to user request, or the low level drivers event report, or OTG timer timeout action, then the semaphore will be released by these handler routines, and thus this OTG manager task would be activated by the event. This is to avoid the need for **taskDelay( )** which would always block the task the fixed amount of time.

2) If there is no parameter change event notification, it doesn't mean there is no parameter change. Some parameters may not be reported by event notifications, for example, some hardware can not report ID pin changes through interrupt, thus can not be noticed by the OTG Manager without explicit state polling, these parameters must be polled. For this reason, the semaphore wait time out gives us a chance to perform the parameter state polling and check if we need to advance the OTG State Machine.

4.1.12 **usbotg_manager::otgversion**

The OTG version we support. It is set to the macro **USBOTG_VERSION_SUPPORTED** at compile time. This can be used to construct the OTG descriptor bcdOTG filed when we support OTG 2.0. We currently implement support for OTG 1.3.

Note

Although we implement the support for OTG 1.3, we have added enough code to be easily extensible to support OTG 2.0. We also enabled the OTG 2.0 HNP polling mechanism to support user request of HNP by HNP polling. This is useful when both ends of the OTG connection are using our OTG stack. If the other end doesn't support HNP polling, then the behavior is the same as normal OTG.

```
#define USBOTG_VERSION0100              0x0100
#define USBOTG_VERSION0120              0x0120
#define USBOTG_VERSION0130              0x0130
#define USBOTG_VERSION0200              0x0200
#define USBOTG_VERSION_SUPPORTED        USBOTG_VERSION0130
```

4.1.13 **usbotg_manager::hotgdevice**

This is the device handle of the remote OTG device when we are in host mode. We could use it to suspend the bus (starting HNP), or we could use it to get the OTG status (for HNP polling).

4.1.14 **usbotg_manager::uvalidmagic**

A magic value which is set to **USBOTG_MAGIC_VALID** when the **USBOTG_MANAGER** is created, and set to **USBOTG_MAGIC_INVALID** before it is destroyed. This is handy for code that refers to the **USBOTG_MANAGER** instance to check if it is safe to access the **USBOTG_MANAGER** (its value must be **USBOTG_MAGIC_VALID**).

4.1.15 **usbotg_manager::pusrcallback** and **usbotg_manager::pusrcontext**

Pointer to user application installed message callback and its context.

4.2 **usbotg_ocd** and **usbotg_ocd_funcs**

Each OTG Controller will have an **USBOTG_OCD** instance, in which there is a pointer to the **USBOTG_OCD_FUNCS**. These data structures are defined as below.

```
/*
 * OTG Controller Driver abstraction
 */
```

```
typedef struct usbotg_ocd
    {
    /* USB OTG Controller VxBus Device Instance */

    VXB_DEVICE_ID       pDev;

    /* OTG Controller Driver Functions */

    pUSBOTG_OCD_FUNCS   pOcdFuncs;

    /* Pointer to OTG Controller hardware specific data */

    pVOID               pOcdSpecific;

    /* Pointer to the OTG Manager Framework instance */

    pUSBOTG_MANAGER     pOTG;

    /* The OTG attributes that we support */

    UINT8               bmAttributes;
    }USBOTG_OCD, *pUSBOTG_OCD;

/*
 * USB OTG Controller Driver Functions
 */

typedef struct usbotg_ocd_funcs
    {
    /* Get the VBUS state of OTG port */

    STATUS (* pOtgVbusStateGet)
            (
            struct usbotg_ocd * pOCD,
            USBOTG_VBUS_STATE * pVbusState
            );

    /* Get the ID state of OTG port */

    STATUS (* pOtgIdStateGet)
            (
            struct usbotg_ocd * pOCD,
            USBOTG_ID_STATE *   pIdState
            );

    /* Get the line state of OTG port */

    BOOL (* pOtgIsLineStateSE0)
            (
            struct usbotg_ocd * pOCD
            );

    /* Get the SRP Detection state of OTG port */

    BOOL (* pOtgSrpDetStateGet)
            (
            struct usbotg_ocd * pOCD
```

```
        );

/*
 * Enable or disable driving the VBUS
 */

STATUS (* pOtgDrvVbusEnable)
        (
        struct usbotg_ocd * pOCD,
        BOOL                bEn
        );

/*
 * Enable or disable charging the VBUS
 * (Used for manually control of Vbus pulsing SRP)
 */

STATUS (* pOtgChrgVbusEnable)
        (
        struct usbotg_ocd * pOCD,
        BOOL                bEn
        );

/*
 * Enable or disable discharging the VBUS
 * (Accelerate SRP initial conditions for Vbus pulsing SRP)
 */

STATUS (* pOtgDisChrgVbusEnable)
        (
        struct usbotg_ocd * pOCD,
        BOOL                bEn
        );

/*
 * Enable or disable automatic SRP detection
 * of the OTG Controller
 */

STATUS (* pOtgAutoSrpDetEnable)
        (
        struct usbotg_ocd * pOCD,
        BOOL                bEn
        );

/*
 * Enable or disable automatic Suspend to Disconnect
 * of the OTG Controller (support for automatic HNP)
 */

STATUS (* pOtgAutoSusp2DisEnable)
        (
        struct usbotg_ocd * pOCD,
        BOOL                bEn
        );

/*
```

```
     * Enable or disable automatic Disconnect to Connect
     * of the OTG Controller (support for automatic HNP)
     */

    STATUS (* pOtgAutoDis2ConEnable)
            (
            struct usbotg_ocd * pOCD,
            BOOL                bEn
            );

    /*
     * Enable or disable automatic Connect to Reset
     * of the OTG Controller (support for automatic HNP)
     */

    STATUS (* pOtgAutoCon2RstEnable)
            (
            struct usbotg_ocd * pOCD,
            BOOL                bEn
            );

    /* Enable or disable D+ pull-up of the OTG port */

    STATUS (* pOtgDpPullUpEnable)
            (
            struct usbotg_ocd * pOCD,
            BOOL                bEn
            );

    /* Enable or disable D+ pull-down of the OTG port */

    STATUS (* pOtgDpPullDownEnable)
            (
            struct usbotg_ocd * pOCD,
            BOOL                bEn
            );

    /* Enable or disable D- pull-down of the OTG port */

    STATUS (* pOtgDmPullDownEnable)
            (
            struct usbotg_ocd * pOCD,
            BOOL                bEn
            );

    /* Set the Controller to work in the requested operation mode */

    STATUS (* pOtgSetCtlrMode)
            (
            struct usbotg_ocd * pOCD,
            USBOTG_MODE         mode
            );

    /* Request the OTG Controller to initiate SRP signaling */

    STATUS (* pOtgRequestSrp)
            (
```

```
        struct usbotg_ocd * pOCD,
        UINT32              uSrpTypeFlags
        );

/*
 * Start the hardware OTG timer if the OTG Contorller
 * provides one
 */

STATUS (* pOtgTimerStart)
        (
        struct usbotg_ocd *     pOCD,
        USBOTG_TIMER            timerId,
        int                     timeout,
        usbOtgTimerTimeoutFunc  timeoutFunc
        );

/*
 * Stop the hardware OTG timer if the it is using the
 * hardware timer as the OTG timer
 */

STATUS (* pOtgTimerStop)
        (
        struct usbotg_ocd *     pOCD,
        USBOTG_TIMER            timerId
        );

/* Load the HCD for the OTG enabled Host Controller */

STATUS (* pOtgLoadHCD)
        (
        struct usbotg_ocd *     pOCD
        );

/* Load the TCD for the OTG enabled Target Controller */

STATUS (* pOtgLoadTCD)
        (
        struct usbotg_ocd *     pOCD
        );

/* UnLoad the HCD for the OTG enabled Host Controller */

STATUS (* pOtgUnLoadHCD)
        (
        struct usbotg_ocd *     pOCD
        );

/* UnLoad the TCD for the OTG enabled Target Controller */

STATUS (* pOtgUnLoadTCD)
        (
        struct usbotg_ocd *     pOCD
        );

/* Function pointer to enable the TCD */
```

```
STATUS (* pOtgEnableTCD)
        (
        struct usbotg_ocd *     pOCD
        );

/* Function pointer to disable the TCD */

STATUS (* pOtgDisableTCD)
        (
        struct usbotg_ocd *     pOCD
        );

/* Fucntion pointer to enable the HCD */

STATUS (* pOtgEnableHCD)
        (
        struct usbotg_ocd *     pOCD
        );

/* Fucntion pointer to disable the HCD */

STATUS (* pOtgDisableHCD)
        (
        struct usbotg_ocd *     pOCD
        );
}USBOTG_OCD_FUNCS, *pUSBOTG_OCD_FUNCS;
```

Most of the elements for **USBOTG_OCD** are referred to and described in the section **Controller Driver Initialization**, so this section will only describe these elements in **USBOTG_OCD_FUNCS**.

4.2.1 **usbotg_ocd_funcs::potgvbusstateget( )**

This interface routine must be implemented by the OCD. It must be able to detect and report the current VBUS state. Note that in OTG 1.3 and earlier there were 4 different VBUS threshold levels (SessEnd, A-SessValid, B-SessValid, and Vbus-Valid), but in OTG 2.0 it is reduced to 3 VBUS threshold levels by merging A-SessValid and B-SessValid with SessValid. We also follow the OTG 2.0 defitions, so that the implementation only needs to report 3 different VBUS states:

- USBOTG_VBUS_STATE_SessEnd (VBUS < **VB_SESS_END**)

- USBOTG_VBUS_STATE_SessValid (**VB_SESS_END** <= VBUS < **VA_VBUS_VLD**)

- USBOTG_VBUS_STATE_VBusValid (**VA_VBUS_VLD** <= VBUS)

If the OTG Controller can report VBUS changes through interrupt, such as VBUS at VBUS error conditions, the VBUS state could be reported by the OTG event **USBOTG_EVENT_ID_VBUS_STATE_CHANGED**.

4.2.2 **usbotg_ocd_funcs::potgidstateget( )**

This interface routine must be implemented by the OCD. It must be able to detect and report the current ID state.

If the OTG Controller can report ID changes through interrupt, the ID state could be reported by the OTG event **USBOTG_EVENT_ID_ID_STATE_CHANGED**.

4.2.3 **usbotg_ocd_funcs::potgislinestatese0( )**

This interface routine must be implemented by the OCD. It must be able to detect and report the current D+/D- LINE state. Especially, the **USBOTG_LINE_STATE_SE0** state is an important factor for the B-device to initiate SRP.

4.2.4 **usbotg_ocd_funcs::potgsrpdetstateget( )**

The OCD MAY choose to implement this interface routine, but is optional.

Most OTG Controllers report SRP detection through interrupt, so the SRP detection could be reported by the OTG **USBOTG_EVENT_ID_SRP_DETECTED**.

4.2.5 **usbotg_ocd_funcs::potgdrvvbusenable( )**

This interface routine must be implemented by the OCD. As A-device, it must be able to enable or disable driving the VBUS (so that it could raise and maintain at VbusValid, or fall below SessValid if disabled driving the VBUS). This is to save power if there is no need to power the bus, or start the session if requested by the B-device.

4.2.6 **usbotg_ocd_funcs::potgchrgvbusenable( )**

This interface routine only needs to be implemented if the OTG Controller needs manually control of the VBUS pulsing SRP.

Some OTG Controllers need to explicitly enable or disable the VBUS charging for the VBUS pulsing SRP (such as the ISP1362). For such OTG Controllers, the OCD needs to implement this interface routine to enable or disable the VBUS charging. The OTG Manager will use this interface to initiate the VBUS pulsing with proper timing.

However, most OTG Controllers could implement automatic SRP process without user software intervention, except only to set some register bit to kick start of SRP (the underlying SRP type and timing of pulsing is controlled by hardware). For such OTG Controllers, **USBOTG_OCD_FUNCS::pOtgChrgVbusEnable( )** must be set to **NULL**, and **USBOTG_OCD_FUNCS::pOtgRequestSrp( )** must be implemented to allow the OTG Manger to request to iniate the SRP process.

4.2.7 **usbotg_ocd_funcs::potgdischrgvbusenable( )**

The OCD MAY choose to implement this interface routine, but is optional.

Before SRP can be initiated by the OTG Manager, it needs to make sure all SRP initial conditions are met. One of the SRP initial conditions is the VBUS should fall below the SessEnd threshold so that the VBUS pulsing can be correctly interpreted by the A-device.

Some OTG Controllers provide some means to accelerate the SessEnd condition to be met by using a discharging resister. For such OTG Controllers, the OCD could implement this interface routine to enable or disable the VBUS discharging.

4.2.8 **usbotg_ocd_funcs::potgautosrpdetenable( )**

The OCD MAY choose to implement this interface routine, but is optional.

Most OTG Controllers report SRP detection through interrupt, the OCD may choose to implement this routine to enable or disable the SRP detection interrupt. The OTG Manager only needs to enable SRP detection while in a_idle state, and disable it in other states.

If the SRP detection is always enabled, this interface routine can be no-op, or just set **USBOTG_OCD_FUNCS::pOtgAutoSrpDetEnable( )** to **NULL**.

4.2.9 **usbotg_ocd_funcs::potgautosusp2disenable( )**

The OCD needs to implement this interface routine if the OTG Controller has automatic Suspend to Disconnect feature for HNP and the feature can be enabled or disabled at run time.

During HNP, while the B-device is in b_peripheral state, it shall be able to detect the Suspend from the A-device and transition to the b_wait_acon state within **TB_AIDL_BDIS** (Min 4ms, Max 150ms) (A-Idle to B-Disconnect Time) after Suspend is detected at the B-device, and upon entering b_wait_acon state, the B-device should turns off its pull-up resistor on D+, starts b_ase0_brst_tmr timer, and waits for the A-device to signal a connect.

The A-Idle (Suspend) to B-Disconnect could be done by the software to detect the A-device Suspend, then dessert B-device D+ as soon as possible. However, due to unpredictable interrupt and software latency, it may be risky to use pure software to handle this procedure (possibly exceed the 150ms **TB_AIDL_BDIS** max limitation).

Some hardware provides hardware automatic assistance for such critical timing requirement, so that once the A-device Suspend is detected, the B-device deasserts its D+ pull-up automatically (without software intervention). If the hardware provides such an automatic Suspend to Disconnect feature, the OCD should implement the **OtgAutoSus2DisEnable( )** interface to let the USB OTG Framework enable/disable the feature when required; If the feature is always enabled, then the implementation can just be no-op and return **OK**;

If the hardware doesn't provide such a feature, then the OCD must set the field **USBOTG_OCD_FUNCS::pOtgAutoSusp2DisEnable( )** to be **NULL**, the OTG Framework will use this **NULL** value as an indication that it needs to handle the Suspend to Disconnect procedure by software.

4.2.10 **usbotg_ocd_funcs::potgautodis2conenable( )**

The OCD needs to implement this interface routine if the OTG Controller has automatic Disconnect to Connect feature for HNP and the feature can be enabled or disabled at run time.

During HNP, the B-disconnect occurs from the OTG a_suspend state and within 3ms (**TA_BDIS_ACON** max), the A-device must enable the pull-up on the D+ in the a_peripheral state to signal Connect to the B-device.

This could be done by the software to detect the B-device disconnection, then assert A-device D+ pull-up as soon as possible. However, due to unpredictable interrupt and software latency, it may be risky to use pure software to handle this procedure (possibly exceed the 3ms **TA_BDIS_ACON** max limitation).

Some hardware provides hardware automatic assistance for such critical timing requirement, so that once the B-device disconnection is detected, the A-device signals connection to the B-device automatically (without software intervention). If the hardware provides such an automatic Disconnect to Connect feature, the OCD should implement the **USBOTG_OCD_FUNCS::pOtgAutoDis2ConEnable( )** interface to let the USB OTG Manager to enable/disable the feature when required; If the feature is always enabled, then the implementation can just be no-op and return **OK**;

If the hardware doesn't provide such a feature, then the OCD must set the field **USBOTG_OCD_FUNCS::pOtgAutoDis2ConEnable( )** as **NULL**, the OTG Manager will use this **NULL** value as an indication that it needs to handle the disconnection to connection procedure by software.

4.2.11 **usbotg_ocd_funcs::potgautocon2rstenable( )**

The OCD needs to implement this interface routine if the OTG Controller has automatic Connect to Reset feature for HNP and the feature can be enabled or disabled at run time.

During HNP, while the B-device is in b_wait_acon state, it shall be able to detect the connect from the A-device and transition to the b_host state within 1ms (**TB_ACON_BSE0** max) after D+ is detected high at the B-device, and upon entering b_host state, the B-device should issue a bus reset, and start generating SOF.

This could be done by the software to detect the A-device connection, then assert B-device Bus Reset as soon as possible. However, due to unpredictable interrupt and software latency, it may be risky to use pure software to handle this procedure (possibly exceed the 1ms **TB_ACON_BSE0** max limitation).

Some hardware provides hardware automatic assistance for such critical timing requirement, so that once the A-device connection is detected, the B-device signals Bus Reset to the A-device automatically (without software intervention). If the hardware provides such an automatic Connect to Reset feature, the OCD should implement the **USBOTG_OCD_FUNCS::pOtgAutoCon2RstEnable( )** interface to let the USB OTG Manager enable/disable the feature when required; If the feature is always enabled, then the implementation can just be no-op and return **OK**;

If the hardware doesn't provide such a feature, then the OCD must set the field **USBOTG_OCD_FUNCS::pOtgAutoCon2RstEnable( )** as **NULL**, the OTG Manager will use this **NULL** value as an indication that it needs to handle the connection to reset procedure by software.

4.2.12 **usbotg_ocd_funcs::potgdppullupenable( )**

The OCD needs to implement this interface routine if the OTG Controller needs to initiate data line pulsing SRP manually by enabling and then disabling of D+ line pull-up (in which case the **USBOTG_OCD_FUNCS::pOtgRequestSrp( )** is set as **NULL**).

The OCD needs to implement this interface routine if the OTG Controller has no hardware assistance for automatic B-Disconnect to A-Connect handling during HNP, (in which case the **USBOTG_OCD_FUNCS::pOtgAutoDis2ConEnable( )** is set as **NULL**). And the OTG Manager should assert the A-device D+ pull-up to signal connection to B-device.

In other cases, if the OTG Controller can perform autoamtic data line SRP, and can do automatic B-Disconnect to A-Connect handling during HNP, then OCD should set **USBOTG_OCD_FUNCS::pOtgDpPullUpEnable( )** as **NULL**.

4.2.13 **usbotg_ocd_funcs::potgdppulldownenable( )**

The OCD needs to implement this interface routine only if the OTG Controller needs explicitly to enable or disable the D+ line pull-down by software at specific time points.

When an OTG device or Embedded Host is idle or acting as a host, it shall activate both the D+ and D- pull-down resistors. These resistors shall be within the range of RPD as defined in [USB2.0].

When an OTG A-device or B-device is acting as a peripheral, it shall disable the pull-down on the D+ line but shall maintain the D- pull-down.

Most hardware implement the D+ line pull-down enable/disable automatically, thus there is no need for software to control, so in most cases this interface can be set to **NULL** by the OCD. Some OTG Controllers, such as the ISP1362, still needs to implement this controlling over software.

4.2.14 **usbotg_ocd_funcs::potgdmpulldownenable( )**

The OCD needs to implement this interface routine only if the OTG Controller needs explicitly to enable or disable the D- line pull-down by software at specific time points.

The D- line pull-down is always enabled in almost all cases, either it is in host or peripheral mode. Even an OTG device changes between host and peripheral roles, the D- pull-down shall remain asserted. This prevents the D- line on the A-device from floating if the B-device is detached.

An OTG device is allowed to disable both pull-down resistors during the interval of a packet transmission while acting as either a host or a peripheral. However this kind of pull-down is mostly implemented by hardware autoamatically. Some OTG Controllers, such as the ISP1362, still needs to implement this controlling over software.

4.2.15 **usbotg_ocd_funcs::potgsetctlrmode( )**

The OCD needs to implement this interface routine only if the OTG Controller needs explicitly to set the controller operation mode by software at specific time points.

During HNP, the peripheral and host will exchange role to each other. Some hardware can switch into proper mode while detecting specific signaling at specific time points. For example, when HNP is enabled on the B-device, if bus suspend

detected, some hardware could switch into host operation mode automatically. However some hardware may need a manual setting to make the controller work in either peripheral mode or host mode. If the hardware can switch into proper operation mode automatically, the **USBOTG_OCD_FUNCS::pOtgSetCtlrMode( )** could be set to **NULL**, or do nothing. Otherwise, it must be implemented to set to the proper operation mode.

4.2.16 **usbotg_ocd_funcs::potgrequestsrp( )**

The OCD needs to implement this interface routine if the OTG Controller supports automatic SRP signaling (by setting some register bits).

The OTG Controller may perform this request by pulsing the D+ data line, followed by pulsing the VBUS line (if it is OTG 1.3 or earlier). The OCD may choose to drive the signaling directly in this call, or delay the operation at a later time. However it is recommended to do the SRP signaling ASAP.

The OTG Manager will make sure that the initial conditions for driving SRP are satisfied before calling this interface routine.

As per OTG 2.0, the B-device may not attempt to start a new session until 1.5 sec (**TB_SSEND_SRP**) after VBUS has gone below 0.8V (**VOTG_SESS_VLD**), and both the D+ and D- data lines have been low (SE0) for at least 1 sec (**TB_SE0_SRP** min). This ensures both the end of the previous session and that the A-device has detected a disconnect condition from the B-device. To help the OTG Manager to achieve the SRP initial condition, the **USBOTG_OCD_FUNCS::pOtgIsLineStateSE0( )** must be implemented by the OCD; and it is helpfull if the OCD also implements **USBOTG_OCD_FUNCS::pOtgDisChrgVbusEnable( )** interface routine if possible.

The OTG Controller Driver doesn't need to monitor or report SRP status. If the SRP successes, the A-device will raise VBUS and the OTG Manager can get aware of the result through the OTG Notifier callback or by probing the VBUS state.

A watchdog timer will be started once the SRP signaling is requested. If the timer times out and the VBUS is still not valid, then the OTG Manager will report failure to the user application and the user can request the SRP again at some later time.

4.2.17 **usbotg_ocd_funcs::potgtimerstart( )/potgtimerstop( )**

The OCD needs to implement this interface routine if the OTG Controller supports hardware OTG timer facility.

Some USB OTG Controller hardware provides hardware support for these USB OTG timers, because some of the timers are in minimum 1ms granularity and not all OS timers can always provide such fine granularity. If the OCD wants to use such hardware OTG timers, it should implement the pair of interface routines **USBOTG_OCD_FUNCS::pOtgTimerStart( )/pOtgTimerStop( )** and return **OK** when the timer is successfully started/stopped; If the OTG Controller Driver doesn't implement the rotines (set **USBOTG_OCD_FUNCS::pOtgTimerStart( )/pOtgTimerStop( )** as **NULL**), or the call failes, then the OTG Manager will use an OS specific watchdog timer.

4.2.19 **usbotg_ocd_funcs::potgloadhcd( )/potgloadtcd( )/potgunloadhcd( )/potgunloadtcd( )**

The OCD must implement these interface routines. The detailed process and how to implement these routines are described in **Controller Driver Initialization**.

4.2.20 **usbotg_ocd_funcs:: potgenablehcd( )/potgenabletcd( )/potgdisablehcd( )/potgdisabletcd( )**

The OCD must implement these interface routines. The detailed process and how to implement these routines are described in **Controller Driver Initialization**.

5 usb otg srp implementation notes

This section describes the VxWorks USB OTG stack implementation for SRP protocol.

In order to conserve power, an A-device is allowed to leave VBUS turned off when the bus is not being used. The Session Request Protocol (SRP) allows a B-device to request the A-device to turn on the power supply to the USB interface (VBUS) and

start a session. A session is defined as the period of time that VBUS is powered. The session ends when VBUS is no longer powered.

5.1 b-device user application detects vbus state not valid

The B-device user application can get the current VBUS state through IOCTL command **USBOTG_IOCTL_VBUS_STATE_GET**, which in turns calls the **usbOtgVbusStateGet( )** to get the VBUS state. The **usbOtgVbusStateGet( )** actually gets the VBUS state through a call to **USBOTG_OCD_FUNCS::pOtgVbusStateGet( )**.

5.2 b-device requests vbus to be powered by a-device

If the current VBUS state is not valid, the user application could use the IOCTL command **USBOTG_IOCTL_VBUS_POWER_ON** to request A-device to power up VBUS. This in turn calls **usbOtgVbusPowerOn( )**, and it will set the b_bus_req to **TRUE**. Then the OTG Manager task is notified the parameter change.

5.3 b-device waiting for srp initial conditions to be met

The OTG Manager will wait for the SRP initial conditions to be met before actually initiating SRP.

- It needs to call **USBOTG_OCD_FUNCS::pOtgIsLineStateSE0( )** to get the current line state, as one of the inital condition is it should be in SE0 state for enough time (**TB_SE0_SRP** min). In OTG 2.0 this is around 1 sec.

- It may call **USBOTG_OCD_FUNCS::pOtgDisChrgVbusEnable( )** to discharge the VBUS to below SessEnd threashold as another initial condition requires.

5.4 b-device initiate srp

When all initial conditions are met for SRP, the OTG Manager could initiate SRP. Depending on if there is automatic SRP hardware support, there may be two ways to initiate SRP.

a) With automatic SRP hardware support

If there is automatic SRP hardware support, the OCD should implement the interface routine **USBOTG_OCD_FUNCS::pOtgRequestSrp( )**, and it is called with the expected SRP type bit mask (Data line pulsing, VBUS pulsing). It is up to the OCD to decide how and when to drive the SRP signaling (but it is recommended to drive it ASAP). Also the SRP timing (Data line and VBUS pulsing length) is controlled by the hardware.

b) Without automatic SRP hardware support

If there is no automatic SRP hardware support, the OCD should set the interface routine **USBOTG_OCD_FUNCS::pOtgRequestSrp( )** as **NULL**. The OCD should implement the **USBOTG_OCD_FUNCS::pOtgDpPullUpEnable( )**; Optionally, if VBUS pulsing SRP needs to be implemented, **USBOTG_OCD_FUNCS::pOtgChrgVbusEnable( )** should be implemented by the OCD.

Once the SRP initial conditions are met, the OTG Manager starts data line pulsing by asserting D+ pull-up with a call to **USBOTG_OCD_FUNCS::pOtgDpPullUpEnable( )**; It then starts a watchdog timer **USBOTG_TIMER_b_data_pulse_tmr**, with timeout set to be around 7ms (in the range 5ms to 10ms); Once the timer times out, in the timer timeout handling routine, it deasserts the D+ pull-up; This forms a data line pulse. It then initiates the VBUS pulsing by asserting the VBUS charging using **USBOTG_OCD_FUNCS::pOtgChrgVbusEnable( )** and starts a watchdog timer with timeout set to around 20ms (in the range 0ms to 30ms), and in the timeout handling it deasserts the VBUS charging, which effectively forms a VBUS pulse.

Note that the data line pulsing is required to be in the range 5ms to 10ms, if the OTG controller has hardware timer support and the OCD provides implementation for the hardware timer interface routines, then the data line pulsing will be accurate. However, if there is no hardware timer support, the system watchdog timer is used. In this case, the default sysClkRate of 60 is not accurate enough to get a 5ms to 10ms data line pulsing. It is recommended to set the sysClkRate to be at least 1000 in such a case.

No matter if there is automatic SRP support or not, once the SRP initiation is done, the b_srp_done is set to **TRUE** and the OTG state moves back to b_idle again.

5.5 b-device waits vbus to be valid

Once the OTG state moves from b_srp_init back to b_idle and the b_srp_done is **TRUE** (id pin also **TRUE**), the OTG Manager starts a **USBOTG_TIMER_b_srp_fail_tmr** timer, with timeout around 6000ms (6sec) (in the range 5sec to 6sec). If the VBUS raises above VBUS valid threshold within the timeout, then the timer is canceled. If the timer times out but VBUS still not valid, the OTG Manager could report the SRP failure to the user, and the user could request SRP again.

5.6 a-device detects srp request

Once the B-device initiates the SRP, the data line and/or the VBUS pulsing could be detected by the A-device. The A-device may trigger an interrupt, the OCD ISR should report the SRP detection to the OTG Manager.

5.7 a-device responses srp by driving vbus

In response to the SRP detection event, if the A-device user application could allow the bus to be used (a_bus_drop set to **FALSE**), then the OTG Manager will start driving the VBUS by calling **USBOTG_OCD_FUNCS::pOtgDrvVbusEnable( )**, and moves from a_idle state to a_wait_vrise, and up through to a_wait_bcon, then a_host if the B-device signals connection.

6 usb otg hnp implementation notes

This section describes the VxWorks USB OTG stack implementation for HNP protocol.

The Host Negotiation Protocol (HNP) allows the host function to be transferred between two directly connected OTG devices and eliminates the need for a user to switch the cable connections in order to allow a change in control of communications between the devices. HNP will typically be initiated in response to input from the user or an Application on the OTG B-device. HNP may only be implemented through the Micro-AB receptacle on a device. The A-device is always responsible for powering the USB interface regardless of whether it is acting in host or peripheral role.

At the start of a session, the A-device defaults to having the role of host. During a session, the role of host can be transferred back and forth between the A-device and the B-device any number of times, using HNP.

The HNP protocol is the key of the USB OTG to allow dynamical role switching.

The HNP procedure is described in this section(assume the A-device is currently working as a_host and B-device is working as b_peripheral).

6.1 b-device exhibits hnp capability to a-device

B-device indicates its HNP capability in its USB OTG descriptor bmAttributes HNP support bit field; This is done by B-device side Target Stack, with the knowledge of the existence of the OTG support, by checking if **USBOTG_OCD::pOTG** is set to non-**NULL** and **USBOTG_OCD::usbOtgEnabled** is set to **TRUE**. And then retrieve the **USBOTG_OCD::bmAttributes** to fill the OTG descriptor when a GetDescriptor(Configuration) is received. If we OTG 2.0 support is enabled, we will also return the OTG descriptor when GetDescriptor(OTG) is received from the host.

6.2 a-device learns hnp capability from b-device

A-device learns B-device HNP capability from the USB OTG descriptor while enumerating the B-device; This is done by the A-device side Host Stack. And the Host Stack will inform HNP capability to the OTG Manager through the OTG event **USBOTG_EVENT_ID_OTG_DEVICE_ENUMERATED**.

6.3 a-device enables b-device for hnp

A-device sends a SetFeature(b_hnp_enable) command through the default control pipe any time while it is working in the a_host state; Also it sets A-device side a_set_b_hnp_en flag to **TRUE** if the previous command successfully completed; Note that it is not required for the HNP procedure to happen immediately when the SetFeature(b_hnp_enable) is issued, it is merely to tell the B-device that it could consider that HNP is now allowed by the A-device and when the B-device detects a bus suspend later, the B-device can assume the HNP procedure has been actually started by the A-device, and it can follow the HNP procedure to proceed.

Note

Although SetFeature(b_hnp_enable) could be sent any time while the A-device is working as host, we do this just after the OTG device is enumerated.

6.4 b-device gets notified hnp to have been enabled

The B-device will also set its B-device side b_hnp_en flag to **TRUE** when it receives the SetFeature(b_hnp_enable) command; This is done by the B-device side Target Stack, informing the OTG Manager to set the flag, by raising OTG event **USBOTG_EVENT_ID_SET_OTG_FEATURE_RECEIVED**.

6.5 a-device actually starts hnp under user control

A-device finishes using the bus (in a_host), and tries to give a chance for the B-device to take over the bus (switch from current b_peripheral state to b_host state); This involves the following steps:

6.5.1 a-device user application gives up host role

A-device side OTG Manager suspends the bus by calling the Host Stack **usbHstSuspendDevice( )** function in response to user application call to **usbOtgHostRoleGiveup( )** routine; This causes the Host Controller to stop generating any SOF on the bus and thus there is no bus activity on the bus; The D+/D- data lines go into IDLE line state (3ms later it will be bus suspend); And the A-device goes into a_suspend state;

At this point in time the A-device OTG Manager will also start a watchdog timer a_aidl_bdis_tmr. This timer is started by the A-device when it enters the a_suspend state if HNP has been enabled (a_set_b_hnp_en = **TRUE**) (which should have already happened in previous steps). During the watchdog timer period **TA_AIDL_BDIS** (minimum 200ms) the A-device is expecting to detect B-device disconnection. If the A-device does not detect a disconnect before **TA_AIDL_BDIS** (a_aidl_bdis_tmout = **TRUE**), then the A-device is allowed to stop waiting for a disconnect and end the session (a_aidl_bdis_tmout triggers a_wait_vfall).

6.5.2 b-device detects a-device has started hnp, then disconnects from a-device

B-device will recognize the bus entering bus suspend state after 3ms of the D+/D- going into IDLE line state; Since the B-device has indicated its HNP capability to the A-device and b_hnp_en flag has been set, it will consider this bus suspend state entry as an indication from the A-device to have accepted and started the HNP role switch procedure; Note that there may be a suspend event (maybe interrupt) generated, the OTG Controller Driver should notify this event to the OTG Manager through raising the OTG event **USBOTG_EVENT_ID_SUSPEND_RESUME_DETECTED**. The OTG Manager will then proceed to cause a role switch as detailed below.

If the two connected devices were previously working in HS, the B-device will now turn up it D+ pull-up, which effectively makes the data lines go into FS (can be under software control or done by hardware automatically); If the two devices were already working in FS, the B-device D+ should have already been pulled-up; The B-device will now disconnect from the bus by removing the D+ pull-up. Note that this disconnection should happen in no more than 150ms (**TB_AIDL_BDIS** max) after the detection of the bus suspend; If not, the A-device may have turned off its VBUS and the session will be end.

The disconnection can be done by the hardware automatically or by software intervention;

a) If the disconnection is done by hardware automatically upon detecting bus suspend state, then that capability of auto-disconnection should be enabled through **USBOTG_OCD_FUNCS::pOtgAutoSusp2DisEnable( )**. By programming some register control bits so that when the suspend is recognized by the hardware can perform auto-disconnection.

Note that **USBOTG_OCD_FUNCS::pOtgAutoSusp2DisEnable( )** should be called by the OTG Manager in response to user application request on the B-device before the HNP actually takes place.

Note

B-device user application requests host role by calling the API routine **usbOtgHostRoleRequest( )**, which sets b_bus_req to **TRUE**. The user application may also request host role with ASAP set to **TRUE**, in which case the host request flag is set and it may force the host to give up host role if it is doing HNP polling.

Note

This API routine **usbOtgHostRoleRequest( )** and other APIs are encapsulated by **iosLib** interface, user applications should call them through IOCTL commands. See **USER APPLICATION IOCTL COMMANDS** for more details.

b) If the disconnection is not done by hardware automatically upon detection of bus suspend, for example, if the disconnection should be done by some Soft Connect capability of the hardware through direct software control, then the **USBOTG_OCD_FUNCS::pOtgAutoSusp2DisEnable( )** must be set as **NULL** to indicate there is no hardware assistance for "Suspend to Disconnect", which leaves the disconnection to be handled by the OTG Manager to call another interface **USBOTG_OCD_FUNCS::pOtgDpPullUpEnable( )** to control D+ pull-up on detection of bus suspend.

No matter what caused the B-device disconnection, the B-device has now turned off its D+ pull-up (and both D+/D- pull-down should also be asserted), so the data line now goes into SE0 state; The OTG Manager will move B-device OTG State Machine from previous b_peripheral state into b_wait_acon state;

At this point in time, the B-device OTG Framework will start a watchdog timer b_ase0_brst_tmr. This timer is used by B-device in the b_wait_acon state, to wait for the A-device to signal a connection (a_conn = **TRUE**). During this timer period **TB_ASE0_BRST** (3.125ms minimum), the B-device is expecting A-device to connect. If the A-device does not connect before **TB_ASE0_BRST** (b_ase0_brst_tmout = **TRUE**), then the B-device shall return to the b_peripheral state. Note that the **TB_ASE0_BRST** is specified as 3.125ms min in OTG 1.3, but is specified as 155ms min in OTG 2.0. Anyway, to cover the worst case, we will set the b_ase0_brst_tmr timeout period to be longer than 155ms, even **WAIT_FOREVER**.

6.5.3 a-device detects b-device disconnect, and connects to b-device as peripheral

A-device side Host Stack detects the B-device disconnection (although the A-device bus has been suspended);

Note that there is a strict timing requirement at this point. The A-device must transition to the peripheral role within 3ms (**TA_BDIS_ACON** max) after detecting the bus becoming SE0 (due to the B-device turning off its D+ pull-up to disconnect). The A-device must also turn on its A-device side D+ pull-up quickly at this stage (in the 3ms duration); The **TA_BDIS_ACON** max is specified in OTG 1.3 as 3ms, but in OTG 2.0 it is specified as 150ms. Anyway the A-device role switch and connection should happen as quickly as it can, ensure worst case of 3ms.

The A-device is still in a_suspend state at this point. Once the B-device disconnection is detected, the OTG Controller Driver should turn on the D+ pull-up immediately (maybe in the ISR directly) and switch the hardware into peripheral mode (if such a hardware mode switching is required). In fact, to meet the 3ms **TA_BDIS_ACON** max time requirement, some OTG Controllers may support some kind of hardware automatic assist, once enabled, the hardware will detect the "B-device disconnection during suspend" situation and switch the hardware into peripheral mode and assert the D+ pull-up automatically. The OTG specification (6.7.2 A-device becoming Peripheral) describes a case for this kind of automatic HNP response. If the hardware has such automatic HNP response, then **USBOTG_OCD_FUNCS::pOtgAutoDis2ConEnable( )** should be set as the routine to be used to enable/disable such feature. If the feature is always enabled, then this routine can just do nothing (but still needs to be set as non-**NULL**). If there is no such hardware automatic assist, then the **USBOTG_OCD_FUNCS::pOtgAutoDis2ConEnable( )** of the OCD should be set as **NULL**, which indicates to the OTG Manager to do the proper handling by the D+ pull-up interface **USBOTG_OCD_FUNCS::pOtgDpPullUpEnable( )**.

No matter if the hardware assist exists or not, once the B-device disconnection event is detected by the A-device Host Controller Driver, it should report it to the BusM through the Hub Class Diver Interrupt URB. The A-device Host Stack will handle the B-device disconnection normally (for example, free its class driver allocated resources and do any other device unloading stuff). This OTG Controller Driver ISR should notify the B-device disconnection to OTG Manager by raising OTG event **USBOTG_EVENT_ID_CONNECT_DISCONNECT_DETECTED**, and the OTG Manager will advance the OTG State Machine into a_peripheral state if it was in a_suspend before.

It might be desirable that we could switch from Host Stack into Target Stack at this point (when the B-device disconnection is detected). However, the following facts would also affect the design decision:

a) The stack switching would include firstly disable the Host Stack, and then enable the Target Stack. This process may have to be done in task context (maybe in the OTG Manager task context), the time taken to do this is not fully predictable (although we would try the best to minimize the operations taken in this process, we could not avoid the possibility of being preempted by other higher priority task in the system), thus chances are that when we finally successfully switch into the Target Stack, we may have missed the Bus Reset issued from the B-device.

b) Considering **TB_ACON_BSE0** (Max 1ms in OTG 1.3, Max 150ms in OTG 2.0), the B-device at this point will also in a critical time, that it needs to assert the Bus Reset as soon as it detects the A-device connection. The B-device side may very likely involve some kind of hardware assist, so that once the A-device signals the connection, the Bus Reset may be asserted immediately.

c) There is also a case that the B-device is truly physically disconnected (detached) while in the HNP process (after A-device suspends the bus) (e.g, the Mini-A plug or Mini-B plug removal, or simply the B-device fails HNP). In this case switching into Target Stack makes no sense.

So, one point to be made clear is that once the D+ pull-up is asserted, the B-device may immediately signal the Bus Reset to the A-device (quick as in a few us time). So the design decision here is that we do not perform the actual stack switching at the time when B-device disconnection is detected, instead we continue in the Host Stack until a Bus Reset is received.

This design decision requires a common ISR used by both Host and Target Controller Driver. Once the Bus Reset (mostly an interrupt) is seen, the common ISR notifies the OTG Manager of this event through rasing OTG event **USBOTG_EVENT_ID_BUS_RESET_DETECTED**. The OTG Manager responds to "Bus Reset event" by calling **usbOtgStackSwitch( )** to switch stack from Host Stack to Target Stack (OTG state should still be a_peripheral state);

Two important points should be noted here:

a) Note that once entering the Target Stack (through the Target Controller Driver **USBTGT_TCD::pTcdEnable( )** interface), the Target Controller Driver should report a "Bus Reset" (through callback) to the upper Target HAL (or Target Application) to bring the upper layers out of reset (like in normal TCD Bus Reset handling). In this case, the OTG Controller Driver common ISR will not deliver the "Bus Reset" event to the Target Controller ISR.

b) Note that the stack switch process should happen as quickly as possible. The Target Controller Driver should make sure it is prepared to receive the Setup packets, because after the Bus Reset, the B-device (b_host) may start to enumerate the A-device by reading its descriptors. However, this may not be as critical as the **TB_ACON_BSE0** or **TA_BDIS_ACON**, even if the Setup fails once, the B-device (b_host) may involve some kind of retry reset mechanism at this stage.

At this point in time, the A-device is in a_peripheral state and the Target Stack is working (Host Stack disabled).

6.5.4 b-device recognizes a-device connection, issues bus reset as b_host

B-device detects the data line J-state (D+ pull-up) before b_ase0_brst_tmr timer times out (we set **TB_ASE0_BRST** as more than 155ms) and takes this as A-device connection. B-device side OTG Controller Diver ISR will detect this A-device connection and notify the A-device connection event to the OTG Manager through the rasing OTG event **USBOTG_EVENT_ID_CONNECT_DISCONNECT_DETECTED**, the OTG Manager will take this notification to transition the OTG State Machine into b_host state (and also switch the stack from Target Stack to Host Stack).

At this point B-device (in b_host) Host Stack generates a bus Reset (SE0) to the A-device (which is now in a_peripheral) within 1ms (**TB_ACON_BSE0** max) upon detecting of A-device connection; Note that **TB_ACON_BSE0** max is specified in OTG 1.3 as

1ms, but in OTG 2.0 it is specified as 150ms. However, the specification doesn't seem to specify what will happen if the Reset doesn't happen in time. Anyway, we should respect the worst case requirement by asserting the Reset within 1ms.

The B-device starts to work as a normal host and A-device starts to work as a normal peripheral. The forward part of the HNP procedure is completed here.

6.6 b-device returns host role to a-device

B-device finishes using the bus, and try to return the host role to the A-device (switch from current a_peripheral state to a_host state); This involves the following steps:

6.6.1 b-device user application gives up host role

B-device side OTG Manager suspends the bus by calling the Host Stack **usbHstSuspendDevice( )** function in response to the B-device side user call to **usbOtgHostRoleGiveup( )**; This causes the Host Controller to stop generating any SOF on the bus and thus there is no bus activity on the bus; The D+/D- data lines go into IDLE line state (3ms later it will be bus suspend); And the B-device goes directly into b_peripheral state (Note that there is no such a thing as "b_suspend" state);

The B-device OTG Manager calls **usbOtgStackSwitch( )** to switch from Host into Target stack (OTG State Machine will also be changed from b_host into b_peripheral).

At this point in time, the B-device side Target Stack is working.

Optionally, the B-device may turn on its D+ pull-up when a FS idle condition is detected on the bus (Done by the B-device side Target Controller Driver).

6.6.2 a-device detects bus suspend, and then disconnects from b-device

A-device detects lack of bus activity for more than **TA_BIDL_ADIS** min (3ms) and turns off its D+ pull-up. Alternatively, if the A-device has no further need to communicate with the B-device, the A-device may turn off VBUS and end the session (the user application calls **usbOtgVbusPowerOff( )**).

Once the A-device detects the bus enters IDLE state while it is in a_peripheral state, it starts a watchdog timer a_bidl_adis_tmr. During the period of the timer, A-device is expecting to go into a_wait_bcon state. The A-device transitions from the a_peripheral state to the a_wait_bcon state if the a_bidl_adis_tmr times out (a_bidl_adis_tmout = **TRUE**). If the A-device detects more than **TA_BIDL_ADIS** min (155ms) of continuous idle (i.e. J_state), on the bus, then the A-device may transition to the a_wait_bcon state. If no activity is detected after **TA_BIDL_ADIS** max (200ms) the A-device shall transition back to the a_wait_bcon state.

Note that this a_bidl_adis_tmr is added in USB OTG 2.0, but it is listed in the a_peripheral state "output variable" part of the A-device state diagram. It seems from the state diagram that this timer should be started when the A-device enters the a_peripheral state; If the a_bidl_adis_tmr timer times out, the A-device will enter a_wait_bcon state; But the timeout value **TA_BIDL_ADIS** (B-Idle to A-Disconnect) is 155 ms (min) to 200 ms (max); So, if this timer is started once the a_peripheral state is entered, it means the A-device can at most stay in a_peripheral state for 200ms, and that doesn't make sense because the B-device (b_host) may need more time to work with the A-device (a_pheripheral). Seen from the name of this timer, it implies it should be started when the A-device detects B-device idles the bus (or suspend the bus) while the A-device is in a_peripheral state (and this is our choice of time point to start it). We should note that the "out put variables" in the OTG state diagram are not necessarily to be output immediately when entering the state, but may be at some point during the state.

Note

According to USB-IF explanation, the a_bidl_adis_tmr should be started once a_peripheral is entered, and the timer must be reset every SOF because if there is SOF, the bus is not idle so the timer should not timeout. This indicates we should reset the timer at around every 1ms, and mostly we should do it at SOF interrupt. However, SOF interrupt is not necessarily always enabled, and that is some overhead doing so in software. In this regard, the a_bidl_adis_tmr should ideally implemented by the OTG Controller hardware. From software perspective, we will still only to start the timer when we detect the bus suspend by the B-device.

A-device will recognize the bus entering bus suspend state after 3ms of the D+/D- going into IDLE state; A-device will consider this bus suspend state entry as an indication from the B-device to return the host role back (A-device goes from a_peripheral into a_wait_bcon); Note that there may be a suspend event (maybe interrupt) generated, the A-device side OTG Controller Driver common ISR which detects this suspend event should notify the A-device OTG Manager by rasing OTG event **USBOTG_EVENT_ID_SUSPEND_RESUME_DETECTED**; The OTG Manager will then proceed to cause a role switch as detailed below.

If the two connected devices were previously working in HS, the A-device will now turn up it D+ pull-up, which effectively makes the data lines go into FS (can be under software control or done by hardware automatically); If the two devices were already working in FS, the A-device D+ should have already been pulled-up; The A-device will now disconnect from the bus by removing the D+ pull-up. Note that this disconnection should happen in no more than 200ms (**TA_BIDL_ADIS** max) after the detection of the bus suspend;

The disconnection can also be done by the hardware automatically or by software intervention;

No matter what caused the A-device disconnection, the A-device has now turned off its D+ pull-up (and D+/D- pull-down should also be turned on), so the data line now goes into SE0 state; The OTG Manager will switch into host stack by calling **usbOtgStackSwitch( )**; At this time the A-device Host Stack is working and Target Stack is disabled. The OTG Manager will also move its OTG State Machine from previous a_peripheral state into a_wait_bcon state;

Note that even the A-device doesn't disconnect, it will still be put into a_wait_bcon state once the a_bidl_adis_tmr times out. If the Mini-A plug is removed (id float) or the A-device application doesn't want to use the bus (a_bus_drop set to **TRUE**), it will enter a_wait_vfall state.

The A-device starts to work as a normal host and B-device starts to work as a normal peripheral. The revert part of the HNP procedure is completed here.

Note that the Host Controller Driver or the Target Controller Driver will always report device events (e.g, connect, disconnect, suspend, resume, reset) normally, and that at the proper points the Host Stack or Target Stack will communicate with the OTG Manager so that the OTG State Machine can be properly advanced.

The OTG Controller Driver doesn't need to monitor or report HNP status. These OTG Notifier functions should be enough to make the OTG Manager aware of the HNP success or failure.

7 usb otg protocol compliance implementation notes

This section describes the VxWorks USB OTG Stack implementation details with regard to OTG protocol checklist as outlined in the **OTG Compliance Checklist**.

7.1 otg descriptor

For an OTG B-device or SRP capable peripheral, an OTG descriptor will be included in the response to all valid GetDescriptor(Configuration) requests. Note that this is the requirment of OTG 1.3. In OTG 2.0, B-device that supports either HNP or SRP or ADP shall respond to a GetDescriptor(OTG) request with its OTG descriptor.

Note

For the current implementation we may only include the OTG descriptor along with the Configuration descriptor. If a GetDescriptor(OTG) is received, we should STALL it (unless when we fully support OTG 2.0).

7.2 srp initial conditions

For an OTG B-device or SRP capable peripheral, the SRP can only be initiated when the voltage on VBUS is thought to be below **VB_SESS_END** and the bus has been in the SE0 state for at least 2ms (**TB_SE0_SRP**).

Note

The OTG Controller Driver should be able to provide VBUS state and the line state (such as SE0). OTG Controller Driver may need to access some PHY register to get these information (e.g, the ULPI DEBUG register provides the line state information).

Note

We may need to create some generic PHY driver, but these are to be encapsulated by the OTG Controller Driver to provide functionalities as reporting VBUS state and line state.

7.3 indication of srp in progress

For an OTG B-device or SRP capable peripheral, before SRP is initiated, an indication should be made to the user that communications are trying to be established.

Note

The user application may call **usbOtgVbusStateGet( )** to get the current session state and inform the user of the progress information. The user can also be notified by OTG Manager notification mechanism.

Note

We have some choices to implement user notification:

a) The user may install a callback through **USBOTG_IOCTL_CALLBACK_INSTALL**; the user callback will be called once b_srp_init is entered, with a message ID of **USBOTG_USR_MSG_ID_SRP_ACTIVE**.

b) The user application may create a task, and a msgQ to get event notifications from the OTG Manager.

c) Use select interface of the **iosLib**, add the message to the ring buffer, and notify user of message data availability, and let the user read the message.

d) Use signal interface

Currently we just implemented the user callback mechanism; we may choose to add the select interface.

7.4 indication of srp failure

For an OTG B-device or SRP capable peripheral, an indication should be made to the user that SRP has failed if a session has not been started within **TB_SRP_FAIL** of initiating SRP.

Note

When the B-device user application calls **usbOtgVbusPowerOn( )** to request SRP, it returns immediately after the b_bus_req request flag is set, it is up to the OTG Manager to wait for the SRP inital condition to be met and actually starts SRP. Once the SRP is initiated, the USBOTG_TIMER_b_srp_fail_tmr will be started, and if the timer times out but VBUS still not valid, the SRP is marked as failure.

Note

The user may install a callback through **USBOTG_IOCTL_CALLBACK_INSTALL**; the user callback will be called once USBOTG_TIMER_b_srp_fail_tmr times out but VBUS still not valid, with a message ID of **USBOTG_USR_MSG_ID_SRP_FAIL**.

7.5 srp init time requirement

For an OTG B-device or SRP capable peripheral, the SRP procedure should take less than 100ms (**TB_SRP_INIT**).

Note

This is the time required for B-device to complete its SRP activities, this may involve writing some register bits to initiate the hardware to trigger Data-line pulse (and VBUS pulse if implemented). Most of this is accomplished by hardware and thus is not a big software concern. If there has to be software controlled data line and/or VBUS pulsing, two timers will be used to trigger the pulses with timeout value set to meet the pulse widths required, but the total time doesn't exceed the **TB_SRP_INIT**. See the section **USB OTG SRP IMPLEMENTATION NOTES** for details.

7.6 indication of failure to raise vbus

For an OTG A-device, an indication should be made to the user that the B-device is not supported when VBUS takes longer than **TA_VBUS_RISE** to reach **VA_VBUS_VLD**.

Note

When this situation happens, the OTG Manager will instruct the Controller Driver to power off the VBUS and show a warning message that the attached B-device can not be supported. The user application callback can be called with message ID of **USBOTG_USR_MSG_ID_VRISE_FAIL**.

7.7 choice on debounce interval

For an OTG A-device, there are two possible debounce interval to be used (short **TA_BCON_SDB** [Min 2.5us], or long **TA_BCON_LDB** [Min 100ms]). The details are as below:

If b_conn is **FALSE**, it is set to **TRUE** if the B-device pulls D+ or D- high for longer than the debounce interval. The debounce interval varies depending on how the a_wait_bcon state was entered, and how long it has been since either D+ or D- has been pulled high.

1) If the a_wait_bcon state was entered from the a_peripheral, a_host or a_suspend states, then the short debounce interval (**TA_BCON_SDB**) is allowed.

2) If the a_wait_bcon state was entered from the a_wait_vrise state then the long debounce interval (**TA_BCON_LDB**) is required.

The A-device is only allowed to apply the short debounce to b_conn in a window of time. If the A-device stays in the a_wait_bcon state for longer than **TA_BCON_SDB_WIN** max (100ms), then the long debounce interval applies no matter how the a_wait_bcon state was entered. Note that if the state machine is timing the short debounce interval, changes to D- and D+ do not affect the setting of b_conn if they occur before **TLDIS_DSCHG** min.

Note

Since **TA_BCON_SDB** is only "allowed", not "required", we will always use **TA_BCON_LDB** for the A-device.

Note

However, to meet the 1ms limitation of **TB_ACON_BSE0**, during HNP, when B-device detects A-device connection, we will issue the Bus Reset immediately once the connection is detected (the BusM debounce will be dummy process in this case). This doesn't affect the debounce internal of the A-device, because this is applied for the B-device.

7.8 srp capable host debounce using **ta_bcon_ldb**

For an SRP capable host, when waiting for a B-device connection, the connection can be debounced using **TA_BCON_LDB**.

Note

As discussed above, we will not use **TA_BCON_SDB**, and **TA_BCON_LDB** is used always.

7.9 **ta_bcon_sdb** debounce interval starting point

For an OTG A-device, if waiting for a B-device connection and using a debounce of **TA_BCON_SDB**, the debounce interval should be started after **TLDIS_DSCHG** (Local Disconnect to Data Line Discharge Time).

Note

This doesn't apply to our situation since we don't use **TA_BCON_SDB**.

7.10 b-device hnp suspend to disconnect time (**tb_aidl_bdis**)

For an OTG B-device, when operating as a peripheral, the HNP should only be started (with a disconnect) if the bus has been idle for more than 5ms (**TB_AIDL_BDIS** min).

Note

Both OTG 1.3 and 2.0 specify **TB_AIDL_BDIS** as (Min 4ms, Max 150ms). However the compliance checklist specifies as 5ms which gives some 1ms slit. Also, most OTG Controller will handle this in hardware and thus not a big software concern (meaning once HNP is enabled on B-device, when detecting A-device suspending the bus, the B-device hardware will start the disconnection automatically within **TB_AIDL_BDIS**).

7.11 hs b-device switching to fs before hnp starting

For a HS capable OTG B-device, operating as a HS peripheral, when no bus activity is detected for 5 ms (**TB_AIDL_BDIS**), FS mode should be entered, with the D+ pull-up resistor turned on, and the D+ line checked to be high for at least 1ms (**TB_FS_BDIS** min) before starting HNP.

Note

This should be a OTG Controller hardware concern for most USB OTG Controllers. For fully software controlled HNP, we have to conform to the timing requirements.

7.12 hs b-device responding to a-device bus reset during hnp

For a HS capable OTG B-device, after operating as a HS peripheral and entering FS mode so that HNP can be started, if the D+ line is not high within TWTRSTHS of turning on the D+ pull-up, then a HS chirp should be started.

Twtrsths

Time a device must wait after reverting to full-speed before sampling the bus state for SE0 and beginning the high-speed detection handshake.

Note

Starting HS B-device HS chirp means response to the A-device Reset. This maybe due to the fact that after the A-device suspends the bus, and at the point the B-device wants to start HNP, the A-device wants to exit the suspend and Reset the bus (may be due to user intervention).

Note

This should be a OTG Controller hardware concern.

7.13 b-device starts hnp but a-device doesn't respond the hnp

For an OTG B-device, if after initiating HNP and 3.125ms (**TB_ASE0_BRST** min) of SE0 is seen, the B-device should return to being a peripheral and start to process the reset before (**TB_ASE0_BRST** max).

After the B-device turns off its pull-up resistor the bus will be pulled to the SE0 state by the pull-downs on the A and B devices. This SE0 is not a resume indication. If the SE0 persists for more than **TB_ASE0_BRST** min, then the A-device is not responding to the HNP request from the B-device and the B-device shall treat the SE0 on the bus as a bus reset indication. This takes the B-device back to the b_peripheral state.

Note

This adds a design challenge for the HNP timing. The following facts contibute to the design choice for when to do the controller mode and stack switching:

1) There is a strict timing requirement for B-device **TB_ACON_BSE0** (Max 1ms) (A-Connect to B-SE0 Time). Meanwhile, there is also a requirement for the A-device to meet **TA_BDIS_ACON** (Max 3ms) (B-Disconnect to A-Connect Time). This means once the B-device disconnects, the A-device may soon (Max 3ms) signal connection to the B-device, and the B-device should immediately respond to the A-device connection very quickly in Max of 1ms (to assert the Bus Rest to the A-device). With the strict B-device timing constraints, it might be desirable that the OTG Manager switch into host stack as soon as possible by calling **usbOtgStackSwitch( )** just after B-device disconnects from the A-device;

2) However, considering the case that the A-device may not actually respond to HNP (not to signal connection to the B-device, but to signal Bus Reset to the B-device while the B-device is expecting connection), we may need to wait around 3.125ms (**TB_ASE0_BRST** min) to ensure there is no SE0 from the A-device and then actually starts the stack switching (so that we are still in the Target Stack and can response to the Reset in this condition).

3) The above two scenarios indicates that we need to monitor both for the A-device connection case and the A-device reset (or resume) cases during the 3ms time (or around 3.125ms). To meet both goals, the design decision is as below:

3.1) When the B-device sees the bus suspend and disconnects (starting HNP), we do not switch stack immediately (but the controller mode can be changed); instead, at this point, the B-device OTG Manager starts a watchdog timer b_ase0_brst_tmr (and move the B-device OTG state into b_wait_acon). The b_ase0_brst_tmr timer is used by B-device in the b_wait_acon state, to wait for the A-device to signal a connection (a_conn = **TRUE**). During this timer period **TB_ASE0_BRST** (3.125ms minimum), the B-device is expecting A-device to connect. Note that the **TB_ASE0_BRST** is specified as 3.125ms min in OTG 1.3, but is specified as 155ms min in OTG 2.0. Anyway, to cover the worst case, we will set the b_ase0_brst_tmr timeout period to be longer than 155ms, even **WAIT_FOREVER**.

3.2) If the A-device does not connect to the B-device before **TB_ASE0_BRST** (b_ase0_brst_tmout = **TRUE**), then the B-device shall return to the b_peripheral state.

3.3) If the A-device signals Bus Reset (SE0), or signals resume (J-to-K) (if the B-device is in the b_wait_acon state, then only a J-to-K transition is treated as a resume), then the B-device will return to b_peripheral state.

3.4) If during the b_ase0_brst_tmr period, the A-device connects to B-device (the A-device actually reponds to the HNP), then the B-device transition to b_host state. To guarantee the **TB_ACON_BSE0** (Max 1ms) limit, once the B-device recognizes the A-device connection (while still in b_wait_acon state and the stack is still Target Stack), the OTG Controller Driver ISR should assert the Bus Reset to the A-device immediately (can be in the ISR directly). Once the Bus Reset is done, the A-device connection event is notified to the OTG Manager through **usbOtgNotifyConnectDisconnect( )**, and then the OTG Manager can call **usbOtgStackSwitch( )** to switch the stack from Target Stack to Host Stack and the OTG state is also moved from b_wait_acon to b_host state.

3.5) At this time the B-device Host Stack is working and Target Stack is disabled. Once the Host Stack is entered, the Host Controller Driver would construct a dummy Connection Change and report to the BusM through Hub Class Driver Interrupt URB

(before this the Host Controller Driver could issue a Bus Reset once again, a Bus Reset at this point doesn't cause any harm). Since the Bus Reset has been issued to the A-device, the b_host Host Stack could proceed with normal device enumeration.

7.14 b-device starts hnp and responds to the hnp by connecting to b-device

For an OTG B-device, if after initiating HNP and the D+ line is high after 25us (**TLDIS_DSCHG** min) for at least 2.5us (**TB_ACON_DBNC** min) after the B-device turns off its D+ pull-up, the B-device should become the host.

If the B-device is in the b_wait_acon state, the B-device sets a_conn **TRUE** if the B-device detects a connection from the A-device. In order to detect a connection from the A-device, the B-device must insure that there is no residual voltage on the D+ line from the B-device's pull-up (**TLDIS_DSCHG**). When the B-device has qualified the high level on the D+ line as being from the A-device, and the level has been present for at least **TB_ACON_DBNC** min then a_conn is set to **TRUE**.

Note

(See Also Note 13). While in the b_wait_acon state, when detected A-device connectin (D+ pull-up) (still in Target Stack), we will issue Bus Reset immediately to meet **TB_ACON_BSE0** (Max 1ms) limit and then notify the OTG Manager to call **usbOtgStackSwitch( )** to switch the stack from Target Stack to Host Stack and the OTG state is also moved from b_wait_acon to b_host state.

Note

An OTG device is required to operate as a FS or HS peripheral. Therefore, a B-device shall only accept a connection from an A-device when D+ is pulled up. If the B-device detects a high on the D- line after disconnecting, this shall be interpreted as resume signaling from the A-device.

7.15 b-device in b_host driving bus reset

For an OTG B-device, after successfully becoming host, a bus reset asserted within 1ms (**TB_ACON_BSE0** max) of detecting the A-device connect.

Note

(See Also Note 13). Once the B-device recognizes the A-device connection (while still in b_wait_acon state and the stack is still Target Stack), the OTG Controller Driver ISR should assert the Bus Reset to the A-device immediately (maybe in the ISR directly).

7.16 b-device accepting otg setfeature commands from a-device

For an OTG B-device, SetFeature(b_hnp_enable), SetFeature(a_hnp_support) and SetFeature(a_alt_hnp_support) should all be accepted in the Default, Address and Configured states.

It should be noted that the USB 2.0 specification [USB2.0] states that setting a feature in the Default state for other than test modes is unspecified. But OTG specification adds to the list of features that can be set in the Default state.

Note

The OTG event **USBOTG_EVENT_ID_SET_OTG_FEATURE_RECEIVED** is reported by the Target Stack to notify the OTG Manager to set the features.

Note

Allowing setting these features in these states inidicate that HNP could happen in these states. However VxWorks OTG Stack will only issue these OTG SetFeature commands after setting the device address.

7.17 otg feature clear time points

For an OTG device, the features b_hnp_enable, a_hnp_support and a_alt_hnp_support are cleared on a bus reset and at the end of a session but not with a **ClearFeature( )** command.

Note

The OTG Manager will clear these features when these events happen:

- **USBOTG_EVENT_ID_BUS_RESET_DETECTED**

- **USBOTG_EVENT_ID_VBUS_STATE_CHANGED**

- in **usbOtgIdVbusProbe( )** when it finds the VBUS is below SessEnd

7.18 a-device setfeature(b_hnp_enable) only to directly connected b-device

For an OTG A-device, a SetFeature(b_hnp_enable) command should only be sent to a directly connected B-device (no intervening hubs).

Note

The Host Stack USBD will determine the tier of the connected device and make sure the device connected on the root hub port is not a hub before reporting to the OTG Manager of a B-device connection. Only when a normal device is connected on the root port and that it is determined to be OTG HNP capable device through the OTG descriptor, we can issue the SetFeature(b_hnp_enable) command (and also set the a_set_b_hnp_en flag) with the instructuon from the OTG Manager.

7.19 a-device setfeature(a_hnp_support) time point

For an OTG A-device, a SetFeature(a_hnp_support) command should only be sent between the start of a session and the selecting of an OTG B-device configuration.

Note

Before putting the legacy B-device into a configuration, the A-device has the following three options with regards to the b_hnp_enable and a_hnp_support features:

- set the b_hnp_enable feature - set the a_hnp_support feature but not the b_hnp_enable feature - set neither the b_hnp_enable nor a_hnp_support features

If the b_hnp_enable feature is set, the B-device is allowed to do HNP, regardless of whether or not the a_hnp_support feature is set.

If the a_hnp_support feature is set, but the b_hnp_enable feature is not set, then it is likely that b_hnp_enable will be set later when the A-device is finished using the bus.

If neither the b_hnp_enable or a_hnp_support features are set before the A-device puts the B-device in a non-default configuration, then the B-device may indicate to the user that HNP is not supported through the current connection.

Note

A notification message will be sent to the user application callback, informing **USBOTG_USR_MSG_ID_HOST_NO_HNP**.

7.20 a-device connecting to b-device, responding hnp from b-device

For an OTG A-device, if it was successful in setting b_hnp_enable during the current session and a disconnect was detected during suspend, it's D+ pull-up resistor should be turned on within 3ms (**TA_BDIS_ACON**) of detecting the disconnect.

Note

This is another critical timing requirement enforced by the USB OTG specification.

The A-device is still in a_suspend state at this point. Once the B-device disconnection is detected, the OTG Controller Driver should turn on the D+ pull-up immediately (maybe in the ISR directly) and switch the hardware into peripheral mode (if such a hardware mode switching is required). In fact, to meet the 3ms **TA_BDIS_ACON** max requirement, some OTG Controllers may support some kind of hardware automatic assist, once enabled, the hardware will detect the "B-device disconnection during suspend" situation and switch the hardware into peripheral mode and assert the D+ pull-up automatically. The OTG 1.3 specification (6.7.2 A-device becoming Peripheral) describes a case for this kind of automatic HNP response.

No matter if the hardware assist exists or not, once B-device disconnection event is detected by the A-device Host Controller Driver, it should report it to the BusM through the Hub Class Diver Interrupt URB. The A-device Host Stack will handle the B-device disconnection normally (for example, free its class driver allocated resources and do any other device unloading stuff).

7.21 a-device as peripheral responding to b-device bus suspend (return host role)

For an OTG A-device, after detecting 3ms (**TA_BIDL_ADIS** min) of continuous idle whilst acting as a peripheral, the A-device should disconnect within 197ms (**TA_BIDL_ADIS** max - **TA_BIDL_ADIS** min).

The A-device transitions from the a_peripheral state to the a_wait_bcon state if the a_bidl_adis_tmr times out (a_bidl_adis_tmout = **TRUE**). If the A-device detects more than **TA_BIDL_ADIS** min of continuous idle (i.e. J_state), on the bus, then the A-device may transition to the a_wait_bcon state. If no activity is detected after **TA_BIDL_ADIS** max the A-device shall transition back to the a_wait_bcon state.

Note

The A-device hardware should detect the bus IDLE for more than 3ms as a bus suspend, and OTG Controller Driver ISR (A-device in a_peripheral) should report to the OTG Manager as **USBOTG_EVENT_ID_SUSPEND_RESUME_DETECTED**.

Note

Once detecting the bus suspend while in a_peripheral, the OTG Manager will start a watchdog timer a_bidl_adis_tmr (added in OTG 2.0). The timer triggers at **TA_BIDL_ADIS** (Min 150ms, Max 200ms).

7.22 hs capable otg a-device responding to b-device bus reset

For a HS capable OTG A-device, if a bus reset is detected whilst acting as a peripheral, the high-speed detection handshake should be started.

Note

This should be handled by the OTG Controller hardware (there maybe some hardware configurable register bits that allows enable or disable of the HS support). Anyway this is not a big software concern.

7.23 a-device starts session upon detecting mini-a plug

For an OTG A-device, the device can initiate a session upon the insertion of a Mini-A plug.

Note

Normally for OTG A-device, we can keep the VBUS powered off (session not in progress) even there is a B-device insertion, and require B-device to use SRP to request us to power on the VBUS. However, it is very likely that a non-OTG capable device will be connected in the other end of the USB cable, and that device may need the VBUS so that it could signal the device connection to the A-device. For this purpose, we may need to power on the VBUS at the insertion of the Mini-A plug (there will be ID pin state change). We will go through the a_idle -> a_wait_vrise -> a_wait_bcon process, and once the a_wait_bcon state is entered, the a_wait_bcon_tmr is started, on timeout of this timer, we can finally power off the VBUS if there is no device connection signaled. This way, it also saves the SRP time if the other end of the USB cable is actually a OTG SRP capable device.

7.24 a-device supporting srp

For an OTG A-device, although we can initiate a session upon the insertion of a Mini-A plug, the device can support SRP.

Note

The SRP and initiation of a session upon the insertion of a Mini-A plug doesn't conflict with each other.

7.25 a-device supporting remote wakeup

For an OTG A-device, the device should support remote wakeup operation when acting as a host.

Note

If USB system wishes to place the bus in the Suspended state, it commands the Host Controller to stop all bus traffic, including SOFs. This causes all USB devices to enter the Suspended state. In this state, the USB system may enable the Host Controller to respond to bus wakeup events. This allows the Host Controller to respond to bus wakeup signaling to restart the host system.

Note

The USB stack handling of Suspend/Resume is enhanced to support OTG HNP, including Remote Wakeup.

7.26 a-device srp response time

For an OTG A device, a session should be always started within 30 seconds of a valid SRP.

Note

After initiating SRP, the B-device is required to wait at least **TB_SRP_FAIL** min (5 sec) for the A-device to respond. After this the B-device may inform the user that the communication attempt has failed. For this reason, it is recommended that the A-device respond to SRP in less than **TA_SRP_RSPNS** max (4.9 sec). The minimum response from the A-device is to turn on VBUS and generate a bus reset.

Note

We may need to power on the VBUS as soon as possible, within 5 seconds.

7.27 a-device vbus valid time

When an OTG A-device starts a session, the VBUS should always valid for at least **TA_WAIT_BCON** (1 second).

Note

Once a_wait_bcon is entered, a watchdog timer a_wait_bcon_tmr is started. This timer is used by an A-device in the a_wait_bcon state to wait for the B-device to signal a connection, (b_conn = **TRUE**). If the B-device does not connect before **TA_WAIT_BCON**, (a_wait_bcon_tmout = **TRUE**), then the A-device is allowed to stop waiting for a connection.

Note

We may set the **TA_WAIT_BCON** to be **WAIT_FOREVER** (or user configurable), but should respect the **TA_WAIT_BCON** to be Min 1 second.

7.28 a-device bus reset time point

For an OTG A-device, a reset should be always issued within 30 seconds of a B device connect.

Note

(See Also Note 20). With the implementation of Note 20, this is not a software concern anymore (when a connection is detected, we issue Bus Reset immediately).

7.29 a-device set feature hnp_enable time point for unsupported b-device

For an OTG A-device, a set feature hnp_enable command should be issued within 30 seconds of the connection of an unsupported B-device.

Note

We should check the OTG descriptor to see if the B-device supports HNP and then if the B-device can not be supported by the A-device, then we should set feature hnp_enable command immediately, 30 seconds is enough to allow this determination.

7.30 a-device bus suspend time point for unsupported b-device

For an OTG A-device, the bus should be suspended within 30 seconds of bus reset with an unsupported B device acknowledges the set feature hnp_enalbe command.

Note

Once we set feature hnp_enable command to the B-device, we should start to suspend the bus immediately (since we can not work with the device), 30 seconds is enough to allow this determination.

Note

We will indicate to the user application the B-device cannot be supported by the A-device by **USBOTG_USR_MSG_ID_DEV_UNSUPPORTED**. The user application should decide if it will load a new driver or use ioctl **USBOTG_IOCTL_HOST_GIVEUP** to call **usbOtgHostRoleGiveup( )** and start the HNP.

7.31 a-device aborting hnp

The OTG A-device could abort HNP after suspending when interacting with an unsupported device.

Note

This may be determined by the user application (for example, the user application on the A-device side could load a class driver that may support the device, thus it could request to abort the HNP).

7.32 a-device should enumerate successfully as a peripheral following hnp

The OTG A-device should always enumerate successfully as a peripheral following an HNP transition.

Note

(See Note 20). We should be able to be enumerated successfully.

7.33 a-device respond to remote wakeup signaling without enabling remote wakeup

The OTG A-device should respond to remote wakeup signaling by driving resume for at least 20 milliseconds even if it has not sent an enable remote wakeup command to the downstream device.

In a_suspend state, the A-device transitions to the a_host state if either a_bus_req is asserted, or if the B-device signals a resume by putting a K state on the bus, even if the remote_wakeup feature has not been enabled.

Note

(See Also Note 25). We should not depend on the downstream device remote wakeup enabled or not.

7.34 b-device hnp response time point

For an OTG B-device, when operating as a peripheral, the HNP should be started (with a disconnection) within 150 ms of the bus being idle (**TB_AIDL_BDIS** max).

Note

In the HNP process, B-device detects that bus is idle for more than **TB_AIDL_BDIS** min and begins HNP by turning off pull-up on D+ (If the bus was operating in HS mode, the B-device will first enter the full-speed mode). This allows the bus to discharge to the SE0 state.

Most OTG Controllers peform this in hardware. When the user application calls **usbOtgHostRoleRequest( )** to request host role for B-device, the OTG Manager will call **USBOTG_OCD_FUNCS::pOtgAutoSusp2DisEnable( )** of the OTG Controller Driver to make the OTG Controller to be prepared for HNP procedure to start; When the Host suspends the bus, the hardware disconnects automatically. So this time limitation would mostly be assured by hardware.

7.35 b-device bus suspend time point for unsupported a-device

The OTG B-device end the session within 30 seconds of becoming host for the unsupported device following an HNP transition.

Note

(See Also Note 30) When B-device finds out it can not support the A-device after HNP, it should suspend the bus as soon as possible (30 seconds should be enough). Following this, the A-device could transition to power off the VBUS when it finds both ends can not use the peer device each other.

8 otg timing constraints and design challenges

This section describes the various OTG specific timing constraints and the challenges they put on for designing a generic USB OTG Stack, it also describes how VxWorks USB OTG Stack solves the challenges.

The USB OTG specification has some timing constraints that must be met by compliant driver software. On some events triggered by the OTG hardware, the software must response to them within limited time defined by the OTG specification, otherwise the system may not perform correctly.

8.1 **thost_req_poll** (min 1sec, max 2sec) - hnp polling period for host request flag

In OTG 2.0 with HNP polling, whenever there is an active session between two OTG devices the host is required to execute a GetStatus(OTG) with a frequency of **THOST_REQ_POLL** in order to determine the state of the Host request flag as defined in the OTG status information. After the host has detected that the Host request flag is set to one, the host shall allow the peripheral to take the host role within **THOST_REQ_SUSP**. If the Host request flag is reset to zero when an A-device has finished its activities it can end the session immediately.

Note

We use the OTG Manager task, with a parameter change notification semaphore, which normally waits for the input parameters to be changed by the event notification or from user application requests, but the timeout value is set to be around 1.5 sec, which meets the **THOST_REQ_POLL** time interval. Once the semaphore times out, it checkes if there is active OTG session between the two connected devices, and if the two devices are capable of doing HNP with each other, then it initiates the HNP polling by **usbOtgHnpPolling( )**.

8.2 **thost_req_susp** (max 2sec) - time from detection of b-device host request until suspend

In OTG 2.0 with HNP polling, the A-device is required to set b_hnp_enable feature and suspend the bus within **THOST_REQ_SUSP** when it determines that the B-device wishes to become host (host_req_flag = **TRUE**).

Note

Once we detect the host request flag, we will notify the user application with a message ID of **USBOTG_USR_MSG_ID_DEV_REQ_HOST**. The user application decides if it want to give up the host role, and if so, it uses **USBOTG_IOCTL_HOST_GIVEUP** ioctl to call **usbOtgHostRoleGiveup( )** to give up its host role. The callback gives the user application a chance to do some clean up work, but it is the user application's responsibility to keep the Max 2sec time requirement to be met.

Note

From the B-device side, if it uses the host request flag to request host role (by calling **usbOtgHostRoleRequest( )** with **ASAP** set to **TRUE**), then that routine will report a notification message ID of **USBOTG_USR_MSG_ID_REQ_HOST_RISKY** to inform the user that he should be take his own risk in doing so.

We have warned you that this is risky, take your own risk in doing so!

8.3 **tb_data_pls** (min 5ms, max 10ms) - data-line pulse time

To indicate a request for a new session using the data-line pulsing SRP, the B-device waits until the initial conditions are met and then turns on its D+ pull-up resistor for a period within the range specified by **TB_DATA_PLS**.

Note that the above **TB_DATA_PLS** requirement has put limitations for both A-device and B-device:

8.3.1 for b-device initating srp

- If the OTG Controller can perform SRP by hardware automatically under software control (for example, by setting a register bit), then the timing is totally controlled by hardware and software can do nothing.

- If the SRP must be controlled by software, for example, to set a register bit to pull-up D+ for **TB_DATA_PLS** of time, and then clear the register bit to disable the pull-up on D+, then there must be a timer that guarantee the pull-up on D+ is disabled in between 5ms and 10ms. Note that a software delay using **taskDelay( )** may not work because the task maybe preempted by other higher priority tasks and cause the **TB_DATA_PLS** to be expired.

Note

In the software controlled SRP case, we use a timer to control the pulse width. If the OTG Controller has hardware timer support, it should implement **USBOTG_OCD_FUNCS::pOtgTimerStart( )/pOtgTimerStop( )** to make the timing more accurate. If not, we fall back to use OS provided watch dog timer, in this case, we recommend to set the sysClkRate to at least 1000 so that the watchdog is able to provide 1ms accuracy.

8.3.2 for a-device detecting srp

Most OTG Controllers detect SRP by hardware. An A-device shall detect SRP if D+ goes high (while VBUS is off). Since the D+ data-line pulse is characterized by both its rising and falling edges; both edges shall be seen before SRP is reported by the A-device. For this reason, the detection of offending device (D+ pull-up longer than 10ms while VBUS is off) could only be done by hardware. The hardware may not report SRP detection if such a device is detected (but it doesn't harm if it reports SRP in this case, because in response to the SRP is to provide VBUS, and providing VBUS for such self-powered USB devices at connection should be **OK**.)

8.4 **tb_srp_fail**, **ta_srp_rspns** and **tb_srp_init**

- **TB_SRP_FAIL** (Min 5sec, Max 6sec) - SRP Fail Time of B-device

- **TA_SRP_RSPNS** (Max 4.9sec) - SRP Response Time of A-device

- **TB_SRP_INIT** (Max 100ms) - SRP Initiate Time (removed from OTG 2.0)

After initiating SRP, the B-device is required to wait at least **TB_SRP_FAIL** min for the A-device to respond. After this the B-device may inform the user that the communication attempt has failed. For this reason, it is recommended that the A-device respond to SRP in less than **TA_SRP_RSPNS** max. The minimum response from the A-device is to turn on VBUS and generate a bus reset.

The B-device is required to complete its SRP activities in less than **TB_SRP_INIT** max. On entry to this state, the B-device shall provide an indication to the user that it is trying to establish communications with the A-device and a timer is started. This timer will continue to run until the B-device enters the b_peripheral state. If the timer reaches a vendor specific limit between **TB_SRP_FAIL** min and **TB_SRP_FAIL** max the B-device will indicate to the user that the A-device did not respond.

See **USB OTG SRP IMPLEMENTATION NOTES** for more details.

Note

We will report **USBOTG_USR_MSG_ID_SRP_ACTIVE** and **USBOTG_USR_MSG_ID_SRP_FAIL** at these specific points.

8.5 **tb_aidl_bdis** (min 4ms, max 150ms) - a-idle to b-disconnect time

In the HNP process, B-device detects that bus is idle for more than **TB_AIDL_BDIS** min and begins HNP by turning off pull-up on D+ (If the bus was operating in HS mode, the B-device will first enter the full-speed mode). This allows the bus to discharge to the SE0 state.

Most OTG Controllers peform this in hardware. When the user application calls **usbOtgHostRoleRequest( )** to request host role for B-device, the OTG Manager will call **OtgAutoSus2DisEnable( )** of the OTG Controller Driver to make the OTG Controller to be prepared for HNP procedure to start; When the Host suspends the bus, the hardware disconnects automatically. So this time limitation would mostly assured by hardware.

See **USB OTG HNP IMPLEMENTATION NOTES** for more details.

8.6 **tb_acon_bse0** and **ta_bidl_adis**

- **TB_ACON_BSE0** (Max 1ms in OTG 1.3, Max 150ms in OTG 2.0) - A-Connect to B-SE0

- **TA_BIDL_ADIS** (Min 3ms in OTG 1.3, Min 155ms in OTG 2.0, Max 200ms)

While during the b_wait_acon state, if the A-device signals a connection (a_conn = **TRUE**) before the b_ase0_brst_tmr expires, then the B-device transitions to the b_host state. The B-device shall be able to detect the connect from the A-device and transition to the b_host state within **TB_ACON_BSE0** max after D+ is detected to be high at the B-device.

After the A-device signals a connect, it shall continue to signal a connect for at least **TA_BIDL_ADIS** min, while waiting for the B-device to issue a bus reset. This ensures that the B-device has at least **TB_ACON_BSE0** max to detect and respond to the A-device connect.

This is the most critical timing requirement by the OTG specification (worst case of 1ms Max for OTG 1.3 compliant)! At a first look at this, for the B-device side, it is desirable that we can switch the Target Stack into Host Stack as soon as possible, just at the point B-device disconnects from A-device (when B-device detects A-device suspending the bus), so that when the A-device connects to the B-device, the Host Stack is working and we can proceed as a normal host detecting a normal device connection and continue the enumeration as normal. However, there are other facts that may make this choice hard to be feasible enough.

The design decision is that we only switch from Target to Host Stack when we actually gets a connection from A-device.

See **USB OTG HNP IMPLEMENTATION NOTES** and **USB OTG PROTOCOL COMPLIANCE IMPLEMENTATION NOTES** for more details.

8.7 **ta_wait_vrise** (otg 1.3)/**ta_vbus_rise** (otg 2.0) (max 100ms) - wait for vbus rise time

The a_wait_vrise_tmr timer is used by an A-device in the a_wait_vrise state to wait for the voltage on VBUS to rise above the A-device VBUS Valid threshold (a_vbus_vld = **TRUE**). If VBUS is not above this threshold before and after **TA_WAIT_VRISE** (a_wait_vrise_tmout = **TRUE**), then this is an indication that the B-device is drawing too much current.

If VBUS does not reach **VA_VBUS_AVG_LO** min (4.4V) within **TA_VBUS_RISE**, then the A-device shall turn off VBUS.

Note

If we fail to raise the VBUS in time, we will notify the user application with a message ID of **USBOTG_USR_MSG_ID_VRISE_FAIL**.

8.8 **ta_bdis_acon** (max 3ms in otg 1.3, max 150ms in otg 2.0) - b-disconnect to a-connect

During HNP process, when the A-device suspends the bus and when A-device detects the SE0 on the bus and recognizes this as a request from the B-device to become host. The A-device responds by turning on its D+ pull-up within **TA_BDIS_ACON** max of first detecting the SE0 on the bus. If the A-device does not detect a disconnect before **TA_AIDL_BDIS** min after suspending the bus, then the A-device is allowed to stop waiting for a disconnect and end the session.

It might be desirable to process the B-device disconnection while the A-device is in suspend as a normal device disconnection, and then reports to the OTG Manager of this disconnection event, which will trigger a stack switch from Host Stack to Target Stack. However, some other facts make this not feasible enough.

The design decision is that we only switch from Host to Target Stack when we actually get a Bus Reset from B-device.

See **USB OTG HNP IMPLEMENTATION NOTES** and **USB OTG PROTOCOL COMPLIANCE IMPLEMENTATION NOTES** for more details.

8.9 **ta_bidl_adis** (min 3ms in otg 1.3, min 155ms in otg 2.0, max 200ms) - b-idle to a-disconnect time

In the a_peripheral state, the A-device transitions from the a_peripheral state to the a_wait_bcon state if the a_bidl_adis_tmr times out (a_bidl_adis_tmout = **TRUE**). If the A-device detects more than **TA_BIDL_ADIS** min of continuous idle (i.e. J_state), on the bus, then the A-device may transition to the a_wait_bcon state. If no activity is detected after **TA_BIDL_ADIS** max the A-device shall transition back to the a_wait_bcon state.

Note

The OTG Manager will switch stack when the B-device bus suspend happens after 3ms. The stack switch from Target Stack to Host Stack should be done in Max 200ms.

9 usb otg usage scenarios

This section describes the OTG usage scenarios.

Note that we distinguish the "attached" and "connected" concepts; being attached is the situation in which the cable is plugged in the USB port; while being "connected" is the situation in which the cable is not only plugged in the USB port, but also the peripheral side applied its D+ pull-up.

9.1 no usb plug attached

In this scenario, the ID pin is floating. The stack should be started as Target Stack and in b_idle state.

9.2 mini-a plug attached, but no device at other end of the cable

In this scenario, the ID pin is grounded. The stack should be started as Host Stack and wait for the device at other end to be attached. The ports VBUS may be powered off to save power (can also be powered on as normal Hosts do!)

Note

Once detecting a ID pin change from HIGH to LOW, we must power on VBUS and see if there is device connection happen (use a_wait_bcon_tmr). This is to watch out for the case A-device is inserted with a normal Non-OTG capable B-device.

9.3 mini-a plug connected with a non-otg device in the other end

In this scenario, the ID pin is grounded. The stack should be started as Host Stack and does normal Host Stack operation with the device.

9.4 mini-a plug connected with a otg dual-role device in the other end

In this scenario, the ID pin is grounded. The stack should be started as Host Stack and does normal Host Stack operation with the device. This device is the A-device and the other end of the cable is the B-device. The Host Stack will check if the device at the other end can be supported (in its TPL list); If the device at the other end can not be supported, then it enables the HNP feature of the OTG B-device, and suspend the bus to start the HNP process (try to allow the B-device to control the bus and see if the B-device can support this A-device in a_peripheral mode).

9.5 mini-b plug attached, but no device at other end of the cable

In this scenario, the ID pin is floating. The stack should be started as Target Stack and in b_idle state.

9.6 mini-b plug attached with a non-otg device (peripheral only) in the other end

In this scenario, the ID pin is floating. The stack starts in Target Stack, and tries to initiate SRP to the device at the other end (since the device in the other end of the cable is non-OTG capable, it doesn't respond to SRP and doesn't raise VBUS). So this device will switch from peripheral to host mode if it can turn on VBUS (in b_host state).

Note

This seems kind of weird, the USB cable may not allow this kind of connection.

9.7 mini-b plug attached with a otg dual-role device in the other end

In this scenario, the ID pin is floating. The stack starts in Target Stack, and tries to initiate SRP to the device at the other end (since the device in the other end of the cable is OTG capable, it can respond to the SRP and raise VBUS to the valid range). So this device will work in peripheral mode if the other end turns on VBUS. If the other end device finds out it can not support this device, then the other end will suspend the bus and give this device a chance to switch into b_host mode to control the bus. If this device also finds out it can not support the other end of the device, then it will suspend the bus again and the other end will find out both device will not communicate, and thus will turn off the VBUS.

9.8 working as a-device but vbus is turned off, gets srp from the other end

In this scenario, the ID pin is grounded. The stack is in Host Stack and will turn on the VBUS in response to the SRP.

9.9 working as a-device host, gets hnp (host request flag) from the other end

In this scenario, the ID pin is grounded. The stack is in Host Stack and will suspend the bus when A-device application finished using the bus, accepting the HNP. When the other end disconnects this device connects to the other end and switch to Target Stack.

9.10 working as a-device host, the other end disconnects or mini-b plug detached

In this scenario, the ID pin is grounded. The stack is in Host Stack and when the other end disconnects, it will handle device disconnection as normal Host Stack, but will still keep in Host Stack (since Mini-A plug is still attached) waiting for B-device attachment and connection.

9.11 working as a-device host, mini-a plug detached

In this scenario, the ID pin is grounded (will become floating soon). The stack is in Host Stack and when the Mini-A plug detached, it will handle device disconnection as normal Host Stack and then will find the ID pin get floating, so the Stack will switch to Target Stack and in b_idle.

9.12 working as b-device peripheral, hnp feature enabled from the other end

In this scenario, the ID pin is floating. The stack is in Target Stack. When the other end suspend the bus this device disconnects from the other end and switch to Host Stack if a connection is detected from A-device.

9.13 working as b-device peripheral, mini-a plug or mini-b plug detached

In this scenario, the ID pin is floating. The stack is in Target Stack. When detects either Mini-A plug or Mini-B plug detached, it will handle peripheral side disconnection but will still keep in Target Stack (if the device is self powered); If the device is bus powered, then the B-device is dead of course.

10. supported ioctl commands

The VxWorks USB OTG Stack **iosLib** interface supports the following ioctls. These ioctls are furhter encapsulated to form the user level API routines.

10.1 **usbotg_ioctl_otg_state_get** :

Get current USB OTG state; See comments for **usrUsbOtgStateGet( )**.

10.2 **usbotg_ioctl_vbus_state_get** :

Get the VBUS state; See comments for **usrUsbOtgVbusStateGet( )**.

10.3 **usbotg_ioctl_dev_type_get** :

Get the local OTG device type (A-device or B-device); See comments for **usrUsbOtgDeviceTypeGet( )**.

10.4 **usbotg_ioctl_host_request** :

Request for host role; See comments for **usrUsbOtgHostRequest( )**.

10.5 **usbotg_ioctl_host_giveup** :

Give up host role; See comments for **usrUsbOtgHostGiveup( )**.

10.6 **usbotg_ioctl_vbus_power_on** :

Request to power on VBUS; See comments for **usrUsbOtgVbusPowerOn( )**.

10.7 **usbotg_ioctl_vbus_power_off** :

Request to power off VBUS; See comments for **usrUsbOtgVbusPowerOff( )**.

10.8 **usbotg_ioctl_callback_install** :

Request to install user message callback. See comments for **usrUsbOtgCallbackInstall( )**.

10.9 **usbotg_ioctl_stack_mode_get** :

Get current USB OTG stack mode; See comments for **usrUsbOtgStackModeGet( )**.

Include files

iosLib.h drv/erf/erfLib.h errnoLib.h poolLib.h , **usb/usbOsalDebug.h usb/usbOtg.h**

Name

usrUsbOtgVbusStateNameGet( ) - convert OTG VBUS state into printable name string

Layer

USB_OTG

Synopsis

```
char * usrUsbOtgVbusStateNameGet
    (
    USBOTG_VBUS_STATE state
    )
```

Description

This routine is to convert the OTG VBUS state into printable name string.

Returns

Pointer to the printable OTG VBUS state name string

Errno

N/A

See also

usbOtg

Name

usrUsbOtgStateNameGet( ) - convert OTG state into printable name string

Layer

USB_OTG

Synopsis

```
char * usrUsbOtgStateNameGet
    (
    USBOTG_STATE state
    )
```

Description

This routine is to convert the OTG state into printable name string.

Returns

Pointer to the printable OTG state name string

Errno

N/A

See also

usbOtg

Name

usrUsbOtgTimerNameGet( ) - convert the OTG state into printable name string

Layer

USB_OTG

Synopsis

```
char * usrUsbOtgTimerNameGet
    (
    USBOTG_TIMER timerId
    )
```

Description

This routine is to convert the OTG state into printable name string.

Returns

Pointer to the printable OTG state name string

Errno

N/A

See also

usbOtg

Name

usrUsbOtgOpen( ) - open an OTG controller

Layer

USB_OTG

Synopsis

```
int usrUsbOtgOpen
    (
    char * otgName
    )
```

Description

This routine is to open an USB OTG controller. If the otgName is **NULL** or empty string, then it will use the first available OTG controller name as the default name to open.

Call it like below from the shell or user application.

otgFd = usrUsbOtgOpen("/usbOtg0");

Returns

Non-negative file descriptor if opened, or negative value if failed.

Errno

N/A

See also

usbOtg

Name

usrUsbOtgClose( ) - close an OTG controller

Layer

USB_OTG

Synopsis

```
STATUS usrUsbOtgClose
    (
    int otgFd
    )
```

Description

This routine is to close an USB OTG controller which has been opened before.

Call it like below from the shell or user application.

sts = usrUsbOtgClose(otgFd);

Returns

OK file descriptor if closed, or **ERROR** value if failed.

Errno

N/A

See also

usbOtg

Name

usrUsbOtgStateGet( ) - get OTG state of an OTG controller

Layer

USB_OTG

Synopsis

```
STATUS usrUsbOtgStateGet
    (
    int            otgFd,
    USBOTG_STATE * pOtgState
    )
```

Description

This routine is to get OTG state of an USB OTG controller which has been opened before.

Call it as below from the shell or user application.

sts = usrUsbOtgStateGet(otgFd, &otgState);

Returns

OK if the status is get successfully, **ERROR** otherwise; if it returns **OK**, then **USBOTG_STATE** is saved in the **pOtgState**.

Errno

OK if the status is get successfully **ERROR** otherwise

See also

usbOtg

Name

usrUsbOtgVbusStateGet( ) - get VBUS state of an OTG controller

Layer

USB_OTG

Synopsis

```
STATUS usrUsbOtgVbusStateGet
    (
    int                 otgFd,
    USBOTG_VBUS_STATE * pVbusState
    )
```

Description

This routine is to get OTG state of an USB OTG controller which has been opened before.

Call it as below from the shell or user application.

sts = usrUsbOtgVbusStateGet(otgFd, &vbusState);

Returns

OK if the VBUS state is got successfully, **ERROR** otherwise; if it returns **OK**, then the **USBOTG_VBUS_STATE** is saved in **pVbusState**.

Errno

N/A

See also

usbOtg

Name

usrUsbOtgDeviceTypeGet( ) - get device type of an OTG controller

Layer

USB_OTG

Synopsis

```
STATUS usrUsbOtgDeviceTypeGet
    (
    int                   otgFd,
    USBOTG_DEVICE_TYPE * pDeviceType
    )
```

Description

This routine is to get device type of an USB OTG controller which has been opened before.

Call it like below from the shell or user application.

sts = usrUsbOtgDeviceTypeGet(otgFd, &devType);

Returns

OK if the device type is got successfully, **ERROR** otherwise; if it returns **OK**, then the **USBOTG_DEVICE_TYPE** is saved in **pDeviceType**.

Errno

N/A

See also

usbOtg

Name

usrUsbOtgStackModeGet( ) - get stack mode of an OTG controller

Layer

USB_OTG

Synopsis

```
STATUS usrUsbOtgStackModeGet
    (
    int           otgFd,
    USBOTG_MODE * pStackMode
    )
```

Description

This routine is to get stack mode of an USB OTG controller which has been opened before.

Call it like below from the shell or user application.

sts = usrUsbOtgStackModeGet(otgFd, &stackMode);

Returns

OK if the stack mode is got successfully, **ERROR** otherwise; if it returns **OK**, then the **USBOTG_MODE** is saved in **pStackMode**.

Errno

N/A

See also

usbOtg

Name

usrUsbOtgHostRequest( ) - request to work in host role for an USB OTG controller

Layer

USB_OTG

Synopsis

```
STATUS usrUsbOtgHostRequest
    (
    int                       otgFd,
    USBOTG_HOST_REQUEST_TYPE reqType,
    _Vx_ticks_t               waitTime
    )
```

Description

This routine is to request to work in host role for an USB OTG controller.

Call it like below from the shell or user application:

sts = usrUsbOtgHostRequest(otgFd, reqType, waitTime);

The **reqType** specifies the protocol to be used for the host request. It has two possible values according to the latest USB OTG specification, **USBOTG_HOST_REQUEST_HNP_BASIC** and **USBOTG_HOST_REQUEST_HNP_POLLING**.

1) **USBOTG_HOST_REQUEST_HNP_BASIC** - basic HNP method

This type of HNP has been introduced since the initial version of USB OTG specification and is the core protocol used by USB OTG for role switching between two Dual-Role capable USB OTG devices.

This protocol is initiated by the device in peripheral role (e.g, a B-device in b_periperial state) by specifying that the applicaiton on it needs to use the USB bus; With that requirement specified, the device in peripheral role monitors the USB bus to be suspended by the host; Once such a suspend event is detected after the host request requirement is specified, it will disconnect from the peer host (e.g, a A-device in a_host state), and the peer host will switch its role into peripheral role and then connects to the current peripheral role device, if that connection event is detected, then it will switch its role from the peripheral role into host role, which completes the role switching as defined by the basic HNP protocol.

Note, however, there is no OTG defined contract between two connected OTG devices on when the current peripheral role device has actually "requested" the host role. That is to say, the current host role device has no knowledge when it needs to "giveup" the host role by suspending the USB bus except that the user application explicitly directs it to do so (for example, by using **usrUsbOtgHostGiveup( )**). If the peripheral role device does not request host role, a suspend from the host role device is not considered as a chance to switch roles between them.

This may present usability complication for the user. For example, with two OTG Dual-Role capable cell phones connected and you want to switch their roles, you may need to press one button on one phone (in peripheral role) which initiates the host role request; Then you may need to press another button on another phone (in host role) to actually starts the role switching (which then suspends the USB bus).

The above example might be simplified by application on the peripheral role device to initiate the host role request once the HNP feature is enabled by the host role device during the enumeration process. This way, the peripheral role device could continue to work in peripheral role until the host role device gives up its host role, starts the HNP procedulre (suspending USB bus). This makes it possible to just press the button on the phone which is in host role to switch the roles (as the host role request has already been made by the application when the HNP feature is enabled). To do so, the application could install an user message callback by **usrUsbOtgCallbackInstall( )**, and in **USBOTG_USR_MSG_ID_HNP_ENABLED** handling code, call **usrUsbOtgHostRequest( )** with **USBOTG_HOST_REQUEST_HNP_BASIC** to initiate the host role request.

Note that the aforementioned way to simplify the usability may not suit all cases. For example, if you do not want to take any "bus suspend" as a trigger for role switching, then this method should not be used. A noted case when using this method is that it may fail the USB OTG CV Test. In the USB OTG CV test, the test tool is installed on a standard PC system in which the host controller is a standard, non-OTG capable host controller. The test would exercise the various OTG specific requests and commands, including the OTG specific SetFeature(b_hnp_enable) command even if the host does not actually support OTG/HNP. During the test, the test tool may also suspend the bus, and this creates a problem if the aforementioned way is used, which would cause the Device Under Test (DUT) to disconnect from the test tool host controller, because, the DUT "thought" it's a real HNP capable host giving up host role. However, due to the fact that test tool host is not OTG capable, it would not connect to the DUT according to the HNP protocol. The effect is that the DUT is considered as being "phsically disconnected" from the test tool host, thus the USB OTG CV Test would fail in this case. Therefore, this method may not suit for production usage, but it may be used to facilitate autoamtic HNP testing during development.

The complication is due to the fact that there is no OTG defined way (before the OTG 2.0 specification is released) for the peripheral role device to inform the host role device that host role is requested by the peripheral role device.

The OTG 2.0 specification adds, among other capabilities, a way for the peripheral role device to inform the host role device that host role is being requested by the peripheral role device. This is called "HNP polling" and is supported using the **reqType** of **USBOTG_HOST_REQUEST_HNP_POLLING** as described below.

2) **USBOTG_HOST_REQUEST_HNP_POLLING** - HNP polling method

As described above, for basic HNP protocol, the lack of a way to inform host role device about when a peripheral device has "requested" host role, created some usbility complication. The OTG 2.0 specification has added a protocol called "HNP polling" which effectively addressed this problem. Basically, the "HNP polling" is done through a control transfer, namely GetStatus(OTG status). When a OTG device is enumerated, the host side OTG stack would issue GetStatus(OTG status) regularly, to check if the peripheral device has set the "host request flag" in the returned status. If the request succeeded, and the "host request flag" is set, then it knows the peripheral role device is requesting host role and it can inform the user application on the host role device. The user applcation could then respond to the request by doing some cleanup work then giveup its host role by **usrUsbOtgHostGiveup( )**. If the request for GetStatus(OTG status) failed, it indicates the peripheral role device may not support "HNP polling" thus it has to resort to basic HNP protocol. For the peripheral role device, if "HNP polling" is supported and a GetStatus(OTG status) is received, it returns "host request flag" telling the host role device about the current host request status (cleared as not requested, set as requested).

By using "HNP polling", the host side could automatically detect that the host role is being requested by the peripheral role device. Thus it is possible to trigger automatic response, eliminating the need for user to "press a button to start". If such automatic response is desired by the user, the application could install a user message callback, and in the message handling code for **USBOTG_USR_MSG_ID_DEV_REQ_HOST**, call **usrUsbOtgHostGiveup( )**. Note that calling **usrUsbOtgHostGiveup( )** may trigger role switch, during the time the communication between the two device will be shut down by force. This might cause some risk for potential data loss. So it might be more reasonable to perform some cleanup before calling **usrUsbOtgHostGiveup( )**.

The OTG 2.0 specification also made clear that it might be risky to request host role using "HNP polling", since the host side may immdiately shut down communicaton for role switching in response to "host request flag" being set. As a result, you will also get a **USBOTG_USR_MSG_ID_REQ_HOST_RISKY** user message if you called **usrUsbOtgHostRequest( )** with **USBOTG_HOST_REQUEST_HNP_POLLING**. The message is to warn the user of this potential risk, the application may disaplay this warnning message to the end user. But once this request is issued, role switching may be on the way according to host side capability. Thus it is the applicaiton's responsibility to make sure any cleanup work has been done so that the risk has been eliminated.

Note that "HNP polling" still needs the basic HNP protocol to be in effect. That is, requesting using **USBOTG_HOST_REQUEST_HNP_POLLING** naturally enables the **USBOTG_HOST_REQUEST_HNP_BASIC** to be in effect. Without basic HNP protocol, "HNP polling" would not work to actually switch the roles. "HNP polling" only adds a way to let the host role device to decide when to start the basic HNP protocol (by suspending USB bus). Without "HNP polling", the decision has to be made by the user mannualy, for example, by "pressing a button". Also, the "HNP polling" will only work if both ends of the OTG connection supports it.

Note also that "HNP polling" is only introduced in OTG 2.0 specification. Although this OTG stack is designed with OTG 2.0 in mind, due to lack of real OTG 2.0 hardware for testing, the current OTG stack is configured to support OTG 1.3 (the latest OTG specification version as found before OTG 2.0). However, since the "HNP polling" feature is pure software based protocol, no special need for hardware assist, and it addresses the usbility complication problem for basic HNP protocol without any backword compatibility issues, it is believed that this feature could coexist within the current OTG stack. With this insight, the **USBOTG_HOST_REQUEST_HNP_POLLING** related code is still included in the stack, thus the described usages for this **reqType** still apply for the current OTG stack configuration.

The need for **waitTime** parameter is described below:

Before the host role request is actually made, some prerequisite condition must be met, for example, for a B-device to be able to request host role from the A-device, the A-device should have enabled the B-device for HNP feature. The **waitTime** parameter is used by the implementation to control how to wait such prerequisite condition to be met. If **waitTime** is set as **NO_WAIT** (0), then it tries to check if the prerequisite condition can be met, if not, it will return **ERROR**; if **waitTime** is set as **WAIT_FOREVER** (-1U), then the prerequisite condition check will last forever until the condition is met; if **waitTime** is set as other positive value, the prerequisite condition check will be done at least with the specified amount of time before it retuerns **ERROR** if the condition can not be met; If prerequisite condition check is passed, then the actual host role request will be made.

Note that the host role switching procedure may be performed asynchronously with the calling task for host role request. So even if **usrUsbOtgHostRequest( )** returns **OK**, it does not necessarily mean the calling device has become host role. There are several factors which may affect if the role switching will actually take place. For example, as noted in the descriptions above, the role has to be given by the peer host role device with its own decision whether to giveup its host role or not. In any cases,

the calling task could monitor OTG state to become USBOTG_STATE_a_host or USBOTG_STATE_b_host by **usrUsbOtgStateGet( )**.

Along with role switching, there may be stack mode switching to happen. The user application may install a user message callback to receive the related user messages **USBOTG_USR_MSG_ID_HOST_ACTIVE** and **USBOTG_USR_MSG_ID_TARGET_ACTIVE**.

Returns

OK if the host role request has been made, **ERROR** otherwise.

Errno

N/A

See also

usbOtg, **usrUsbOtgHostGiveup( )**, **usrUsbOtgStackModeGet( )**, **usrUsbOtgStateGet( )**

Name

usrUsbOtgHostGiveup( ) - request to giveup host role for an USB OTG controller

Layer

USB_OTG

Synopsis

```
STATUS usrUsbOtgHostGiveup
    (
    int         otgFd,
    _Vx_ticks_t waitTime
    )
```

Description

This routine is to request to giveup host role for an USB OTG controller which has been opened before.

Call it as below from the shell or user application.

sts = usrUsbOtgHostGiveup(otgFd, waitTime);

The need for **waitTime** parameter is described below:

Before the host role give up request is actually made, some prerequisite conditions must be met, for example, for a A-device to be able to giveup host role, the A-device should have connected with an OTG/HNP capable B-device and has enabled the B-device for HNP feature. The **waitTime** parameter is used by the implementation to control how to wait such prerequisite conditions to be met. If **waitTime** is set as **NO_WAIT** (0), then it tries to check if the prerequisite condition can be met, if not, it will return **ERROR**; if **waitTime** is set as **WAIT_FOREVER** (-1U), then the prerequisite condition check will last forever until the condition is met; if **waitTime** is set as other positive value, the prerequisite condition check will be done at least with the specified amount of time before it returns **ERROR** if the condition can not be met; If prerequisite condition check is passed, then the actual host role giveup request will be made.

Note that the host role switching procedure may be performed asynchronously with the calling task for host role giveup. So even if **usrUsbOtgHostGiveup( )** returns **OK**, it does not necessarily mean the calling device has become peripheral role. The calling task could monitor the OTG state to become USBOTG_STATE_a_peripheral or USBOTG_STATE_b_peripheral by **usrUsbOtgStateGet( )**.

Along with role switching, there may be stack mode switching to happen. The user application may install a user message callback to receive the related user messages **USBOTG_USR_MSG_ID_HOST_ACTIVE** and **USBOTG_USR_MSG_ID_TARGET_ACTIVE**.

Returns

OK if the host role giveup request has been made, **ERROR** otherwise.

Errno

N/A

See also

usbOtg, **usrUsbOtgHostRequest( )**, **usrUsbOtgStackModeGet( )**, **usrUsbOtgStateGet( )**

Name

usrUsbOtgVbusPowerOn( ) - request to power on VBUS for an OTG controller

Layer

USB_OTG

Synopsis

```
STATUS usrUsbOtgVbusPowerOn
    (
    int         otgFd,
    _Vx_ticks_t onTime
    )
```

Description

This routine is to request to power on VBUS for an USB OTG controller.

For A-device, it requests the VBUS to be powered up (if not already) by the OTG Controller Driver.

For B-device, since VBUS is provided and controlled by A-device, if VBUS is not currently powered up, then it requests the A-device to provide VBUS through the OTG SRP procedure. Depending on the A-device capability (whether it could detect SRP signal), the A-device may, or may not respond to SRP.

The actual VBUS control may be done asynchronously to the calling task and it may take some time for VBUS to transition to USBOTG_VBUS_STATE_VBusValid due to hardware effect. So after calling this routine, the application could monitor VBUS to become USBOTG_VBUS_STATE_VBusValid by querying the VBUS state using **usrUsbOtgVbusStateGet( )**.

A **onTime** parameter is required for this routine, which is used when the OTG Controller is working as A-device in "a_wait_bcon" state to wait for B-device connection. It is used to setup a timer once "a_wait_bcon" state is entered. If the timer expires before any device connection is detected, then the VBUS will be powered off automatically. If there is any device connection detected before the timer expires, then the timer is canceled and the OTG state transtion to "a_host" state in which

VBUS will not be powered off automatically (but application could power off VBUS by force). The value passed in could be **WAIT_FOREVER** (-1U), which means once it enters "a_wait_bcon" state, it does not power off VBUS even if there is no B-device connection is detected (as the case for normal USB host). Although the value could be any other positive time in ticks, or even **NO_WAIT** (0), if the time is too short, the B-device connection might be lost because it poweres off VBUS too soon to make the connection event being recognized and processed. For this reason, there is a minimum default time (**TA_WAIT_BCON** min) used to prevent the aforementioned situation. If the value passed in is smaller than the minimum default time, then the minimum default time will take effect.

Returns

OK when the VBUS power on request is signaled, **ERROR** otherwise.

Errno

N/A

See also

usbOtg, **usrUsbOtgVbusPowerOff( )**, **usrUsbOtgVbusStateGet( )**

Name

usrUsbOtgVbusPowerOff( ) - request to power off VBUS for an OTG controller

Layer

USB_OTG

Synopsis

```
STATUS usrUsbOtgVbusPowerOff
    (
    int         otgFd,
    _Vx_ticks_t offTime
    )
```

Description

This routine is to request to power off VBUS by the USB OTG Controller Driver.

For A-device, this will request the OTG Controller Driver to power off the VBUS. For B-device, since VBUS is provided and controlled by A-device, there is no OTG defined way to request VBUS to be powered off. So this routine is only supported when the OTG Controller is working as A-device. When the OTG Controller is working as B-device, calling this routine will have no effect and will return **ERROR**.

The actual VBUS control may be done asynchronously to the calling task and it may take some time for VBUS to transition to USBOTG_VBUS_STATE_SessEnd due to hardware effect. So after calling this routine, the application could monitor VBUS to become USBOTG_VBUS_STATE_SessEnd by querying the VBUS state using **usrUsbOtgVbusStateGet( )**.

A **offTime** parameter is required for this routine and the reason for it is given below:

USB host device connection detection and operation needs VBUS to be provided. In OTG environment, if the A-device VBUS is powered off, the B-device could request A-device to power up the VBUS through the SRP procedure. However, some OTG controllers may have trouble in detecting SRP signal from the B-device; Or the B-device might be not able to signal the SRP signal (for example, a non-OTG capable B-device). In these cases, if the A-device VBUS is powered off, then it can not detect

any device connection until the user on the A-device has explictly powered up the VBUS. While it may help to save some power when VBUS is powered off, this behaviour may decrease user experiences since B-device connection may not be automatically detected when A-device VBUS is powered off as compared to basic USB host/device model.

To provide better user experiences, the OTG stack provides an automatic VBUS power up control option. In this mode, after A-device VBUS is powered off, it will wait in the "a_idle" state for an amount of time as specified by **offTime**; then it tries to power up the VBUS and see if there is new device connection in "a_wait_bcon" state, if after another amount of time as specified by **onTime** for the **usrUsbOtgVbusPowerOn( )** routine in the "a_wait_bcon" state there is no device connection, then it poweres off the VBUS automatically. This effectively makes a loop in which VBUS transtions between on and off until a new device connection is detected, or an SRP signal is detected.

The **offTime** parameter can be **NO_WAIT** (0), **WAIT_FOREVER** (-1U), or any other positive time in ticks. The value can be used to setup a timer, once the timer expires, it signals the OTG Manager to power on the VBUS. If set to **NO_WAIT**, then the effect is that once the OTG state enters "a_idle" state after VBUS goes off, it will immediately signal for VBUS to power up; if set to **WAIT_FOREVER**, then the timer will not expire, the effect is that the automatic VBUS power up is disabled, and the VBUS will only be powered up explicitly by the A-device application, or in response to a B-device SRP.

Returns

OK when the request is signaled for A-device, **ERROR** otherwise.

Errno

N/A

See also

usbOtg, **usrUsbOtgVbusPowerOn( )**, **usrUsbOtgVbusStateGet( )**

Name

usrUsbOtgCallbackInstall( ) - install user message callback for an OTG controller

Layer

USB_OTG

Synopsis

```
STATUS usrUsbOtgCallbackInstall
    (
    int                    otgFd,
    USBOTG_USR_CALLBACK * pUsrCallback,
    void *                 pUsrContext
    )
```

Description

This routine is to request to install usr message callback for an USB OTG controller which has been opened before.

Call it like below from the shell or user application:

sts = usrUsbOtgCallbackInstall(otgFd, pUsrCallback, pUsrContext);

The **pUsrCallback** is a pointer to a user application implemented user message callback which takes the prototype as below:

typedef VOID **USBOTG_USR_CALLBACK** (void * pUsrMsgBuff, /* User message buffer */ void * pUsrContext / * User context */);

The **pUsrMsg** points to a message buffer area which could be casted to be of pointer to **USBOTG_USR_MSG_HDR** structure, which is of type as below:

typedef struct usbotg_usr_msg_hdr { UINT32 uMsgId; /* Message ID */
}**USBOTG_USR_MSG_HDR**, *pUSBOTG_USR_MSG_HDR;

Based on the **uMsgId** in the **USBOTG_USR_MSG_HDR** structure, the **pUsrMsg** could be further casted to specific message structure for the message ID. The message IDs are defined by the following enum type:

/* Message ID to send to user application */

typedef enum usbotg_usr_msg_id { /* * B-device SRP procedure is in progress, waiting * for A-device to raise VBUS. */

USBOTG_USR_MSG_ID_SRP_ACTIVE = 1,

/* * B-device SRP procdure failed, A-device does not * provide valid VBUS in time. */

USBOTG_USR_MSG_ID_SRP_FAIL = 2,

/* * A-device tried to raise the VBUS but failed to * make it into a valid VBUS in time. */

USBOTG_USR_MSG_ID_VRISE_FAIL = 3,

/* The peer host does not support HNP */

USBOTG_USR_MSG_ID_HOST_NO_HNP = 4,

/* An device on the root hub is detected but no * matching class driver is found. The messeage
* has a paramter specifying the device handle * for which the device is bound to. */

USBOTG_USR_MSG_ID_DEV_UNSUPPORTED = 5,

/* * HNP pooling detected that the peer device is * requesting host role. */

USBOTG_USR_MSG_ID_DEV_REQ_HOST = 6,

/* * The application is requesting host role by specifying request
* type to be **USBOTG_HOST_REQUEST_HNP_POLLING**, which might cause
* the HNP pooling mechanism to be effective when appropriate,
* but that is considered to be risky since it may immediatly * cease the host-device communication. */

USBOTG_USR_MSG_ID_REQ_HOST_RISKY = 7,

/* * The active stack mode has switched from target stack * to host stack. */
USBOTG_USR_MSG_ID_HOST_ACTIVE = 8,

/* * The active stack mode has switched from host stack * to target stack. */

USBOTG_USR_MSG_ID_TARGET_ACTIVE = 9,

/* The device does not respond to requests made by the host */

USBOTG_USR_MSG_ID_DEV_UNRESPONDING = 10,

/* * A-device detected that B-device is asking for VBUS * to be provided (SRP signaling is detected) */

USBOTG_USR_MSG_ID_SRP_DETECTED = 11,

/* * The peripheral role device received a SetFeature(b_hnp_enable) * command from the host role device. */

USBOTG_USR_MSG_ID_HNP_ENABLED = 12 }**USBOTG_USR_MSG_ID**;

The following data structures have been defined for each of the message IDs.

typedef struct usbotg_usr_msg_srp_active { **USBOTG_USR_MSG_HDR** header;
}**USBOTG_USR_MSG_SRP_ACTIVE**, *pUSBOTG_USR_MSG_SRP_ACTIVE;

typedef struct usbotg_usr_msg_srp_fail { **USBOTG_USR_MSG_HDR** header;
}**USBOTG_USR_MSG_SRP_FAIL**, *pUSBOTG_USR_MSG_SRP_FAIL;

typedef struct usbotg_usr_msg_vrise_fail { **USBOTG_USR_MSG_HDR** header;
}**USBOTG_USR_MSG_VRISE_FAIL**, *pUSBOTG_USR_MSG_VRISE_FAIL;

typedef struct usbotg_usr_msg_host_no_hnp { **USBOTG_USR_MSG_HDR** header;
}**USBOTG_USR_MSG_HOST_NO_HNP**, *pUSBOTG_USR_MSG_HOST_NO_HNP;

typedef struct usbotg_usr_msg_dev_unsupported { **USBOTG_USR_MSG_HDR** header; UINT32 hDevice;
}**USBOTG_USR_MSG_DEV_UNSUPPORTED**, *pUSBOTG_USR_MSG_DEV_UNSUPPORTED;

typedef struct usbotg_usr_msg_dev_req_host { **USBOTG_USR_MSG_HDR** header; UINT32 hDevice;
}**USBOTG_USR_MSG_DEV_REQ_HOST**, *pUSBOTG_USR_MSG_DEV_REQ_HOST;

typedef struct usbotg_usr_msg_req_host_risky { **USBOTG_USR_MSG_HDR** header;
}**USBOTG_USR_MSG_REQ_HOST_RISKY**, *pUSBOTG_USR_MSG_REQ_HOST_RISKY;

typedef struct usbotg_usr_msg_host_active { **USBOTG_USR_MSG_HDR** header;
}**USBOTG_USR_MSG_HOST_ACTIVE**, *pUSBOTG_USR_MSG_HOST_ACTIVE;

typedef struct usbotg_usr_msg_target_active { **USBOTG_USR_MSG_HDR** header;
}**USBOTG_USR_MSG_TARGET_ACTIVE**, *pUSBOTG_USR_MSG_TARGET_ACTIVE;

typedef struct usbotg_usr_msg_dev_unresponding { **USBOTG_USR_MSG_HDR** header; UINT32 hDevice;
}**USBOTG_USR_MSG_DEV_UNRESPONDING**, *pUSBOTG_USR_MSG_DEV_UNRESPONDING;

typedef struct usbotg_usr_msg_srp_detected { **USBOTG_USR_MSG_HDR** header;
}**USBOTG_USR_MSG_SRP_DETECTED**, *pUSBOTG_USR_MSG_SRP_DETECTED;

typedef struct usbotg_usr_msg_hnp_enabled { **USBOTG_USR_MSG_HDR** header;
}**USBOTG_USR_MSG_HNP_ENABLED**, *pUSBOTG_USR_MSG_HNP_ENABLED;

Some of these message structures are only with the a single header element, while some others may have companion parameters further describing them. Especially, the **hDevice** element specifies the associated device handle when that message occurs. The application may use that device handle to get more information about the device and then take appropriate action. For an example, if a message of ID **USBOTG_USR_MSG_ID_DEV_REQ_HOST** is received, the application may have a chance to decide whether to give up its host role to the requesting device.

Note

The installed user message callback may be called in the context of some system management task, such as the tErfTask if the implementation uses **erfLib** to report such messages. With this insight, it is vital for the user application to not take too long to process just in the callback. If a long time processing is required, you may consider to record the message and then put the actual processing into an application specific task.

Returns

OK if the user message callback is installed, **ERROR** otherwise.

Errno

N/A

See also

usbOtg

Name

usbOtgShow( ) - show the internal information of the OTG Manager

Layer

USB_OTG

Synopsis

```
void usbOtgShow (void)
```

Description

This routine is to show the internal information of the OTG Manager.

Returns

N/A

Errno

N/A

See also

usbOtg

11. USBTGT

Name

usbTgt - USB Target Transfer and Management Module

Layer

USB_TARGET

Routines

usbTgtTargChannelToFuncDriver( ) - find the pFuncDriver according to targChannel **usbTgtTcdVxDevGet( )** - get the Tcd's vxbDev pointer **usbTgtCreatePipe( )** - create a pipe for communication on an endpoint **usbTgtDeletePipe( )** - delete an pipe created for an endpoint **usbTgtUnConfigPipe( )** - un-configure a pipe for communication on an endpoint **usbTgtConfigPipe( )** - configure a pipe for communication on an endpoint **usbTgtSubmitErp( )** - submit one Erp to transfer data through the pipe **usbTgtCancelErp( )** - cancel a previously submitted **USB_ERP** **usbTgtSetPipeStatus( )** - set pipe stalled/unstalled status **usbTgtGetPipeStatus( )** - get the status of the pipe **usbTgtTcdSoftConnect( )** - do soft connect of the TCD **usbTgtFuncMngmtCallback( )** - do the function driver management callback **usbTgtErpCallback( )** - invoked when ERP completes **usbTgtFuncRemoteWakeUp( )** - issue a remote wakeup signal from the USB device **usbTgtFuncFrameNumGet( )** - get the current frame number from the TCD **usbTgtFuncControlPipeStall( )** - stall the function driver's control pipe **usbTgtFuncControlSetupErpSubmit( )** - submit the SETUP pecket on the control pipe **usbTgtFuncControlDataErpSubmit( )** - submit the DATA IN/OUT stage transaction **usbTgtFuncControlStatusErpSubmit( )** - submit the STATUS stage ERP on control pipe **usbTgtTcdDestroy( )** - destroy the **USBTGT_TCD** resource which unused anymore **usbTgtTcdCreate( )** - create one **USBTGT_TCD** resource for target stack **usbTgtFuncDestroy( )** - destroy the target function driver resource **usbTgtFuncCreate( )** - create one target function driver resource **usbTgtTcdEventNotify( )** - send one Tcd event notification **usbTgtFindFuncByName( )** - find the function driver pointer by name and unit number **usbTgtFindTargChannelByName( )** - find function targChannel by name and unit num **usbTgtFindTcdByName( )** - find the Tcd pointer by the name and unit number **usbTgtFindTcdByName( )** - find the Tcd pointer by the name and unit number **usbTgtFuncRemove( )** - remove the function driver from the target stack. **usbTgtFuncAdd( )** - add the function driver to the target stack **usbTgtTcdRemove( )** - remove one Target controller driver from the the target stack. **usbTgtTcdAdd( )** - add one target controller driver in the the target stack. **usbTgtFuncUnRegister( )** - un-register the function driver from the target stack **usbTgtFuncRegister( )** - register the function driver to the target stack **usbTgtTcdUnRegister( )** - un-register the TCD from the target stack **usbTgtTcdRegister( )** - register the tcd controller to the target stack **usbTgtExit( )** - uninitialize the global usb target resource **usbTgtInit( )** - initialize the usb target stack

Description

This module implements the USB target transfer and management module of the USB target stack for the VxWorks operating system. This is written using the Wind River USB 2.x API.

Vxworks usb target stack overview

The chart below is a simplified architecture of the whole USB target stack:

```
+-----------------------------------------+
|              User Application           |
|             (ioLib,Network,Fs...)        |
+--------------^----------@---------------+
               |          |
               |          |
+--------------@----------v---------------+
|                    TFD                  |
|        USB Target Function Driver(s)    |
|                                         |
```

```
| +---------------+     +---------------+ |
| |Mass Storage   |     |RNDIS          | |
| |Function Driver| ... |Function Driver| |
| +---------------+     +---------------+ |
+--------------^----------@---------------+
               |          |
               |          |
+--------------@----------v---------------+
|                   TML                   |
|       USB Target Management Layer       |
|                                         |
|+--------++----------++-----++----------+|
||Control ||Hardware  ||Data ||Registra- ||
||Requests||Events    ||Tx/Rx||tion      ||
||        ||(Reset...)||     ||Management||
|+--------++----------++-----++----------+|
+--------------^----------@---------------+
               |          |
               |          |
+--------------@----------v---------------+
|                  TCD                    |
|      USB Target Controller Driver(s)    |
+--------------^----------@---------------+
               |          |
               |          |
+--------------@----------v---------------+
|                Hardware                 |
|      USB Target Controller Hardware     |
+-----------------------------------------+
```

Hardware

Which is the USB target controller hardware

TCD

The USB target controller driver(TCD) who need be registered to the target management layer(TML).

TML

The USB target management layer(TML), which contains the process of all the control requests and the hardware events, and the transfer the data flow between the TCD and TFL.

TFL

The USB target function driver layer(TFL). This layer contains different independent function drivers. If you need use one kind of function on the target, you need to register the function driver to the TML, and make sure the correct TCD name which need be attached to has been configured.

User Application

The user application may needed by the TFL. Different function driver may use different applications. Such as the network stack application for the RNDIS virtual END driver or the FS framework for the mass storage function driver. This is totally depend on the function driver and the user's use case.

Initialization

To use this module, please make sure the **usbTgtInit( )** routine has been called in advance and successfully. This routine will initialize two lists (gUsbTgtTcdList and gUsbTgtFuncDriverList), which will be used when a new TCD or function driver be added to the target stack.

Data structures

1. **usbtgt_tcd**

Each USB Target Controller will have a **USBTGT_TCD** data structure, which plays the core role to connect the TCD with function drivers. The structure is defined as below:

```
/*
 * USB Target Management Abstraction
 */

typedef struct usbtgt_tcd
    {
    NODE                    usbTgtTcdNode; /* Tcd node in the global list */
    SEM_ID                  tcdMutex;      /* Used to protect the resource */
    LIST                    funcList;      /* Function drivers list */
    LIST                    configList;    /* Configuration list */
    LIST                    strDescList;   /* String descriptors list */

    atomic_t                controlErpUsed; /* The control ERP is used or not */
    USB_TARG_PIPE           controlPipe;    /* Handle to default control pipe */
    USB_ERP                 controlErp;     /* Erp used to transfer to/from */
    USB_SETUP               SetupPkt;       /* Setup pecket */
    UINT8                   dataBfr[USBTGT_MAX_CONTORL_BUF_SIZE]; /* Sending/receiving data */

    /* User configurable parameters */

    UINT32                  uTCDMagicID;  /* TCD magic ID */
    UINT16                  uVendorID;    /* VID */
    UINT16                  uProductID;   /* PID */
    char *                  pMfgString;   /* Manufacture string */
    char *                  pProdString;  /* Producture string */
    char *                  pSerialString;/* Serial num string */
    char *                  pTcdName;     /* TCD name */
    UINT8                   uTcdUnit;     /* Tcd unit number */

    /* Device specific parameters */

    struct vxbDev *         pDev;         /* Pointer to VxBus Device instance */
    USBTGT_TCD_FUNCS *      pTcdFuncs;    /* Functions implemented by the TCD */
    pVOID                   pTcdSpecific; /* Pointer to TCD specific structure */
    USBTGT_DEVICE_INFO      DeviceInfo;   /* Device information */

    USB_DEVICE_DESCR        DeviceDesc;   /* device Descriptor */
    USB_DEVICE_QUALIFIER_DESCR devQualifierDesc;/* Device qualifier descriptor */
    USB_LANGUAGE_DESCR *    pUsbTgtLangDescr; /* Language descriptor */

    UINT8                   uSpeed;
    UINT8                   uDeviceAddress; /* Device Address */
```

```
    UINT8                   uAddrToSet;     /* Address we want to set */
    UINT8                   uCurrentConfig; /* 0 means the target is unconfigured */
    BOOL                    bWakeupEnable;  /* Is the target enabled the wakeup feature */

    }USBTGT_TCD, *pUSBTGT_TCD;
```

The following items describe the major elements of the **USBTGT_TCD** data structure.

1.1 **usbtgt_tcd::usbtgttcdnode**

This element is used to link the resource to the global TCD list **gUsbTgtTcdList**. So the target mamagement can find the right **pUSBTGT_TCD** from the list to connect/disconnect with the right function driver **pUSBTGT_FUNC_DRIVER**. Specially when the TCD has initialized, and the user can dynamicly attach/detach the function driver. The target mamagement can find the right **pUSBTGT_TCD** for using.

1.2 **usbtgt_tcd::tcdmutex**

This is a mutex semaphore which is used to protect the **USBTGT_TCD** elements being concurrent updated.

1.3 **usbtgt_tcd::funclist**

This element is the function driver list which links all the function driver attached to the TCD.This is used for the support of multiple function drivers on one TCD.

1.4 **usbtgt_tcd::configlist**

This element is the configuration list which links all the configurations belong to the TCD. The user may configure more than one configuration on the TCD.

1.5 **usbtgt_tcd::strdesclist**

This element is the string descriptor list which links all the string for the TCD. Such as the string descriptor for different configuration, different interfaces. This string descriptor list also used to management the descriptor index used by each members.

1.6 **usbtgt_tcd::controlpipe**, **usbtgt_tcd::controlerp** and **usbtgt_tcd::controlerpused**

Those elements are used for the control endpoint to transfer the control requests and responce. **USBTGT_TCD::controlPipe** will be created/re-created when get one BUS RESET by the USB target controller, according to the working speed of the controller. All the transaction will use the **USBTGT_TCD::controlErp** which need be initialized before submitted by calling the routine **usbTgtSubmitControlErp( )**. Meanwhile, the **USBTGT_TCD::controlErpUsed** element is the flag which is used to indicate whether the control pipe is using or not.

There are there kinds of stage of control transfer, which are SETUP stage, DATA IN/OUT stage and STATUS IN/OUT stage. Following are the APIs to be used to initialize the transaction for the control pipe. Please go to **usbTgtControlRequest.c** for detail.

usbTgtInitSetupErp( )

Initialize the control ERP for the SETUP stage.

usbTgtInitDataErp( )

Initialize the control ERP for the DATA IN/OUT stage.

usbTgtInitStatusErp

Initialize the control ERP for the STATUS IN/OUT stage.

usbTgtControlResponseSend

Initialize the control ERP for the DATA IN stage, which is an interface for the function drivers.

usbTgtControlPayloadRcv

Initialize the control ERP for the DATA OUT stage, which is an interface for the function drivers.

usbTgtControlStatusSend

Initialize the control ERP for the DATA OUT stage, which is an interface for the function drivers.

1.7 **usbtgt_tcd::setuppkt** and **usbtgt_tcd::databfr**

USBTGT_TCD::SetupPkt is used to store the received SETUP packet from the USB host.**USBTGT_TCD::dataBfr** is used to store the data to be sent to or recieved from the USB host. **USBTGT_MAX_CONTORL_BUF_SIZE** is the buffer size of the **USBTGT_TCD::dataBfr**. The user can adjust it according to his needed.

1.8 **usbtgt_tcd::utcdmagicid**

This element is used to mark the valid TCD controller type. The user can see what is the hardware USB target controller is attached.

1.9 **usbtgt_tcd::uvendorid** and **usbtgt_tcd::uproductid**

Those two elements are get from the user's configuration or the default value if the user do not set them.They are indentified the USB target's VID/PID. They are very usful on the host driver (such as the vendor's class driver on Windows).

1.10 **usbtgt_tcd::pmfgstring**, **usbtgt_tcd::pprodstring** and **usbtgt_tcd::pserialstring**

Those elements are also get from the user's configuration or the default value if the user do not set them. They provide the device description information which can be read by the USB host. **USBTGT_TCD::pMfgString** is the manufacture string of the USB target, **USBTGT_TCD::pProdString** is product string of the USB target and **USBTGT_TCD::pSerialString** is the serial string of the USB target.

1.11 **usbtgt_tcd::ptcdname** and **usbtgt_tcd::utcdunit**

Those two elements indicate the TCD's name and unit number be managed by the USB target management.

1.12 **usbtgt_tcd::pdev**

USBTGT_TCD::pDev specifics the VxBus device instance, if this TCD compatible with the VxBus

1.13 **usbtgt_tcd::ptcdfuncs** and **usbtgt_tcd::ptcdspecific**

Those two elements are used for the TCD. The TCD need specific them befor register to the TML by calling the routine **usbTgtTcdAdd( )**.

USBTGT_TCD::pTcdFuncs is the USB target controller driver(TCD) function driver table, which contains the useful routines such as submitErp function pointer which can be called by the TML.

USBTGT_TCD::pTcdSpecific is the TCD spcific data structure pointer, it used to identify the validation of the real hardware which can be used or not.

1.14 **usbtgt_tcd::deviceinfo**

USBTGT_TCD::DeviceInfo specifics the TCD hardware feature. Such as the endpoints features and controller features.

1.15 **usbtgt_tcd::devicedesc**

This element is the device descriptor structure which is used to store the device descriptor of the target.

1.16 **usbtgt_tcd::devqualifierdesc**

This element is the device qualifier descriptor structure which is used to store the device qualifier descriptor of the target if it have one.

1.17 **usbtgt_tcd::pusbtgtlangdescr**

This element is the language descriptor structure pointer which is used to indicate the language supported of all the string descriptor of the target.

1.18 **usbtgt_tcd::uspeed**

This element indicates the current speed the TCD controller working at.

1.19 **usbtgt_tcd::udeviceaddress** and **usbtgt_tcd::uaddrtoset**

This two elements indicate the current devcie address and the address to be set by the host.

1.20 **usbtgt_tcd::udeviceaddress** and **usbtgt_tcd::uaddrtoset**

This two elements indicate the current devcie address and the address to be set by the host.

1.21 **usbtgt_tcd::ucurrentconfig**

USBTGT_TCD::uCurrentConfig specifics the current configuration set by the USB host. It must be one of the **USBTGT_TCD::configList**

2. **usbtgt_func_driver**

Each USB function driver will have a **USBTGT_FUNC_DRIVER** data structure, which plays the core role to connect the TCD with function drivers. The structure is defined as below:

```
/*
 * USB Target Function Driver Abstraction
 */

typedef struct usbtgt_funcion_driver
    {
    NODE                        funcDriverNode;       /* Function driver node in global list */
    NODE                        funcInTcdListNode;    /* Function driver node in TCD list */
    struct vxbDev *             pDev;                 /* Pointer to VxBus Device instance *
/
    SEM_ID                      funcMutex;            /* Used to protect the resource */
    UINT32                      uFuncMagicID;         /* Func magic Id */
    pVOID                       pFuncSpecific;        /* Specific data */
```

```
    USB_DESCR_HDR **           ppFsFuncDescTable;    /* Function descriptors table */
    USB_DESCR_HDR **           ppHsFuncDescTable;    /* Function descriptors table in High spe
ed mode */
    USB_DESCR_HDR **           ppSsFuncDescTable;    /* Function descriptors table in Super sp
eed mode */
    USB_TARG_CALLBACK_TABLE *  pFuncCallbackTable;   /* Function driver callback table */
    pVOID                      pCallbackParam;       /* Callback parameter */
    USB_TARG_CHANNEL           targChannel;          /* Targ channel */
    LIST                       pipeList;             /* List of the pipes */
    struct usbtgt_tcd *        pTcd;                 /* Attach to the TCD */
    char *                     pFuncName;            /* Function driver name */
    char *                     pTcdName;             /* Used for the TCD's name */
    UINT8                      uFuncUnit;            /* Function driver unit */
    UINT8                      uTcdUnit;             /* Used for the TCD's unit */
    UINT8                      uConfigToBind;        /* The configuration group in */
    UINT8                      uCurrentConfig;       /* The configuration num included in *
/
    pUSBTGT_CONFIG             pConfig;
    UINT8 *                    pDescBuf;
    UINT16                     uDescBufLen;
    UINT8                      uIfNumUsed;           /* How many ifterface in this function */
    UINT8                      uIfNumMin;            /* The min interface num */
    UINT8                      uIfNumMax;            /* The max interface num */
    }USBTGT_FUNC_DRIVER, *pUSBTGT_FUNC_DRIVER;
```

The following items describe the major elements of the **USBTGT_FUNC_DRIVER** data structure.

2.1 usbtgt_func_driver::funcdrivernode

This element is one list node which is used to link the function driver in the global function driver list **gUsbTgtFuncDriverList**. So others, such as TCD can find it to connect it with each other.

2.2 usbtgt_func_driver::funcintcdlistnode

This element is one list node which is used to link the function driver in the TCD function driver list **USBTGT_TCD::funcList**. When the function driver was found marched with the right TCD which to be attached. The target management will link the function driver to the TCD's function list.

2.3 usbtgt_func_driver::pdev

USBTGT_FUNC_DRIVER::pDev specifics the VxBus device instance, if this function driver compatible with the VxBus.

2.4 usbtgt_func_driver::funcmutex

This is a mutex semaphore which is used to protect the **USBTGT_FUNC_DRIVER** elements being concurrent updated.

2.5 usbtgt_func_driver::ufuncmagicid

This element is used to mark the valid function driver type. The user can see what the function driver it is.

2.6 usbtgt_func_driver::pfuncspecific

USBTGT_FUNC_DRIVER::pFuncSpecific is the function driver specific data structure pointer, it used to identify the validation of the function driver which can be used or not.

2.7 **usbtgt_func_driver::ppfsfuncdesctable**, **usbtgt_func_driver::pphsfuncdesctable**

USBTGT_FUNC_DRIVER::ppSsFuncDescTable

Those elements are the full, high and super speed function descriptors table. They contains the interface descriptors and endpoint descriptors and other descriptors which will be added to the global configuration descriptor of the target. And the TML will use this information to require the endpoint resource from the TCD.

For example:

This is the RNDIS function driver which contains 2 interfaces and 3 endpoints. And the descriptors table is configured as following.

```
LOCAL pUSB_DESCR_HDR gUsbTgtRndisDescHdr[] =
    {
    (pUSB_DESCR_HDR )&gUsbTgtRndisCommIfDescr,       /* Communication Interface descriptor *
/
    (pUSB_DESCR_HDR )&gUsbTgtRndisCommEndPtDescr,    /* Interrupt Endpoint descriptor */
    (pUSB_DESCR_HDR )&gUsbTgtRndisDataIfDescr,       /* Data Interface descriptor */
    (pUSB_DESCR_HDR )&gUsbTgtRndisDataInEndPtDescr,  /* Data IN Endpoint descriptor */
    (pUSB_DESCR_HDR )&gUsbTgtRndisDataOutEndPtDescr, /* Data OUT Endpoint descriptor */
    NULL                                             /* End of descriptors, must have */
    };
```

The TML will use this information to require 2 interface numbers and 3 endpoints can be used from the TCD.

USBTGT_FUNC_DRIVER::ppHsFuncDescTable is the function descriptors table for working at high speed. Normally, only the endpoints max packet size different with the **USBTGT_FUNC_DRIVER::ppFsFuncDescTable**.

2.8 **usbtgt_func_driver::pfunccallbacktable** and **usbtgt_func_driver::pcallbackparam**

Those two elements are the callback function table and callback parameters which are used to manage the function driver and deal with the control requests(both standard requests and class/vendor requests).

Since the TML has be do some standard request in usbTgtControlRequest.c, the callback table no need fill all the elements.

Follow is the **USB_TARG_CALLBACK_TABLE** structure definition.

```
/* USB_TARG_CALLBACK_TABLE */

typedef struct usbTargCallbackTable
    {
    /* device management callbacks */

    USB_TARG_MANAGEMENT_FUNC        mngmtFunc; /* management callback */

    /* Control pipe callbacks */

    USB_TARG_FEATURE_CLEAR_FUNC     featureClear;   /* feature clear */
    USB_TARG_FEATURE_SET_FUNC       featureSet;     /* feature set */
    USB_TARG_CONFIGURATION_GET_FUNC configurationGet; /*configuration get*/
    USB_TARG_CONFIGURATION_SET_FUNC configurationSet; /*configuration set*/
    USB_TARG_DESCRIPTOR_GET_FUNC    descriptorGet;  /* descriptor get */
    USB_TARG_DESCRIPTOR_SET_FUNC    descriptorSet;  /* descriptor set */
    USB_TARG_INTERFACE_GET_FUNC     interfaceGet;   /* interface get */
```

```
    USB_TARG_INTERFACE_SET_FUNC     interfaceSet;   /* interface set */
    USB_TARG_STATUS_GET_FUNC        statusGet;      /* status get */
    USB_TARG_ADDRESS_SET_FUNC       addressSet;     /* address set */
    USB_TARG_SYNCH_FRAME_GET_FUNC   synchFrameGet;  /* frame get */
    USB_TARG_VENDOR_SPECIFIC_FUNC   vendorSpecific; /* vendor specific */
    } USB_TARG_CALLBACK_TABLE, *pUSB_TARG_CALLBACK_TABLE;
```

2.9 **usbtgt_func_driver::targchannel**

USBTGT_FUNC_DRIVER::targChannel is used for multiple function drivers used on one TCD to indicate different instance. It can also be used to compatible with the GEN1 target stack.

2.10 **usbtgt_func_driver::pipelist**

USBTGT_FUNC_DRIVER::pipeList is the **USBTGT_PIPE** list which likes all the created **USBTGT_PIPE** for the function driver by calling the routine **usbTgtCreatePipe**.

2.11 **usbtgt_func_driver::ptcd**

USBTGT_FUNC_DRIVER::pipeList specifics the **USBTGT_TCD** attached by the function driver function driver.

2.12 **usbtgt_func_driver::pfuncname** and **usbtgt_func_driver::pfuncname**

Those two elements are used to identify the function driver name and unit which configured by the user. .

2.13 **usbtgt_func_driver::ptcdname** and **usbtgt_func_driver::ptcdname**

Those two elements are used to identify the **USBTGT_TCD** name and unit which need be attached by the function driver.

2.14 **usbtgt_func_driver::uconfigtobind** and **usbtgt_func_driver::ucurrentconfig**

USBTGT_FUNC_DRIVER::uConfigToBind specifics the configuration number which the function driver will be build in. If the user do not set this, the default value configuration #1 will be used.

USBTGT_FUNC_DRIVER::uCurrentConfig specifics the current configuration the function driver build in. When there are multiple function drivers need build in to one configuration, and the hardware endpoint resource may be not enough for more function driver. So the function drive can not be used correctly. This design also wants to cover such case. It will build the function driver into a new configuration automatically. But the problem is the Host side driver need be changed to switch the configuration if the user wants to use the function driver.

2.15 **usbtgt_func_driver::pconfig**

USBTGT_FUNC_DRIVER::pConfig specifics the current configuration pointer which build in.

2.16 **usbtgt_func_driver::pdescbuf** and **usbtgt_func_driver::udescbuflen**

Those two elements are used to store the function driver descriptors which will linked to the configuration descriptors.

2.17 **usbtgt_func_driver::uifnumused**

USBTGT_FUNC_DRIVER::uIfNumUsed specifics how many interface count occupied by the function driver.

2.18 **usbtgt_func_driver::uifnummin** and **usbtgt_func_driver::uifnummax**

USBTGT_FUNC_DRIVER::uIfNumMin specifics the min interface number occupied by the function driver.Meanwhile **USBTGT_FUNC_DRIVER::uIfNumMax** specifics the max interface number occupied by the function driver.

3. **usbtgt_tcd_funcs**

Each USB Target Controller will have a **USBTGT_TCD_FUNCS** data structure, which as the element of the **USBTGT_TCD::pTcdFuncs**. This elements contains the interfaces between the TML and the **USBTGT_TCD**. The structure is defined as below:

```
/*
 * Target Controller Driver abstraction
 */

typedef struct usbtgt_tcd_funcs
    {
    /*
     * Enable the Target Controller so that the hardware can be considered
     * as running!
     */

    STATUS (* pTcdEnable)(struct usbtgt_tcd * pTcd);

    /*
     * Disable the Target Controller so that the hardware can be considered
     * as stopped!
     */

    STATUS (* pTcdDisable)(struct usbtgt_tcd * pTcd);
    /*
     * Suspend the Target Controller hardware
     */

    STATUS (* pTcdSuspend)(struct usbtgt_tcd * pTcd);

    /*
     * Resume the Target Controller hardware
     */

    STATUS (* pTcdResume)(struct usbtgt_tcd * pTcd);

    /*
     * Wakeup the Target Controller hardware
     */

    STATUS (* pTcdWakeup)(struct usbtgt_tcd * pTcd);

    /*
     * Disconnect the Target Controller hardware, maybe NULL
     */


    STATUS (* pTcdDisconnect)(struct usbtgt_tcd * pTcd);

    /*
```

```
     * Reset the Target Controller hardware
     */

    STATUS (* pTcdReset)(struct usbtgt_tcd * pTcd);

    /*
     * Get the Frame Number of the Target Controller hardware
     */

    STATUS (* pTcdGetFrameNum)(struct usbtgt_tcd * pTcd,UINT16 * pFrameNum );

    /*
     * Soft Connect/Disconnect of the Target Controller hardware
     */

    STATUS (* pTcdSoftConnect)(struct usbtgt_tcd * pTcd,BOOL isConnectUp);

    /*
     * TCD interface to create pipe, which used by the TML
     */

    STATUS (* pTcdCreatePipe)
        (
        struct usbtgt_tcd * pTcd,
        pUSB_ENDPOINT_DESCR pEndpointDesc,
        UINT16              uConfigurationValue,
        UINT16              uInterface,
        UINT16              uAltSetting,
        pUSB_TARG_PIPE      pPipeHandle
        );

    /*
     * TCD interface to delete pipe, which used by the TML
     */

    STATUS (* pTcdDeletePipe)
        (
        struct usbtgt_tcd * pTcd,
        USB_TARG_PIPE       pipeHandle
        );

    /*
     * TCD interface to submit ERP, which used by the TML
     */

    STATUS (* pTcdSubmitErp)
        (
        struct usbtgt_tcd * pTcd,
        USB_TARG_PIPE       pipeHandle,
        pUSB_ERP            pErp
        );

    /*
     * TCD interface to cancel ERP, which used by the TML
     */

    STATUS (* pTcdCancelErp)
```

```
        (
        struct usbtgt_tcd * pTcd,
        USB_TARG_PIPE       pipeHandle,
        pUSB_ERP            pErp
        );

    /*
     * TCD interface to get status of the pipe, which used by the TML
     */

    STATUS (* pTcdPipeStatusSet)
        (
        struct usbtgt_tcd * pTcd,
        USB_TARG_PIPE       pipeHandle,
        UINT16              uStatus
        );

    /*
     * TCD interface to set status of the pipe, which used by the TML
     */

    STATUS (* pTcdPipeStatusGet)
        (
        struct usbtgt_tcd * pTcd,
        USB_TARG_PIPE       pipeHandle,
        UINT16 *            pStatus
        );

    /*
     * TCD interface to notifiy the USB irq, which used by the TML
     */

    void   (* usbIsrNotify)(struct usbtgt_tcd * pTcd, void * pContext);

    /*
     * TCD interface to notifiy the DMA irq, which used by the TML
     */

    void   (* dmaIsrNotify)(struct usbtgt_tcd * pTcd, void * pContext);

    /*
     * TCD interface Ioctl interface, which used by the TML
     */

    int  (* pTcdIoctl)(struct usbtgt_tcd * pTcd,int cmd, void * pContext);

    }USBTGT_TCD_FUNCS, *pUSBTGT_TCD_FUNCS;
```

The following items describe the major elements of the **USBTGT_TCD_FUNCS** data structure.

3.1 **usbtgt_tcd_funcs::ptcdenable** and **usbtgt_tcd_funcs::ptcddisable**

Those two elements are used by the OTG stack, which can be used to switch the Host stack and the Target Stack.

3.2 **usbtgt_tcd_funcs::ptcdsuspend** and **usbtgt_tcd_funcs::ptcdresume**

Those two elements are not must have. They are used to suspend/resume the USB target controller.

3.3 **usbtgt_tcd_funcs::ptcdwakeup**

This element is not must have. It is used to issue the remote Wakeup to the USB host.

3.4 **usbtgt_tcd_funcs::ptcdreset**

This element is not must have. It is used to issue the reset by force to the USB target controller.

3.5 **usbtgt_tcd_funcs::ptcdgetframenum**

This element is not must have, but better have. It is used to get the current frame number from the USB target controller. It may used in some ISO transaction.

3.6 **usbtgt_tcd_funcs::ptcdsoftconnect**

This element is not must have, but better have. This should according to the real hardware. It is used to issue the soft connect/disconnect of the USB target controller. Some hardware support this feature to simulate the connect/disconnect the USB target to/from the USB host.

Note

And some hardware will be in disconnect state after BUS RESET, the software should do the soft connect to make the USB target controller work. So this element should not be **NULL** in this case.

USBTGT_TCD_FUNCS::pTcdSoftConnect also can be used in case that the user need dynamicly attach/detach the function driver. The TML need to do the soft disconnect first and attach/detach the function drivers then do soft connect to make the USB target be updated.

3.7 **usbtgt_tcd_funcs::ptcdcreatepipe**

Must have, this element is used by the TML to issue the pipe create to require the hardware resource such as occuping the endpoint. Please go to the **usbTgtCreatePipe( )** routine for detail.

3.8 **usbtgt_tcd_funcs::ptcddeletepipe**

Must have, this element is used by the TML to issue the pipe delete to release the hardware resource such as the occupied endpoint. Please go to the **usbTgtDeletePipe( )** routine for detail.

3.9 **usbtgt_tcd_funcs::ptcdsubmiterp**

Must have, this element is used by the TML to issue the one transaction on one endpoint. Please go to the **usbTgtSubmitErp( )** routine for detail.

3.10 **usbtgt_tcd_funcs::ptcdcancelerp**

Must have, this element is used by the TML to cancel the one transaction on one endpoint. Please go to the **usbTgtCancelErp( )** routine for detail.

3.11 **usbtgt_tcd_funcs::ptcdpipestatusset**

Must have, this element is used by the TML to set the status of one endpoint, STALL or UNSTALL.Please go to the **usbTgtSetPipeStatus( )** routine for detail.

3.12 **usbtgt_tcd_funcs::ptcdpipestatusget**

Must have, this element is used by the TML to get the status of one endpoint, STALL or UNSTALL.Please go to the **usbTgtGetPipeStatus( )** routine for detail.

3.13 **usbtgt_tcd_funcs::usbisrnotify**

Not must have, this element is used by the TML to notify the USB target stack one USB interrupt just received, and issue the notification to the USB target ISR to process it.

Note

This can be used by the OTG stack, to issue the interrupt notification to the USB target stack.Since the OTG stack will hand over the IRQ of the USB controller. If the USB target controller do not support OTG, or the OTG stack do not use this interface, the element will be **NULL**.

3.14 **usbtgt_tcd_funcs::dmaisrnotify**

Not must have, this element is used by the TML to notify the USB target stack one USB DMA interrupt just received, and issue the notification to the USB target DMA ISR to process it.

Note

This can be used by the OTG stack, to issue the interrupt notification to the USB target stack.Since the OTG stack will hand over the DMA IRQ of the USB controller. If the USB target controller do not support OTG, or the OTG stack do not use this interface, the element will be **NULL**.

On some platform, the DMA IRQ is separated with the USB IRQ. So DMA ISR notification may needed. But on some platform, which do not have the separated DMA IRQ, this elements will be **NULL**.

3.15 **usbtgt_tcd_funcs::ptcdioctl**

This is the ioctl interface used by the TML, which also can be used to compatible with the GEN1 target stack.

How to add one tcd to tml

Note

Before one valid target controller driver (TCD) ready to used, it must have been initialized and added to the target management layer.

1.create **usbtgt_tcd** data structure

The routine **usbTgtTcdResourceCreate( )** is used to create the **USBTGT_TCD** data structure and add itself to the global TCD list **gUsbTgtTcdList**.With return a valid **USBTGT_TCD** pointer or **NULL** of something wrong.
Now, the resource can be used by the hardware TCD driver to use.

2.set the user's configuration

Following list the parameters which can be get from the user's configuration.

```
    /* Parameters get from the user's configuration to initialize the TCD */

    UINT32                uTCDMagicID;  /* TCD magic ID */
    UINT16                uVendorID;    /* VID */
    UINT16                uProductID;   /* PID */
    char *                pMfgString;   /* Manufacture string */
    char *                pProdString;  /* Producture string */
    char *                pSerialString;/* Serial num string */
    char *                pTcdName;     /* TCD name */
    UINT8                 uTcdUnit;     /* Tcd unit number */
```

uTCDMagicID

The magic ID to identify the TCD type

uVendorID

The vendor ID of the USB target

uProductID

The product ID of the USB target

pMfgString

The manufacture string of the USB target

pProdString

The manufacture string of the USB target

pSerialString

The serial string of the USB target

pTcdName

The tcd name to indentify the hardware

uTcdUnit

The tcd unit number to indentify the hardware

Note

There maybe other parameters need be set by the user which not be included. The developer should configure them in the same way.

3.initialize the tcd

The USB target controller driver(TCD) who need be intialized and registered to the target management layer(TML). Please refor to the **usrUsbTgtTcdInit( )**.

3.1 get one unused tcd resource with the magic id

Get the TCD resource with the magic ID by calling **usbTgtTcdResourceRequest( )**.
The magic ID to identify the TCD type. **pTcdSpecific** be **NULL** indicate to get one unused resource by others.

NOTE: This step is not necessary, if the user directly use the resource created by step 1.

3.2 initialize the tcd related parameters

Followed are the parameters of **USBTGT_TCD** data structure (please refer to the **usbTgt.h** for detail). Those parameters need be initialized in the TCD initialization routines before add to the TML.

```
    /* Parameters which need be initalized in the TCD */

    struct vxbDev *       pDev;         /* Pointer to VxBus Device instance */
    USBTGT_TCD_FUNCS *    pTcdFuncs;    /* Functions implemented by the TCD */
    pVOID                 pTcdSpecific; /* Pointer to TCD specific structure */
    USBTGT_DEVICE_INFO    DeviceInfo;   /* Device information */
```

pDev

VxBus device instance, if this TCD compatible with the VxBus

pTcdFuncs

The USB target controller driver(TCD) function table, which contains some useful routines such as submitErp function pointer which can be called by the TML. Must have.

pTcdSpecific

This is the TCD specific data structure pointer, it used to identify the real hardware.Used by the TML to check if this TCD has bind with the hardware or not. Must have.

DeviceInfo

This data structure specifics the TCD hardware feature.Such as the endpoints features(Supported Tx and Rx endpoints) and controller features.

4.add the tcd to the tml

Add the TCD to the TML by calling the routine **usbTgtTcdAdd( )**. This routine will check if there are any function driver need attach to it. If yes, it will attach all the function drivers onto it, and generate the device characters such as device descriptor and configuration descriptor ready for using.

How to add function driver to tml

NOTE:Before one valid function driver ready to used, it must have been initialized and added to the target management layer.

1.create **usbtgt_func_driver** data structure

The routine **usbTgtFuncResourceCreate( )** is used to create one USBTGT_FUNC_DRIVER.data structure and add itself to the global function driver list **gUsbTgtFuncDriverList**.With return a valid **USBTGT_FUNC_DRIVER** pointer if successed or **NULL** of something wrong. called in the **usrTgtInit.c**, and the user's configration will be set when it be initialized.

2.set the user's configuration

Following list the parameters which can be get from the user's configuration.

```
/* Parameters get from the user's configuration to initalize the TCD */
UINT32                    uFuncMagicID;         /* Func magic Id */
char *                    pTcdName;             /* Used for the TCD's name */
UINT8                     uTcdUnit;             /* Used for the TCD's unit */
UINT8                     uConfigToBind;        /* The configuration group in */
```

uTCDMagicID

The magic ID to identify the function driver type

pTcdName

The TCD name witch need to be attached.Must have, or the function driver will do not know which TCD to be attached. The target stack will use the user's configuration, or use the default value.

uTcdUnit

The TCD unit number which need be attached.Must have, or the function driver will do not know which TCD to be attached. The target stack will use the user's configuration, or use the default value.

uConfigToBind

The configuration which the function driver expect to build in.Must have, or the function driver will do not know which configuration to be bind. The target stack will use the user's configuration, or use the default value.

Note

There maybe other parameters need be set by the user which not be included. The developer should configure them in the same way.

3.initialize the function driver

The USB function driver who need be initialized and registered to the target management layer(TML).

3.1 get the function resource with the magic id

Get the function resource with the magic ID by calling **usbTgtFuncResourceRequest( )**. The magic ID to identify the function type. **pFuncSpecific** be **NULL** indicate to get one unused resource by others.

NOTE: This step is not necessary, if the user directly use the resource created by step 1.

3.2 initialize the function driver related parameters

Followed are the parameters of **USBTGT_FUNC_DRIVER** data structure (please refer to the **usbTgt.h**). Those parameters need be initialized in the function initialization routines before add to the TML.

```
/* Parameters which need be initalized in the TCD */

pVOID                     pFuncSpecific;      /* Specific data */
USB_DESCR_HDR **          ppFsFuncDescTable;  /* Function descriptors table */
USB_DESCR_HDR **          ppHsFuncDescTable;  /* Function descriptors table in High spee
```

```
d mode */
    USB_DESCR_HDR **           ppSsFuncDescTable;  /* Function descriptors table in super spee
d mode */
    USB_TARG_CALLBACK_TABLE *  pFuncCallbackTable; /* Function driver callback table */
    pVOID                      pCallbackParam;     /* Callback parameter */
```

pFuncSpecific

This is the function driver specific data structure pointer, it used to used by the TML to check if this function driver has bind with the hardware or not, and also used by the function driver itself. Must have.

ppFsFuncDescTable

The descriptors table which can describe the characters of the function driver. Such as interface descriptors to describ the Class/Subclass/Protocol the function driver use.

ppHsFuncDescTable

The descriptors table which can describe the characters of the function driver. Such as interface descriptors to describe the Class/Subclass/Protocol the function driver use. This table describe the characters in the high speed.

ppSsFuncDescTable

The descriptors table which can describe the characters of the function driver. Such as interface descriptors to describe the Class/Subclass/Protocol the function driver use. This table describe the characters in the super speed.

pFuncCallbackTable

The function driver's callback routines table.

pCallbackParam

The function driver's callback routines parameter.

4.add the function driver to the tml

Add the function to the TML by calling the routine **usbTgtFuncAdd( )**. This routine will check if the exactly TCD exist or not who has the right pTcdName and uTcdUnit. If yes, It will attach them together by **usbTgtFuncAttach( )**.

How to detach one function driver from tcd

The user may want to dynamically remove one attached function driver from one TCD.Which can switch the TCD emulate as other function else. Such as switch the Mass storage device to RNDIS device.

The routine **usbTgtFuncRemove** can achieve this. It will detach the function driver from the TCD, and also call the detach call back routine **usbTgtFuncMngmtCallback** to notify the function driver to detach itself from the pUSBTGT_FUNC_DRIVER resource. Then it will remove the connection from the TCD which has been attached.

The user can find the **pUSBTGT_FUNC_DRIVER** resource by name and remove it by the **usbTgtFuncRemove** routine.

Note

soft disconnect will be seen on the BUS.

How to attach one function driver to tcd

The user may want to dynamically add one detached function driver to one TCD which asigned.Which can switch the TCD emulate as other function else. Such as switch the Mass storage device to RNDIS device.

The routine **usbTgtFuncAdd** can achieve this. It will attach the function driver to the TCD, and also call the attach call back routine **usbTgtFuncMngmtCallback** to notify the function driver to attach itself to the pUSBTGT_FUNC_DRIVER resource. Then it will add the connection to the TCD which has been attached.

The user can find the **pUSBTGT_FUNC_DRIVER** resource by name and add it by the **usbTgtFuncAdd** routine.

Note

soft connect will be seen on the BUS.

Include files

vxWorks.h lstLib.h usb/usb.h usb/usbCdc.h usb/usbOsal.h, **usb/usbTgt.h usbTgtControlRequest.h**

Name

usbTgtTargChannelToFuncDriver( ) - find the pFuncDriver according to targChannel

Layer

USB_TARGET

Synopsis

```
pUSBTGT_FUNC_DRIVER usbTgtTargChannelToFuncDriver
    (
    USB_TARG_CHANNEL targChannel
    )
```

Description

This routine is used to find the 'pFuncDriver" according to **targChannel**

Returns

NULL, or a valid pUSBTGT_FUNC_DRIVER pointer

Errno

N/A

See also

usbTgt

Name

usbTgtTcdVxDevGet( ) - get the Tcd's vxbDev pointer

Layer

USB_TARGET

Synopsis

```
struct vxbDev * usbTgtTcdVxDevGet
    (
    USB_TARG_CHANNEL targChannel
    )
```

Description

This routine is used to find the Tcd's vxbDev pointer according to **targChannel**

Returns

NULL, or a valid vxbDev pointer

Errno

N/A

See also

usbTgt

Name

usbTgtCreatePipe( ) - create a pipe for communication on an endpoint

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtCreatePipe
    (
    USB_TARG_CHANNEL    targChannel,           /* Target channel */
    pUSB_ENDPOINT_DESCR pEndpointDesc,         /* USB_ENDPOINT_DESCR */
    UINT8               uConfigurationValue,   /* Configuration value */
    UINT8               uInterface,            /* Number of interface */
    UINT8               uAltSetting,           /* Alternate Setting */
    pUSB_TARG_PIPE      pPipeHandle            /* Pointer to pipe handle */
    )
```

Description

This routine creates a pipe for communication on an endpoint. The endpoint's information such as the address and max packet size, can be get from the **pEndpointDesc**. It will call the TCD specific **pTcdCreatePipe** to create one pipe handle which be set as the **pUsbTgtPipe-pipeHandle**> value, and return one valided pUSB_TARG_PIPE to the caller.

This routine will also record the pipe in the function driver's pipe list **pFuncDriver-pipeList**>.

Returns

OK, or **ERROR** if unable to create pipe

Errno

N/A

See also

usbTgt

Name

usbTgtDeletePipe( ) - delete an pipe created for an endpoint

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtDeletePipe
    (
    USB_TARG_PIPE pipeHandle   /* pipe to be destroyed */
    )
```

Description

This routine deletes an pipe created for one edpoint created by **usbTgtCreatePipe( )** routine.This routine will call the delete pipe routine which registered in the TCD driver table and remove the record of the pipe in the function driver.

Returns

OK, or **ERROR** if unable to destroy pipe.

Errno

N/A

See also

usbTgt

Name

usbTgtUnConfigPipe( ) - un-configure a pipe for communication on an endpoint

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtUnConfigPipe
    (
    USB_TARG_PIPE pipeHandle
    )
```

Description

This routine creates a pipe for communication on an endpoint. The endpoint's information such as the address and max packet size, can be get from the **pEndpointDesc**. It will call the TCD specific **pTcdCreatePipe** to create one pipe handle which be set as the **pUsbTgtPipe-pipeHandle**> value, and return one valided pUSB_TARG_PIPE to the caller.

This routine will also record the pipe in the function driver's pipe list **pFuncDriver-pipeList**>.

Returns

OK, or **ERROR** if unable to create pipe

Errno

N/A

See also

usbTgt

Name

usbTgtConfigPipe( ) - configure a pipe for communication on an endpoint

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtConfigPipe
    (
    USB_TARG_PIPE             pipeHandle,   /* Pipe handle */
    pUSBTGT_PIPE_CONFIG_PARAM pParam        /* Pointer to pipe handle */
    )
```

Description

This routine creates a pipe for communication on an endpoint. The endpoint's information such as the address and max packet size, can be get from the **pEndpointDesc**. It will call the TCD specific **pTcdCreatePipe** to create one pipe handle which be set as the **pUsbTgtPipe-pipeHandle**> value, and return one valided pUSB_TARG_PIPE to the caller.

This routine will also record the pipe in the function driver's pipe list **pFuncDriver-pipeList**>.

Returns

OK, or **ERROR** if unable to create pipe

Errno

N/A

See also

usbTgt

Name

usbTgtSubmitErp( ) - submit one Erp to transfer data through the pipe

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtSubmitErp
    (
    USB_TARG_PIPE pipeHandle,   /* Handle to the pipe */
    pUSB_ERP      pErp          /* ERP to be transfered */
    )
```

Description

This routine is used to submit one Erp to transfer data through a pipe indicated by pipeHandle. The transfer is described by an ERP, or endpoint request packet, which must be allocated and initialized by the caller.This routine will call the submit ERP routine which registered in the TCD driver table

Returns

OK or Error if unable to transfer data.

Errno

N/A

See also

usbTgt

Name

usbTgtCancelErp( ) - cancel a previously submitted **USB_ERP**

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtCancelErp
    (
    USB_TARG_PIPE pipeHandle,   /* pipe for transfer to abort */
    pUSB_ERP      pErp          /* ERP to be aborted */
    )
```

Description

This routine cancels an ERP which was previously submitted through a call to **usbTgtSubmitErp( )**. This routine will call the cancel ERP routine which registered in the TCD driver table.

Returns

OK, or **ERROR** if unable to cancel **USB_ERP**

Errno

N/A

See also

usbTgt

Name

usbTgtSetPipeStatus( ) - set pipe stalled/unstalled status

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtSetPipeStatus
    (
    USB_TARG_PIPE pipeHandle,   /* Handle to the pipe */
    UINT16        state         /* State of the pipe to be set */
    )
```

Description

This routine sets the status of the specific pipe. If the target application detects an error while servicing a pipe, it may choose to stall the endpoint associated with that pipe.This routine allows the caller to set the state of a pipe as "stalled" or "un-stalled".

Returns

OK, or **ERROR** if unable to set indicated state

Errno

N/A

See also

usbTgt

Name

usbTgtGetPipeStatus( ) - get the status of the pipe

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtGetPipeStatus
    (
    USB_TARG_PIPE pipeHandle,   /* Handle to the pipe */
    UINT16 *      pBuf          /* Buffer to hold the pipe status */
    )
```

Description

This routine is used to get the status of the pipes per **GET_STATUS** request. The status of the pipe is stored in the pointer variable pBuf

Returns

OK, or **ERROR** if unable to get the status

Errno

N/A

See also

usbTgt

Name

usbTgtTcdSoftConnect( ) - do soft connect of the TCD

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtTcdSoftConnect
    (
    pUSBTGT_TCD pTcd,
    BOOL        bConnectUp
    )
```

Description

This routine is used to do soft connect of the TCD. bConnectUp is used to indicate to connect (set to be **TRUE**) or disconnect (clear to be **FALSE**).

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtFuncMngmtCallback( ) - do the function driver management callback

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncMngmtCallback
    (
    pUSBTGT_FUNC_DRIVER pFuncDriver,
    UINT16              mngmtCode,
    void *              pContext
    )
```

Description

This routine is used to do the function driver management callback with the management code **mngmtCode** and **pContext**.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtErpCallback( ) - invoked when ERP completes

Layer

USB_TARGET

Synopsis

```
VOID usbTgtErpCallback
    (
    pVOID pErp   /* ptr to ERP */
    )
```

Description

The TCD invokes this callback when an ERP completes. This gives us the opportunity to monitor ERP execution. We reflect the callback to the calling application after we finish our processing.

Returns

N/A

Errno

N/A

See also

usbTgt

Name

usbTgtFuncRemoteWakeUp( ) - issue a remote wakeup signal from the USB device

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncRemoteWakeUp
    (
    pUSBTGT_FUNC_DRIVER pFuncDriver
    )
```

Description

This routine is used to issue a remote wakeup signal from the USB device.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtFuncFrameNumGet( ) - get the current frame number from the TCD

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncFrameNumGet
    (
    pUSBTGT_FUNC_DRIVER pFuncDriver,
    UINT16 *            pFrameNum
    )
```

Description

This routine is used to iget the current frame number from the USB target controller. It mainly be used in the ISO transaction.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtFuncControlPipeStall( ) - stall the function driver's control pipe

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncControlPipeStall
    (
```

```
    pUSBTGT_FUNC_DRIVER pFuncDriver
    )
```

Description

This routine is used to stall the function driver's control pipe. It will be used when someting wrong on the control pipe when process some class/vendor requests.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtFuncControlSetupErpSubmit( ) - submit the SETUP pecket on the control pipe

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncControlSetupErpSubmit
    (
    USB_TARG_CHANNEL targChannel
    )
```

Description

This routine is used to submit the SETUP pecket on the control pipe by the function drivers which is ready for processing next class or vendor request.

Note

targChannel should be indicate to the pUSBTGT_FUNC_DRIVER pointer

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtFuncControlDataErpSubmit( ) - submit the DATA IN/OUT stage transaction

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncControlDataErpSubmit
    (
    USB_TARG_CHANNEL targChannel,    /* Target channel */
    UINT16           bfrLen,         /* Length of response 0 */
    pUINT8           pBfr,           /* Pointer to the data buffer */
    ERP_CALLBACK     userCallback,   /* USB Target Applcaition Callback */
    BOOL             bDataIn,        /* Is this for data in erp */
    USB_ERP_FLAGS    erpFlags        /* The ERP transaction flags */
    )
```

Description

This routine is used to submit the DATA IN/OUT stage transaction on the control pipe by the function drivers which issues the DATA stage transaction after received and processed one class/vendor request.

Note

targChannel should be indicate to the pUSBTGT_FUNC_DRIVER pointer bfrLen is the length of the data to be send pBfr is the pointer of the buffer to store the data userCallback is used as the client callback by the function driver bDataIn is used to indicate to issue the DATA IN/OUT stage erpFlags indicates the ERP transaction flags, such as whether one ZLP pecket will be needed after all the data transfered.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtFuncControlStatusErpSubmit( ) - submit the STATUS stage ERP on control pipe

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncControlStatusErpSubmit
    (
    USB_TARG_CHANNEL targChannel,   /* Target channel */
    BOOL             bStatusIn      /* Is status in */
    )
```

Description

This routine is used to submit the STATUS stage ERP on the control to the host by the function drivers, which issues the DATA STAGE transaction after received and processed one class/vendor request or finished the DATA stage. bStatusIn is the parameter to indicate the direction STATUS IN/OUT.

Note

targChannel should be indicate to the pUSBTGT_FUNC_DRIVER pointer bStatusIn indicates the direction of the STATUS stage , STATUS IN/OUT

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtTcdDestroy( ) - destroy the **USBTGT_TCD** resource which unused anymore

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtTcdDestroy
    (
    pUSBTGT_TCD pTcd
    )
```

Description

This routine destroys the **USBTGT_TCD** resource which unused anymore. It will destroy the mutex and delete it from the global list **gUsbTgtTcdList**.

Note

Before this routine is called, the caller should make sure all the resource has destroied by the caller.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtTcdCreate( ) - create one **USBTGT_TCD** resource for target stack

Layer

USB_TARGET

Synopsis

```
pUSBTGT_TCD usbTgtTcdCreate
    (
    pUSBTGT_TCD_INFO pTcdInfo
    )
```

Description

This routine creates one pUSBTGT_TCD resource for target stack and add to the global list**gUsbTgtTcdList** ready for using.

Note

Before this routine is called,the caller need fill up the **USBTGT_TCD_INFO** data structure, which are the parameters to create the **USBTGT_TCD**

Returns

NULL or one valid **USBTGT_TCD** resource point

Errno

N/A

See also

usbTgt

Name

usbTgtFuncDestroy( ) - destroy the target function driver resource

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncDestroy
    (
    pUSBTGT_FUNC_DRIVER pFuncDriver
    )
```

Description

This routine destroys the target function driver resource **USBTGT_FUNC_DRIVER** of the target stack which created by **usbTgtFuncResourceCreate** routine.

Note

Before this routine is called, the caller should make sure all the resource has destroied by the caller.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtFuncCreate( ) - create one target function driver resource

Layer

USB_TARGET

Synopsis

```
pUSBTGT_FUNC_DRIVER usbTgtFuncCreate
    (
```

```
    pUSBTGT_FUNC_INFO pFuncInfo
    )
```

Description

This routine creates one target function driver resource pUSBTGT_FUNC_DRIVER for target and add to the global list**gUsbTgtFuncDriverList** ready for using.

Note

Before this routine is called,the caller need fill up the **USBTGT_FUNC_INFO** data structure, which are the parameters to create the **pUSBTGT_FUNC_DRIVER**

Returns

NULL or target function driver resource point pUSBTGT_FUNC_DRIVER

Errno

N/A

See also

usbTgt

Name

usbTgtTcdEventNotify( ) - send one Tcd event notification

Layer

USB_TARGET

Synopsis

```
void usbTgtTcdEventNotify
    (
    pUSBTGT_TCD        pTcd,
    USBTGT_NOTIFY_CODE NotifyCode
    )
```

Description

This routine is used to send one Tcd event notification to the target management.It will call the event handler according to the **NotifyCode** code.

Returns

N/A.

Errno

N/A

See also

usbTgt

Name

usbTgtFindFuncByName( ) - find the function driver pointer by name and unit number

Layer

USB_TARGET

Synopsis

```
pUSBTGT_FUNC_DRIVER usbTgtFindFuncByName
    (
    char * pFuncName,
    UINT8  uFuncUnit
    )
```

Description

This routine finds the function driver pointer from the global list **gUsbTgtFuncDriverList** by funtion name **pFuncName**and the unit number **uFuncUnit**.

Returns

pointer of the function driver or **NULL** if no found

Errno

N/A

See also

usbTgt

Name

usbTgtFindTargChannelByName( ) - find function targChannel by name and unit num

Layer

USB_TARGET

Synopsis

```
USB_TARG_CHANNEL usbTgtFindTargChannelByName
    (
    char * pFuncName,
    UINT8  uFuncUnit
    )
```

Description

This routine finds the function driver's targChannel from the global list **gUsbTgtFuncDriverList** by funtion name **pFuncName**and the unit number **uFuncUnit**.

Returns

USBTGT_TARG_CHANNEL_DEAD, or valid function driver targChannel

Errno

N/A

See also

usbTgt

Name

usbTgtFindTcdByName( ) - find the Tcd pointer by the name and unit number

Layer

USB_TARGET

Synopsis

```
pUSBTGT_TCD usbTgtFindTcdByName
    (
    char * pTcdName,
    UINT8  uTcdUnit
    )
```

Description

This routine finds the target pointer from the global list**gUsbTgtTcdList** by the name **pTcdName** and unit number **uTcdUnit**

Returns

pointer to the target or **NULL** if no found

Errno

N/A

See also

usbTgt

Name

usbTgtFindTcdByName( ) - find the Tcd pointer by the name and unit number

Layer

USB_TARGET

Synopsis

```
USBTGT_TCD_HANDLE usbTgtFindTcdHandleByName
    (
    char * pTcdName,
    UINT8  uTcdUnit
    )
```

Description

This routine finds the target pointer from the global list**gUsbTgtTcdList** by the name **pTcdName** and unit number **uTcdUnit**

Returns

pointer to the target or **NULL** if no found

Errno

N/A

See also

usbTgt

Name

usbTgtFuncRemove( ) - remove the function driver from the target stack.

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncRemove
    (
    pUSBTGT_FUNC_DRIVER pFuncDriver
    )
```

Description

This routine removes the function driver from the target stack. It also will detach itslef from the tcd if they have been attached together.

Note

This routine will not destroy the **pUSBTGT_FUNC_DRIVER** data structure. It also will not delelte the **pFuncDriver** from the global function driver list. It is used to remove the function driver from the TML, and detach it from the TCD which attached to it. Meanwhile, it will reset the **pFuncSpecific** item to **NULL**.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtFuncAdd( ) - add the function driver to the target stack

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncAdd
    (
    pUSBTGT_FUNC_DRIVER pFuncDriver
    )
```

Description

This routine adds the function driver to the target stack. It also will check and attach to the specific tcd if it exists in the target stack.

This routine will first check the pTcd exist or not according to the pTcdName and uTcdUnit. It also will check if the resource of the tcd can satisfy the function driver's requirement or not (Such as if there are enough endpoints for the function driver to use.)

Note

Before this routine is called, the caller should make sure the function resource has been initialized. That is the function driver knows which tcd will be attached to.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtTcdRemove( ) - remove one Target controller driver from the the target stack.

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtTcdRemove
    (
    pUSBTGT_TCD pTcd
    )
```

Description

This routine removes one Target controller driver from the the target stack. It will search the global function driver list to find if there is any function drivers need be detached from the target controller driver (TCD). If yes, detach them.

The target stack will automiticlly flush the Device descriptors and the configuration descriptors which used to indicate the USB device chararactics.

Note

This routine will not destroy the **pUSBTGT_TCD** data structure, will not delelte the **pTcd** from the global TCD list. It is used to remove the low level target controller driver from the TML, and detach all the function driver's which attached to it. Meanwhile, it will reset the **pTcdSpecific** item to **NULL**.

Returns

OK, or **ERROR** if there is something wrong.

Errno

N/A

See also

usbTgt

Name

usbTgtTcdAdd( ) - add one target controller driver in the the target stack.

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtTcdAdd
    (
```

```
    pUSBTGT_TCD pTcd
    )
```

Description

This routine adds one target controller driver in the the target stack. It will search the global function driver list to find if there is any function drivers need be attached to the target controller driver. If yes, attach them.

The target stack will automiticlly generate the Device descriptors and the configuration descriptors which will be used to indicate the USB device characteristics.

Returns

OK, or **ERROR** if there is something wrong.

Errno

N/A

See also

usbTgt

Name

usbTgtFuncUnRegister( ) - un-register the function driver from the target stack

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncUnRegister
    (
    USB_TARG_CHANNEL targChannel
    )
```

Description

This routine un-registers the function driver from the target stack. It will detach the function driver from the right TCD, and destroy all the data structure resource.

Returns

always **OK**

Errno

N/A

See also

usbTgt

Name

usbTgtFuncRegister( ) - register the function driver to the target stack

Layer

USB_TARGET

Synopsis

```
USB_TARG_CHANNEL usbTgtFuncRegister
    (
    pUSBTGT_FUNC_INFO pFuncInfo
    )
```

Description

This routine registers the function driver to the target stack.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtTcdUnRegister( ) - un-register the TCD from the target stack

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtTcdUnRegister
    (
    pUSBTGT_TCD pTcd
    )
```

Description

This routine un-registers the TCD from the target stack. It will detach all the function driver from the right TCD, and destroy the TCD data structure resource.

Returns

always **OK**

Errno

N/A

See also

usbTgt

Name

usbTgtTcdRegister( ) - register the tcd controller to the target stack

Layer

USB_TARGET

Synopsis

```
pUSBTGT_TCD usbTgtTcdRegister
    (
    pUSBTGT_TCD_INFO pTcdInfo
    )
```

Description

This routine registers the tcd controller to the target stack.

Returns

NULL, or valid pUSBTGT_TCD pointer

Errno

N/A

See also

usbTgt

Name

usbTgtExit( ) - uninitialize the global usb target resource

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtExit
    (
    void
    )
```

Description

This routine uninitializes the global usb target resource.Before this routine is called, please make sure all the TCD and function driver has been removed and no one will use them.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

Name

usbTgtInit( ) - initialize the usb target stack

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtInit
    (
    void
    )
```

Description

This routine initializes the usb target stack.It will initialize one global TCD list **gUsbTgtTcdList** and one function driver global list **gUsbTgtFuncDriverList** which used to record the added TCD and function driver which installed in the system.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgt

12. USBTGTCONTROLREQUEST

Name

usbTgtControlRequest - The Control Request Process Module

Layer

USB_TARGET

Routines

usbTgtSubmitControlErp( ) - submits the control ERP **usbTgtInitSetupErp( )** - initialize setup ERP **usbTgtControlResponseSend( )** - send data to host on the control pipe **usbTgtControlPayloadRcv( )** - receive data on the default control pipe **usbTgtControlStatusSend( )** - send control transfer status to the host

Description

The file is the control request process module which include the standard requests, class requests and vendor requests process interfaces. The standard requests will be processed in this module, and the class requests and vendor requests will be routed to and processed by its function driver.

In chapter 9.4 of USB 2.0 Specification says: "USB devices must respond to standard device requests, even if the device has not yet been assigned an address or has not been configured."

All USB devices must respond to a variety of requests called standard requests. These requests are used for configuring a device, controlling the state of its interfaces, the status of its endpoints, the OTG status(if the device is OTG capable), and with other miscellaneous features. All the requests are issued by the host using the control transfer(Endpoint zero). Before the device configured, the device use the default address of zero to responce the host standard request until on unique address be set for the device during the configuration.

Additionally, devices may also support class-specific requests. And such kinds of requests will be processed by the corresponding function driver. Similarly, a device may support vendor specific requests that pertain to a given vendor's implementation.

Control transfers, which be used to transmit device requests, consist minimally of a setup stage and a status stage, but may also include a data stage, depending on the type of request being performed.

All the standard requests listed in chapter 9.4 list in the USB 2.0 specification included in this module.

This file created based on the GEN1 target stack, **usbTargDefaultPipe.c**

Control requests

Followed lists all the control request which can be handled by this module.

1.standard request

Following are the standard request in USB 2.0 specification. Some requests in USB 3.0 not list in this module.

1.1 **USB_REQ_CLEAR_FEATURE**

Clear feature request

1.2 **USB_REQ_SET_FEATURE**

Set feature request

1.3 **USB_REQ_GET_CONFIGURATION**

Get configuration request

1.4 **USB_REQ_SET_CONFIGURATION**

Set configuration request

1.5 **USB_REQ_GET_DESCRIPTOR**

Get descriptor request

1.6 **USB_REQ_SET_DESCRIPTOR**

Set descriptor request

1.7 **USB_REQ_SET_INTERFACE**

Set interface request

1.8 **USB_REQ_GET_INTERFACE**

Get interface request

1.9 **USB_REQ_GET_STATUS**

Get status request

1.10 **USB_REQ_SET_ADDRESS**

Set address request

1.11 **USB_REQ_GET_SYNCH_FRAME**

Get synch frame number request

2.vendor/class request

This module do not process the vendor/Class requests. It will parse this request to the function drivers to process it.

Control pipe

The control pipe is used for the control endpoint to process the control requests. It will be created/re-created when the device be RESET by the BUS. The USB target stack can create the pipe according to the hardware's capability(such as the USB target controller's speed).

Control erp

The control ERP is used for the control pipe to carry the information for the control endpoint on the SETUP/DATA/STATUS stage.

1.control erp initialization

Before using the control pipe, the control ERP need be initialized correctly. There are several APIs can be used.

1.1 usbTgtInitSetupErp
Initialize the control ERP for the SETUP stage.

1.2 usbTgtInitDataErp
Initialize the control ERP for the DATA IN/OUT stage.

1.3 usbTgtInitStatusErp
Initialize the control ERP for the STATUS IN/OUT stage.

1.4 usbTgtControlResponseSend
Initialize the control ERP for the DATA IN stage, which is an interface to the function drivers.

1.5 usbTgtControlPayloadRcv
Initialize the control ERP for the DATA OUT stage, which is an interface to the function drivers.

1.6 usbTgtControlStatusSend
Initialize the control ERP for the DATA OUT stage, which is an interface to the function drivers.

2.control erp callback

NOTE: The ERP will be called the targCallback by the TCD, and in the targCallback routine, if **userCallback** is valid, the user's userCallback will be called too to inform the user the ERP has completed.

For different kind of TCD hardware specific, the ERP callback with different return result.

2.1 **S_usbTgtTcdLib_ERP_SUCCESS**
Indicate the ERP complete one stage successfully, and next stage need to be processed. the developer need process next stage according to the SETUP - DATA (If needed) - STATUS by schedule. This case used in many normal case, such as the TCD need some special command to specific the status stage. Meanwhile, the result will be used when the DATA stage is needed.

2.2 **S_usbTgtTcdLib_ERP_RESTART**
Indicate the ERP complete with _no_ error, and next SETUP stage need to be processed. The developer do _not_ need process next stage according to the SETUP - DATA (If needed) - STATUS by schedule. This case used in some TCD. Especially for some TCD can send the STATUS with the DATA IN/OUT. In this case, the STATUS stage will no needed, and next SETUP stage should be specific.
This result can also used to termite one round of transfer. Sometimes the device may receive the termite signal from the host to stop the following data transaction.

2.3 **S_usbTgtTcdLib_ERP_CANCELED**
Indicate the ERP complete by cancelled error. It normally accord when the control pipe was deleted or the ERP be cancelled by the up level. Process will be terminated if the canceled result received.

2.4 Others
Indicate the ERP complete by other error.

Include files

vxWorks.h usb/usb.h usb/usbDescrCopyLib.h usb/usbTgt.h , usb/usbOtg.h usbTgtControlRequest.h

Name

usbTgtSubmitControlErp( ) - submits the control ERP

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtSubmitControlErp
    (
    pUSBTGT_TCD pTcd   /* The TCD pointer */
    )
```

Description

This routine is used to submit the Endpoint Request Packet(ERP) over the default control pipes. If an error occurs, the control pipes are stalled.

Returns

OK or **ERROR** if not able to submit the ERP

Errno

N/A

See also

usbTgtControlRequest

Name

usbTgtInitSetupErp( ) - initialize setup ERP

Layer

USB_TARGET

Synopsis

```
VOID usbTgtInitSetupErp
    (
    pUSBTGT_TCD pTcd   /* The TCD pointer */
    )
```

Description

This routine is used to initialize the Setup ERP for the setup stage transfer one the control pipe of pTcd.

Returns

N/A

Errno

N/A

See also

usbTgtControlRequest

Name

usbTgtControlResponseSend( ) - send data to host on the control pipe

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtControlResponseSend
    (
    USB_TARG_CHANNEL targChannel,   /* Target channel */
    UINT16           bfrLen,        /* Length of response 0 */
    pUINT8           pBfr,          /* Pointer to the data buffer */
    USB_ERP_FLAGS    erpFlags       /* The ERP transaction flags */
    )
```

Description

This routine sends data to host on the control pipe. It is used by the function drivers which need send data to the host by the control endpoint zero. It can be used to response the standard or unstandard requests.

Note

targChannel should be indicate to the pUSBTGT_FUNC_DRIVER pointer **bfrLen** is the length of the data to be send **pBfr** is the pointer of the buffer to store the data **erpFlags** indicates the ERP transaction flags, such as whether one ZLP pecket will be needed after all the data transfered.

Returns

OK or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgtControlRequest

Name

usbTgtControlPayloadRcv( ) - receive data on the default control pipe

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtControlPayloadRcv
    (
    USB_TARG_CHANNEL targChannel,   /* Target channel */
    UINT16           bfrLen,        /* Length of data to be received */
    pUINT8           pBfr,          /* Pointer to buffer */
    ERP_CALLBACK     userCallback   /* USB Target Applcaition Callback */
    )
```

Description

This routine receives data on the default control pipe from the host. It is used by the function drivers which need receive data from the host by the control endpoint zero. It can be used to response the standard or unstandard requests.

Note

targChannel should be indicate to the pUSBTGT_FUNC_DRIVER pointer bfrLen is the length of the data to be recived pBfr is the pointer of the buffer to store the data userCallback is the callback routine which will be called after all the data transfered.

Returns

OK or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgtControlRequest

Name

usbTgtControlStatusSend( ) - send control transfer status to the host

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtControlStatusSend
    (
    USB_TARG_CHANNEL targChannel,   /* Target channel */
    BOOL             bStatusIn      /* Is status in */
    )
```

Description

This routine is used to issue the status stage to the host. It will be called by the function drivers, and used in some normal case which is for some hardware controller to specific the STATUS stage. bStatusIn is the parameter to indicate the direction STATUS IN/OUT.

Note

targChannel should be indicate to the pUSBTGT_FUNC_DRIVER pointer

bStatusIn indicates the direction of the STATUS stage , STATUS IN/OUT

Returns

OK or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgtControlRequest

13. USBTGTFUNC

Name

usbTgtFunc - USB Target Function Class Driver Common Module

Layer

USB_TARGET

Routines

usbTgtFuncInterfaceDescFind( ) - find interface descriptor pointer **usbTgtFuncEndpointDescFind( )** - find endpoint descriptor pointer **usbTgtFuncFindPipeHandleByEpIndex( )** - find the pipe handle by the endpoint index **usbTgtFuncInfoMemberGet( )** - get the information member value of function driver **usbTgtFuncInfoMemberSet( )** - set the information member value of function driver **usbTgtFuncDefaultPipeStall( )** - stall the function driver's control pipe

Description

This mode provides common interfaces for the function drivers.

Include files

usb/usbTgt.h, **usbTgtFunc.h**, **usbTgtControlRequest.h**

Name

usbTgtFuncInterfaceDescFind( ) - find interface descriptor pointer

Layer

USB_TARGET

Synopsis

```
pUSB_INTERFACE_DESCR usbTgtFuncInterfaceDescFind
    (
    USB_TARG_CHANNEL targChannel,     /* targChannel */
    UINT8            uInterfaceNum,   /* Interface number        */
    UINT8            uAltSetting      /* Alternate setting index */
    )
```

Description

This routine is used to find the interface descriptor pointer with the right interface number and altsetting value.

Note

This routine mainly used after the function driver has been attached to the right TCD.

From the USB 2.0 Spec: If there are no class specific or vendor specific descriptors following this interface descriptor, all the endpoint descriptors will immediately follow the interface descriptor. The caller can get all the information of the endpoints

belong to the interface. There may be other class specific or vendor specific descriptors following to the interface descripor if the function driver defined.

Returns

NULL or the pointer of pUSB_INTERFACE_DESCR

Errno

N/A

See also

usbTgtFunc

Name

usbTgtFuncEndpointDescFind( ) - find endpoint descriptor pointer

Layer

USB_TARGET

Synopsis

```
pUSB_ENDPOINT_DESCR usbTgtFuncEndpointDescFind
    (
    USB_TARG_CHANNEL targChannel,     /* targChannel */
    UINT8            uInterfaceNum,   /* Interface number */
    UINT8            uAltSetting,     /* Alternate setting index */
    UINT8            uEpType,         /* Endpoint type */
    UINT8            uEpDir,          /* Endpoint dir */
    UINT8            uSyncType,       /* Synchronisation type */
    UINT8            uUsageType,      /* Usage Type for Iso/Int EP */
    UINT8            uNth             /* The Index of such endpoints, 1 based */
    )
```

Description

This routine is used to find the endpoint descriptor pointer with the given attributes of the right interface.

Note

This routine mainly used after the function driver has been attached to the right TCD.

From the USB 2.0 Spec: If there are no class specific or vendor specific descriptors following this interface descriptor, all the endpoint descriptors will immediately follow the interface descriptor. The caller can get all the information of the endpoints belong to the interface. There may be other class specific or vendor specific descriptors following to the interface descripor if the function driver defined.

Returns

NULL or the pointer of pUSB_ENDPOINT_DESCR

Errno

N/A

See also

usbTgtFunc

Name

usbTgtFuncFindPipeHandleByEpIndex( ) - find the pipe handle by the endpoint index

Layer

USB_TARGET

Synopsis

```
USB_TARG_PIPE usbTgtFuncFindPipeHandleByEpIndex
    (
    USB_TARG_CHANNEL targChannel,
    UINT8            uEpIndex
    )
```

Description

This routine finds the pipe **pUSBTGT_PIPE** from the function driver's usbTgtPipe list by the endpoint index.

Note

the uEpIndex includes the endpoint direction and endpoint address information, such as the endpoint address of endpoint descripor.

Returns

USB_TARG_PIPE indicate one valid pipe or **NULL** if no found

Errno

N/A

See also

usbTgtFunc

Name

usbTgtFuncInfoMemberGet( ) - get the information member value of function driver

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncInfoMemberGet
    (
    USB_TARG_CHANNEL targChannel,
    UINT16           uMemberType,
    void *           pContext
    )
```

Description

This routine is used to get the information member value of function driver.

Returns

always **OK**

Errno

N/A

See also

usbTgtFunc

Name

usbTgtFuncInfoMemberSet( ) - set the information member value of function driver

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncInfoMemberSet
    (
    USB_TARG_CHANNEL targChannel,
    UINT16           uMemberType,
    void *           pContext
    )
```

Description

This routine is used to set the information member value of function driver.

Returns

always **OK**

Errno

N/A

See also

usbTgtFunc

Name

usbTgtFuncDefaultPipeStall( ) - stall the function driver's control pipe

Layer

USB_TARGET

Synopsis

```
STATUS usbTgtFuncDefaultPipeStall
    (
    USB_TARG_CHANNEL targChannel
    )
```

Description

This routine is used to stall the function driver's control pipe. It will be used when someting wrong on the control pipe when process some class/vendor requests.

Returns

OK, or **ERROR** if there is something wrong

Errno

N/A

See also

usbTgtFunc

14. USBTGTUTIL

Name

usbTgtUtil - USB Taget Stack Utility Module

Layer

USB_TARGET

Routines

usbTgtFindPipeByEpIndex( ) - find the pipe pointer by the endpoint index. **usbTgtFindFuncByEpIndex( )** - find the function driver by the endpoint index. **usbTgtFindFuncByInterface( )** - find the function driver by the interface number.

Description

This module defines the internal utlity routines used by the usb target stack only.

Includes

usbTgtUtil.h

Name

usbTgtFindPipeByEpIndex( ) - find the pipe pointer by the endpoint index.

Layer

USB_TARGET

Synopsis

```
pUSBTGT_PIPE usbTgtFindPipeByEpIndex
    (
    pUSBTGT_FUNC_DRIVER pFuncDriver,
    UINT8               uEpIndex
    )
```

Description

This routine finds the pipe **pUSBTGT_PIPE** from the function driver's usbTgtPipe list by the endpoint index.

Note

the uEpIndex includes the endpoint direction and endpoint address information, such as the endpoint address of endpoint descripor.

Returns

pointer to the pipe or **NULL** if no found

Errno

N/A

See also

usbTgtUtil

Name

usbTgtFindFuncByEpIndex( ) - find the function driver by the endpoint index.

Layer

USB_TARGET

Synopsis

```
pUSBTGT_FUNC_DRIVER usbTgtFindFuncByEpIndex
    (
    pUSBTGT_TCD pTcd,
    UINT8       uEpIndex
    )
```

Description

This routine finds the function driver which occupys endpoint who has the endpoint index.

Note

the uEpIndex includes the endpoint direction and endpoint address information.

Returns

pointer to the funtion driver or **NULL** if no found

Errno

N/A

See also

usbTgtUtil

Name

usbTgtFindFuncByInterface( ) - find the function driver by the interface number.

Layer

USB_TARGET

Synopsis

```
pUSBTGT_FUNC_DRIVER usbTgtFindFuncByInterface
    (
    pUSBTGT_TCD pTcd,
    UINT8       uInterface
    )
```

Description

This routine finds the function driver who occupys the interface number from the the function driver list of **pUSBTGT_TCD**.

Returns

pointer to the funtion driver or **NULL** if no found

Errno

N/A

See also

usbTgtUtil

15. VXBFDTUSBAM335XCTRL

Name

vxbFdtUsbAm335xCtrl - VxBus AM335X USB Control module FDT Driver

Layer

USB_PHY

Routines

No user-callable routines

Description

This module is the driver to support AM335X USB Control module.

Each USB PHY device should be bound to a device tree node which requires below properties:

compatible

This property specifies the name of the USB controller driver.

reg

Offset and length of the "USB control registers" in the "Control Module" block. A second offset and length for the USB wake up control in the same memory block.

reg-names

"phy_ctrl" for the "USB control registers" and "wakeup" for the USB wake up control register.

Below is an example:

```
            usb_ctrl_mod: control@44e10620
                {
                compatible = "ti,am335x-usb-ctrl-module";
                reg = <0x44e10620 0x10
                       0x44e10648 0x4>;
                reg-names = "phy_ctrl", "wakeup";
                status = "disabled";
                };
```

Include files

vxWorks.h subsys/timer/**vxbTimerLib**. **hwif/buslib/vxbFdtLib.h**, **vxbFdtUsbAm335xCtrl.h**

16. VXBFDTUSBPHYALTSOCGEN5

Name

vxbFdtUsbPhyAltSocGen5 - VxBus Gen2 Altera SOC Gen5 USB PHY FDT Driver

Layer

USB_PHY

Routines

No user-callable routines

Description

This module is the driver to support Altera SOC Gen5 USB PHY Device.

To add the driver to the vxWorks image, add the following component to the kernel configuration:

```
vxprj component add INCLUDE_USB_PHY_ALT5
```

The name of each function in this group begins with "usb" to represent "Device Driver Services."

External interface

The driver provides a vxBus gen2 external interface. Initialization methods provided by **vxbFdtUsbPhyAltSocGen5Drv** register the driver with VxBus. The BSP's dts entry must specify the following parameters:

compatible

Specify the programming model for the controller. It should be set to "altr,usb-phy" and is used by vxBus gen2 for device driver selection.

reg

Specifies the base address where the controller's CSR registers are mapped into the host's address space. All register offsets are computed relative to this address.

phy_type

Specifies the PHY's interface type. Available types are "ulpi", "utmi", "utmi_wide" and "serial".

Below is an example:

```
        usbphy1: usbphy1@FFB40000
            {
            compatible = "altr,usb-phy";
            reg = <0xFFB40000 0x00001000>;
            phy_type = "ulpi";
            };
```

Include files

vxWorks.h vxbTimerLib.h vxbFdtUsbPhyAltSocGen5.h

See also

usb

17. VXBFDTUSBPHYAM335X

Name

vxbFdtUsbPhyAm335x - VxBus Gen2 AM335X On-Chip USB PHY FDT Driver

Layer

USB_PHY

Routines

No user-callable routines

Description

This module is the driver to support TI AM335X USB PHY Device.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add INCLUDE_USB_PHY_AM335X
```

Each USB PHY device should be bound to a device tree node which requires below properties:

compatible

This property specifies the name of the USB controller driver.

phy_type

This property specifies the PHY's interface type.

Below is an example:

```
            usbphy1: phy@47401b00
                {
                compatible = "am335x,usb-phy";
                phy_type = "ulpi";
                };
```

Include files

errno.h vxbUsbPhyLib.h vxbFdtLib.h vxbTimerLib.h usbOsalDebug.h

18. VXBFDTUSBPHYFSLONCHIP

Name

vxbFdtUsbPhyFslOnChip - VxBus Gen2 FSL On-Chip USB PHY FDT Driver

Layer

USB_PHY

Routines

usbPhyHwInitImx( ) - USB PHY hardware initialization for Imx **usbPhyHwInitVybrid( )** - USB PHY hardware initialization for Vybrid

Description

This module is the driver to support Freescale MXS USB PHY Device.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add INCLUDE_USB_PHY_FSL_ONCHIP
```

Each USB PHY device should be bound to a device tree node which requires below properties:

compatible

This property specifies the name of the USB controller driver.

clocks

This parameter specifies the list of clocks that this module use.

clock-names

This parameter specifies the clock name that this module use.

reg

This property specifies the register base address and length of this module.

phy_type

This property specifies the PHY's interface type.

Below is an example:

```
        usbphy1: usbphy@20ca000
            {
            compatible = "fsl,imx-on-chip-phy";
            platform = "fsl-imx";
            clocks = <&clk 7>;
            clock-names = "pll7Usb2";
            reg = <0x20ca000 0x90 0x20c8200 0x60>;
            pinmux-0 = <&usb_otg_2>;
            gpio-binds = <&gpio0 13 1 1 10>;
```

```
            phy_type = "utmi";
            };
```

Include files

vxbUsbPhyLib.h vxbFdtLib.h vxbClkLib.h vxbPinMuxLib.h, vxbGpioLib.h usbOsal.h usbOsalDebug.h

Name

usbPhyHwInitImx( ) - USB PHY hardware initialization for Imx

Layer

USB_PHY

Synopsis

```
VOID usbPhyHwInitImx
    (
    pFSL_ONCHIP_PHY_CTLR pDevCtlr
    )
```

Description

This routine is used to initialize USB phy hardware.

Return

N/A

Returns

Not Available

Errno

Not Available

See also

vxbFdtUsbPhyFslOnChip

Name

usbPhyHwInitVybrid( ) - USB PHY hardware initialization for Vybrid

Layer

USB_PHY

Synopsis

```
VOID usbPhyHwInitVybrid
    (
    pFSL_ONCHIP_PHY_CTLR pDevCtlr
    )
```

Description

This routine is used to initialize USB phy hardware.

Return

N/A

Returns

Not Available

Errno

Not Available

See also

vxbFdtUsbPhyFslOnChip

19. VXBFDTUSBPHYNXPIMX8M

Name

vxbFdtUsbPhyNxpImx8m - VxBus Gen2 NXP i.MX8M USB PHY FDT Driver

Layer

USB_PHY

Routines

No user-callable routines

Description

This is the vxBus compliant NXP i.MX8M USB PHY driver.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add INCLUDE_USB_PHY_NXP_IMX8M
```

Each USB PHY device should be bound to a device tree node which requires below properties:

compatible

This parameter specifies the name of the USB phy driver. It should be "nxp,imx8m-usb-phy".

reg

This parameter specifies the register base address and length of this module.

interrupt-parent

This parameter specifies the offset of interrupt controller.

clocks

This parameter specifies the list of clocks that this module use.

clock-names

This parameter specifies the clock name that this module use.

Below is an example:

```
    usb2_phy: phy@382f0040
        {
        compatible = "nxp,imx8m-usb-phy";
        reg = <0x0 0x382f0040 0x0 0x40>;
        clocks = <&usb2_phy_clk>;
        clock-names = "usb2_phy_clk";
        };
```

Include files

usbOsal.h vxbUsbPhyLib.h

See also

usb

20. VXBFDTUSBPHYTIAM572X

Name

vxbFdtUsbPhyTiAm572x - VxBus Gen2 TI AM572X USB PHY FDT Driver

Layer

USB_PHY

Routines

No user-callable routines

Description

This module is the driver to support USB PHY controllers on TI AM572X board.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add INCLUDE_USB_PHY_TI_AM572X
```

Each USB PHY device should be bound to a device tree node which requires below properties:

compatible

This property specifies the name of the USB controller driver.

clocks

This parameter specifies the list of clocks that this module use.

clock-names

This parameter specifies the clock name that this module use.

reg

This property specifies the register base address and length of this module.

phy_type

This property specifies the PHY's interface type.

Below is an example:

```
        usb1phy: usb1phy@4a085000
            {
            compatible = "ti,am572x-usb2-phy2";
            platform   = "ti-sitara";
            reg        = <0x4a085000 0x100>,
                         <0x4a002e74 0x04>;
            phy_type   = "utmi";
            clocks     = <&usb_phy2_always_on_clk32k>,
                         <&usb_otg_ss2_refclk960m>;
            clock-names= "usb_phy2_always_on_clk32k",
```

```
                        "usb_otg_ss2_refclk960m";
            };
```

Include files

vxbUsbPhyLib.h vxbFdtLib.h vxbClkLib.h vxbPinMuxLib.h, **usbOsalDebug.h**

21. VXBFDTUSBPHYULPIVIEWPORT

Name

vxbFdtUsbPhyUlpiViewPort - VxBus Gen2 ULPI ViewPort PHY FDT Driver

Layer

USB_PHY

Routines

No user-callable routines

Description

This module is the driver to support other ULPI PHY controllers over ULPI VIEWPORT register such as build in Freescale SOC USB module.

To add the driver to the vxWorks image, add the following component to the kernel configuration.

```
vxprj component add INCLUDE_USB_PHY_ULPI_VIEWPORT
```

Each USB PHY device should be bound to a device tree node which requires below properties:

compatible

This property specifies the name of the USB controller driver.

reg

This property specifies the register base address and length of this module.

phy_type

This property specifies the PHY's interface type. It should be "ulpi" for this driver.

Below is an example:

```
        usbphy:usbphy@22170 {
            compatible = "fsl,ulpi-view-phy";
            reg = <0x22170 0x4>;
            gpio-binds = <&gpio0 5 1 1 10>;
            phy_type = "ulpi";
        };
```

Include files

vxbUsbPhyLib.h vxbFdtLib.h vxbClkLib.h vxbPinMuxLib.h, **vxbGpioLib.h usbOsalDebug.h**

See also

usb

22. VXBUSBBUS

Name

vxbUsbBus - USB Bus Vxbus Type Interface

Layer

USB

Routines

No user-callable routines

Description

This module provides the definition of USB Bus type.

Include files

vxWorks.h hwif/vxBus.h